Hostel Hive

System Design Specification

By

Hasinthaka Piyumal

SE/2021/036

Navon Sanjuni

SE/2021/061

Thagya Kavindi

SE/2021/062

Tasheen Darshika

SE/2021/030

A report submitted in partial fulfillment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2021

Version No	Date	Author(s)	Summary of Changes
1.0	2026/04/19	Hasinthaka, Sanjuni, Thagya, Tasheen	Initial draft of the Software Design Specification.

Table 1: Version History

Table of Contents

1. Architectural Design

1.1 High-Level Architecture 1

1.2 Cloud-Native Architecture 2

1.3 Justification for Architecture Choice 4

1.4 Scalability and Maintainability Considerations 5

1.4.1 Scalability 6

1.4.2 Maintainability 6

2. Database Design

2.1 ER Diagram ……8

2.2 Normalization explanation ……8

2.2.1 Normal Forms Applied ……9

2.2.2 From Unnormalized Form (UNF) to 1NF ……..10

2.2.3 Second Normal Form (2NF) ……...13

2.2.4 Third Normal Form (3NF) — Transitive Dependency Analysis ……16

2.2.5 Boyce-Codd Normal Form (BCNF) ……18

2.2.6 Anomaly Elimination ……..21

2.2.7 Normalized Schema Summary ……22

3. Class Diagram ……..28

4. Sequence Diagrams 29

4.1 Login /Authentication 29

4.2 Reset Password 30

4.3 Manage Student Profiles 31

4.4 Bulk Import Students (CSV) 32

4.5 Scan QR Code 33

4.6 Verify Student Identity 34

4.7 Log Entry/Exit 35

4.8 Detect Unapproved Late Entry 36

4.9 Record Leave Status 37

4.10 Request Late Entry/Exit 38

4.11 Approve/Reject Late Access Request 39

4.12 Submit Complaint 40
4.13 Attach Images 41
4.14 Receive Notifications 42
4.15 Monitor & View Attendance Dashboard 43
4.16 Resolve Maintenance Issues 44
4.17 Generate Analytics & Reports 45
4.18 Allocate Room/Bed 46
4.19 Monitor System Display 47
4.20 Maintain Physical Records 48

5. **UI/UX Design** 49
6. **Security Design** 67
   6.1 Authentication and Authorization Flow 67
   6.2 Full Security Components 68
   6.3 Hardware and Session Security 69
7. **Deployment Design** 70
   7.1 Hosting plan 70
   7.2 Hardware requirements 72
      7.2.1 Server Hardware (VPS) 73
      7.2.2 On-Premises Hardware 74
   7.3 Network diagram 75
   7.4 Environment specification 77
   7.5 CI/CD pipeline overview 79
      7.5.1 Pipeline Stages 80
      7.5.2 Pipeline Implementation Details 80
      7.5.3 IoT Deployment Consideration 82

List of Tables

Table 1: Version History ii

Table 2: Normal Forms Applied 9

Table 3: From Unnormalized Form (UNF) to 1NF Table 10

Table 4: Second Normal Form (2NF) 13

Table 5: Third Normal Form (3NF) — Transitive Dependency Analysis 16

Table 6: Boyce-Codd Normal Form (BCNF) 18

Table 7: Anomaly Elimination 21

Table 8: Normalized Schema Summary 22

Table 9: Hosting Plan Summary 71

Table 10: Production Server Minimum Specifications 73

Table 11: On-Premises Hardware Requirements 74

Table 12: Environment Specifications 78

Table 13: CI/CD Pipeline Specifications 80

List of Figures

Figure 1: High-Level Layered Architecture of HostelHive 1

Figure 2: Cloud-Native Architecture 3

Figure 3: ER Diagram 8

Figure 4: Class Diagram 28

Figure 5: Login /Authentication 29

Figure 6: Reset Password 30

Figure 7: Manage Student Profiles 31

Figure 8: Bulk Import Students (CSV) 32

Figure 9: Scan QR Code 33

Figure 10: Verify Student Identity 34

Figure 11: Log Entry/Exit 35

Figure 12: Detect Unapproved Late Entry 36

Figure 13: Record Leave Status 37

Figure 14: Request Late Entry/Exit 38

Figure 15: Approve/Reject Late Access Request 39

Figure 16: Submit Complaint 40

Figure 17: Attach Images 41

Figure 18: Receive Notifications 42

Figure 19: Monitor & View Attendance Dashboard 43

Figure 20: Resolve Maintenance Issues 44

Figure 21: Generate Analytics & Reports 45

Figure 22: Allocate Room/Bed 46

Figure 23: Monitor System Display 47

Figure 24: Maintain Physical Record 48

Figure 25: Security staff - user flow 63

Figure 26: Student mobile app user flow 64

Figure 27: Warden/Sub warden - web user flow 65

Figure 28: Administrator - web user flow 66

Figure 29: Authentication and Authorisation Flow 67

Figure 30: Full Security Components 68

Figure 31: Hardware and Session Security 69

Figure 32: Hosting plan 70

Figure 33: Hardware Requirements 72

Figure 34: Network Diagram 76

Figure 35: HostelHive Environment Specification Overview 78

Figure 36: HostelHive CI/CD Pipeline Overview (GitHub Actions) 80

CHAPTER 1: ARCHITECTURAL DESIGN

1.1 High-Level Architecture

A layered (N-tier) monolithic architecture is adopted for HostelHive. The system is structured into five layers: external actors, presentation, application, service, and data. Communication is restricted to adjacent layers to ensure modularity and maintainability. The overall architecture is illustrated in Figure 1.

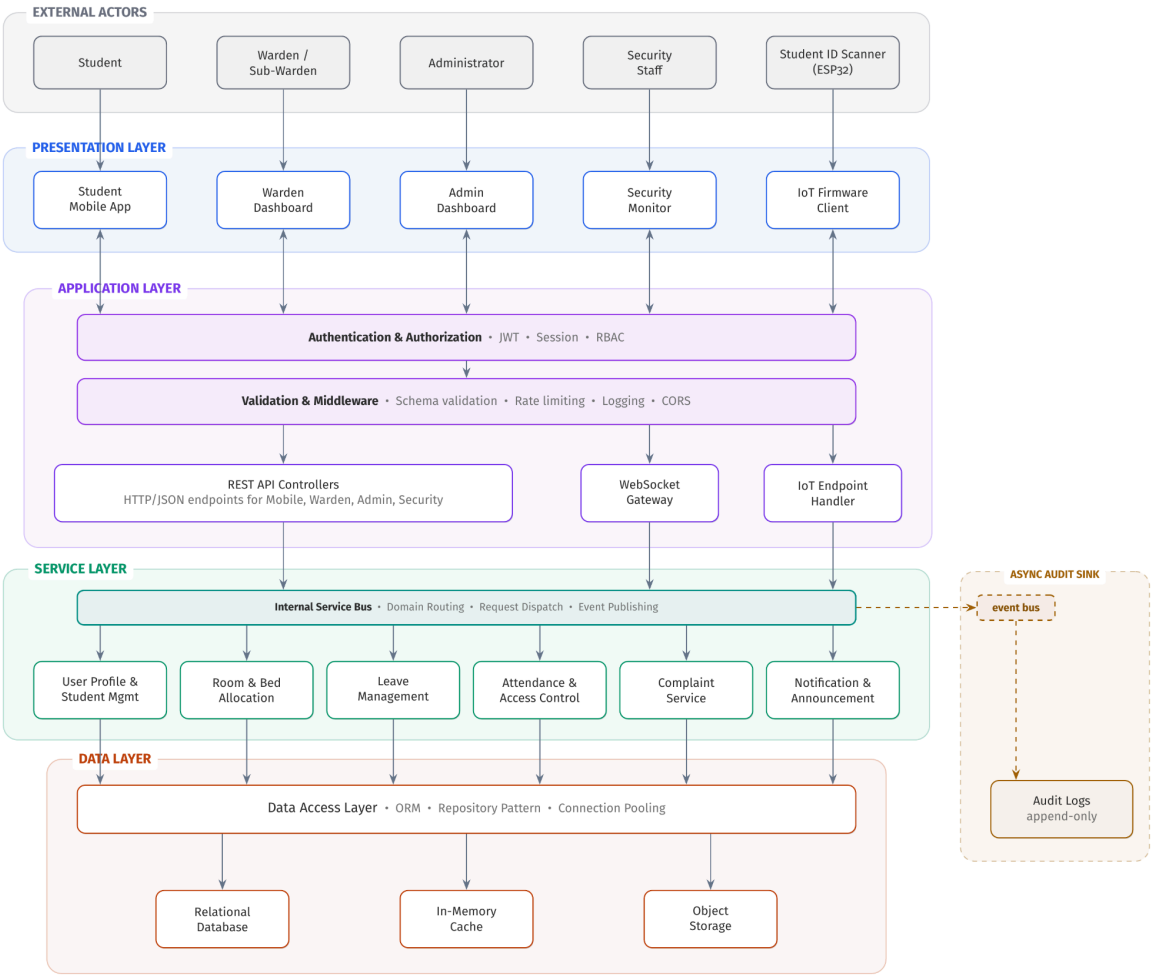


Figure 1: High-Level Layered Architecture of HostelHive

Five external actors are identified: Student (hostel resident), Warden/Sub-Warden (management staff), Administrator (system oversight and late-entry approval), Security Staff (monitoring gate activity and QR scan events), and the ESP32 QR Scanner (IoT device installed at the hostel gate).

User interfaces are provided by the Presentation Layer, including the Student Mobile Application, Warden Dashboard, Admin Dashboard, Security Monitor, and IoT Firmware Client.

Request handling and routing are performed by the Application Layer. Authentication and authorization mechanisms are enforced, and validation and middleware processing are applied. REST API controllers are utilized for standard operations, while real-time communication is enabled through a WebSocket Gateway. Data from the ESP32 scanner is received via an IoT endpoint handler.

Business logic is encapsulated within the Service Layer and is divided into six functional domains: user and student management, room and bed allocation, leave management, attendance and access control, complaint management, and notifications.

Data management is handled by the Data Layer through a repository-based data access layer implemented over an ORM. Data storage is distributed across a relational database, in-memory cache, object storage, and audit logging systems.

1.2 Cloud-Native Architecture

A cloud-based deployment model is implemented using a single application server supported by managed services. The architecture is designed to support a fixed user base of approximately 500 students while maintaining cost efficiency and simplicity. The deployment structure is shown in Figure 2.

The deployment stack is as follows:

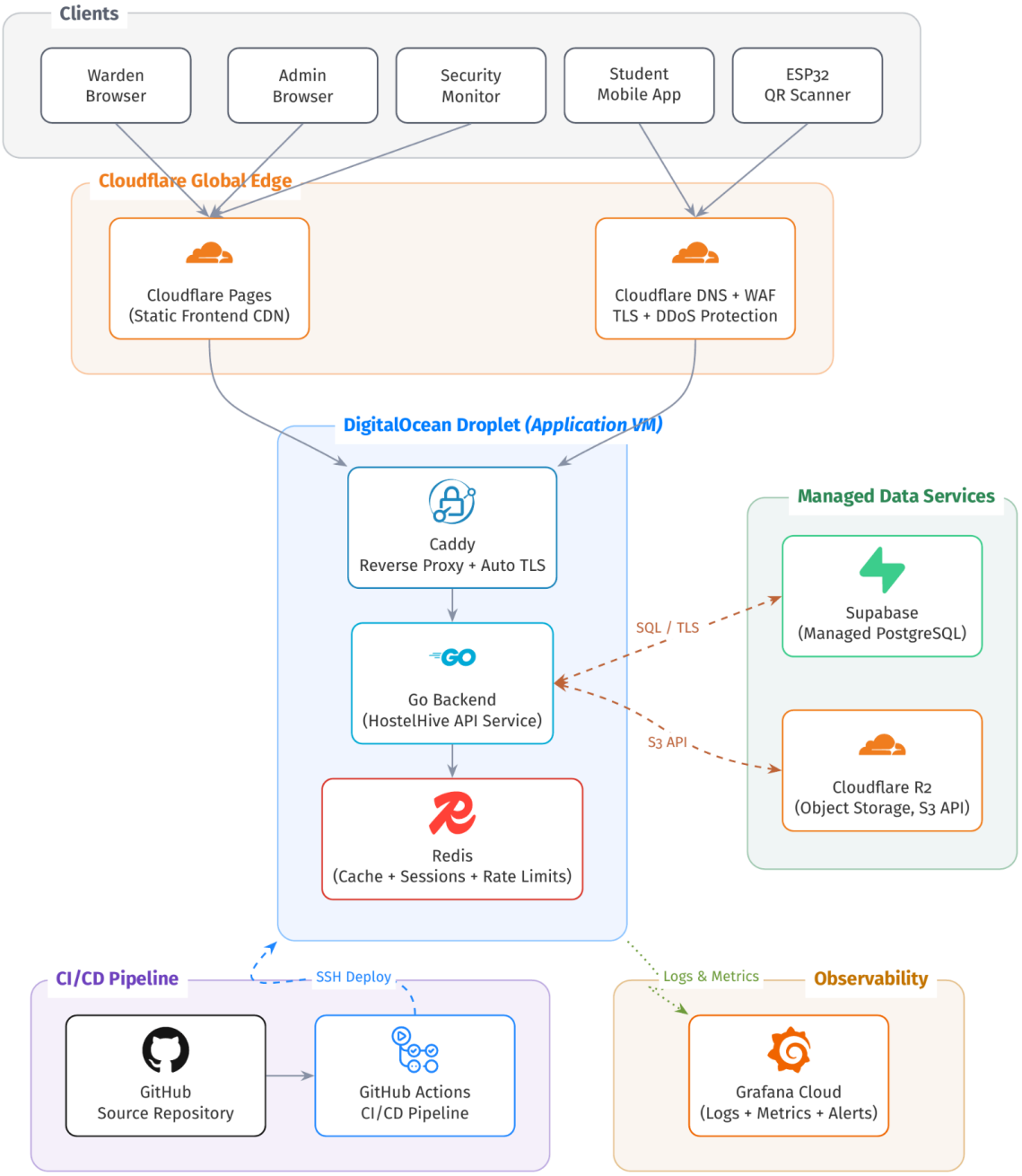


Figure 2: Cloud-Native Architecture

- A DigitalOcean Droplet is used as a single virtual machine on which the Go backend application and Redis cache are hosted. By hosting the application on a single VM, system management is simplified and operational costs are kept predictable.
- Caddy is deployed on the virtual machine as a reverse proxy. TLS certificate management is handled automatically, and all incoming HTTPS traffic is routed to the Go backend without manual configuration.

- The Go backend is used as the main API server, in which all business logic, including leave management, room allocation, attendance tracking, complaint handling, and notification processing, is executed.
- Redis is deployed on the same virtual machine and is used for caching, session management, and rate limiting. Frequently accessed data is stored in memory so that unnecessary database queries are reduced.
- A fully managed PostgreSQL database is provided by Supabase. Database administration tasks such as backups, upgrades, and monitoring are handled externally, reducing operational complexity for the system.
- Cloudflare R2 is used for object storage, where files uploaded by users, such as complaint attachments and profile photos, are stored. The use of R2 ensures that no egress fees are incurred, thereby reducing operational costs.
- Static frontend applications for the Warden and Admin dashboards are served through Cloudflare Pages via a global content delivery network, ensuring fast access and complete separation from the backend server.
- Cloudflare DNS and Web Application Firewall (WAF) are used for domain routing, TLS termination at the edge, and protection against DDoS attacks and malicious traffic before requests reach the virtual machine.
- Continuous integration and deployment are handled through GitHub Actions. When code is pushed to the main branch, the Go application is automatically built and deployed to the virtual machine over SSH.
- System logs and performance metrics are collected and monitored through Grafana Cloud. Alerts are generated when system issues are detected, providing visibility into system health without requiring additional infrastructure.

1.3 Justification for Architecture Choice

- The layered monolithic architecture is selected based on the actual requirements of the system rather than trends or unnecessary complexity.
- A small and predictable user base is supported by the system. Approximately 500 students and a limited number of staff members are accommodated, and therefore the system load is fixed and well understood. As a result, the requirements of the system can be efficiently handled by a monolithic architecture without introducing additional complexity.

- The use of microservices is avoided because significant overhead would be introduced. Multiple deployments, inter-service communication, distributed tracing, and complex debugging processes would be required, which would increase development and maintenance difficulty for a relatively small system.
- Clear separation of concerns is achieved through the layered approach. Responsibilities are divided across presentation, application, service, and data layers, ensuring that each component has a well-defined role. As a result, development, debugging, and system understanding are simplified.
- System handover is facilitated by the use of a layered monolithic design. A single codebase and deployment environment are maintained, making it easier for future maintainers to understand and manage the system.
- The limitations of a basic MVC pattern are addressed by the inclusion of a dedicated service layer. Business logic is separated from controllers, resulting in improved code organization and easier testing as the system evolves.
- Operational overhead is reduced through the use of managed services. Infrastructure-related tasks such as database management, certificate renewal, DDoS protection, and file storage are handled externally, allowing focus to remain on application development.
- Deployment simplicity and reliability are achieved by using a single virtual machine and a statically compiled Go binary. The system can be deployed, restarted, and debugged easily without requiring container orchestration or distributed system management.

1.4 Scalability and Maintainability Considerations

Even though HostelHive is designed for a single hostel with a fixed user base, the architecture is structured in a way that allows the system to grow and remain easy to maintain over time.

1.4.1 Scalability

Vertical scaling of the application server is supported with minimal effort. The DigitalOcean Droplet can be resized to a higher configuration without requiring changes to the application code.

Database scalability is ensured through the use of Supabase, where scaling operations are managed through service configuration without manual intervention. Connection pooling is provided through PgBouncer.

System performance is improved through the use of Redis, where sessions and frequently accessed data are stored in memory. As user activity increases, load is absorbed by the cache before reaching the database.

All incoming traffic is routed through Cloudflare, where DDoS attacks, bot traffic, and unexpected spikes are mitigated at the edge before reaching the application server. Static frontend content is served entirely through Cloudflare Pages, ensuring that the backend server is not affected by frontend traffic.

Future scalability is supported by the layered architecture, in which individual service components can be separated into independent services if required.

Object storage scalability is ensured through Cloudflare R2, where storage capacity is managed independently and does not impact the application server.

1.4.2 Maintainability

Separation of service domains is maintained within the Service Layer. Each domain can be developed and modified independently, reducing the risk of unintended side effects.

Data access is standardized through the repository pattern, ensuring that database operations are centralized and consistent. Changes to the database schema can be managed within the data access layer without affecting other parts of the system.

A single statically compiled binary is produced by the Go backend, simplifying deployment and eliminating dependency-related issues in the production environment.

Deployment processes are automated through GitHub Actions, ensuring consistency and reducing reliance on manual steps.

System monitoring and observability are provided through Grafana Cloud, where logs and metrics are collected and analyzed. Alerts are generated when abnormal behavior is detected.

Long-term maintenance effort is minimized through the use of managed services such as Supabase and Cloudflare, where infrastructure management tasks are handled externally.

CHAPTER 2 : DATABASE DESIGN

2.1 ER Diagram

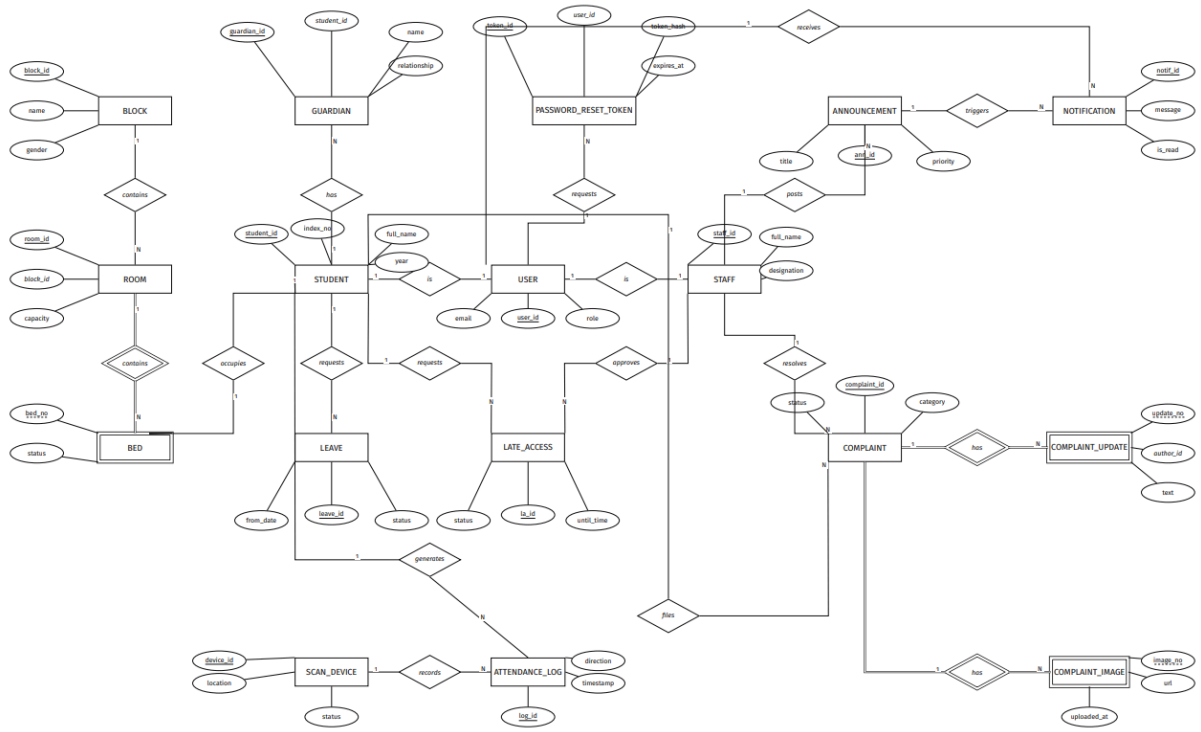


Figure 3: ER Diagram

2.2 Normalization Explanation

- It is demonstrated in this section that the HostelHive database schema satisfies Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF). The seventeen entities identified in the ER Diagram are covered in the analysis and are grounded in the data requirements defined in the SRS document.

2.2.1 Normal Forms Applied

Normal Form	Requirement	Status in HostelHive
1NF — First Normal Form	Every attribute is atomic (single-valued); no repeating groups; every row has a primary key.	Satisfied — All 17 entities use atomic attributes and a single-column primary key. Repeating groups (attendance logs, leave requests, late access requests, complaints, complaint updates, complaint images, notifications) are extracted into separate entities.
2NF — Second Normal Form	In 1NF; every non-key attribute is fully dependent on the entire primary key (no partial dependencies).	Satisfied — All 17 entities use single-column PKs, making partial dependencies structurally impossible. No M:N relationships exist that would require junction tables with composite keys.
3NF — Third Normal Form	In 2NF; no non-key attribute depends on another non-key attribute (no transitive dependencies).	Satisfied — Six transitive dependencies identified and resolved by extracting separate entities (BLOCK, ROOM, BED, USER, GUARDIAN, STAFF).

BCNF — Boyce-Codd Normal Form	For every functional dependency X → Y, X must be a superkey.	Satisfied — Since all non-key attributes depend on the primary key and all candidate keys are superkeys, BCNF is satisfied. Candidate keys (email in USER, index_no in STUDENT, token_hash in PASSWORD_RESET_TOKEN) are all superkeys.

Table 2: Normal Forms Applied

2.2.2 From Unnormalized Form (UNF) to 1NF

- Before normalization, a hypothetical flat HOSTEL_RECORD would combine all hostel data — student details, guardian information, room and bed details, staff data, attendance logs, leave requests, complaints, and announcements — into one wide row with nested repeating groups:
- HOSTEL_RECORD(student_id, index_no, full_name, year, email, guardian_name, guardian_relationship, block_name, block_gender, room_capacity, bed_no, bed_status, staff_name, staff_designation, {log_id, timestamp, direction}, {leave_id, from_date, status}, {la_id, status, until_time}, {complaint_id, category, status, {update_no, author_id, text}, {image_no, url, uploaded_at}}, announcement_title, announcement_priority, {notif_id, message, is_read}, token_hash, expires_at)
- 1NF is violated in nine ways by this UNF record. Each violation and the entity created to resolve it are shown below.

1NF Violation	Problem	ER Diagram Resolution	SRS Reference

Repeating attendance logs	A student row contains a list of entry/exit events, each with timestamp and direction.	Extracted into ATTENDANCE_LOG entity (PK: log_id). SCAN_DEVICE records ATTENDANCE_LOG (1:N).	FR011, FR012 , UC007
Repeating leave requests	Multiple leave requests per student, each with from_date and status, nested inside one row.	Extracted into LEAVE entity (PK: leave_id). STUDENT requests LEAVE (1:N).	FR021 , FR022 , UC009
Repeating late access requests	Multiple late access requests per student, each with status and until_time.	Extracted into LATE_ACCESS entity (PK: la_id). STUDENT requests LATE_ACCESS (1:N).	FR021 , UC010
Repeating complaints	Multiple complaints per student, each with category and status.	Extracted into COMPLAINT entity (PK: complaint_id). STUDENT files COMPLAINT (1:N).	FR025, FR026 , UC012
Repeating complaint updates	Multiple text updates per complaint, each with author_id and text,	Extracted into COMPLAINT_UPDATE entity (PK: update_no). COMPLAINT	FR027 , UC016

	nested inside one complaint row.	has COMPLAINT_UPDATE (1:N).	
Repeating complaint images	Multiple images per complaint, each with url and uploaded_at, nested inside one complaint row.	Extracted into COMPLAINT_IMAGE entity (PK: image_no). COMPLAINT has COMPLAINT_IMAGE (1:N).	UC013
Repeating notifications	Multiple notifications per user, each with message and is_read status.	Extracted into NOTIFICATION entity (PK: notif_id). USER receives NOTIFICATION (1:N).	FR032 , UC014
Guardian data repeated	guardian_name and guardian_relationship duplicated across every student row.	Extracted into GUARDIAN entity (PK: guardian_id). STUDENT has GUARDIAN (1:N).	FR007
Password reset token repeated	token_hash and expires_at stored as composite values inside the user row.	Extracted into PASSWORD_RESET_TOKEN entity (PK: token_id). USER has PASSWORD_RESET_TOKEN (1:N).	FR004 , UC002

Table 3: From Unnormalized Form (UNF) to 1NF Table

- After these extractions, every table contains only atomic, single-valued columns uniquely identified by its primary key. All 17 entities satisfy 1NF.

2.2.3 Second Normal Form (2NF)

- 2NF violations only arise when a table has a composite (multi-column) primary key. In the HostelHive schema, all 17 entities use a single-column primary key. Partial dependencies are therefore structurally impossible across the entire schema.

Entity	Primary Key	Primary Key Type	Partial Dependency Risk	2NF Status
BLOCK	block_id	Single column	None possible	Satisfied ✓
ROOM	room_id	Single column	None possible	Satisfied ✓
BED	bed_no	Single column	None possible	Satisfied ✓
STUDENT	student_id	Single column	None possible	Satisfied ✓

USER	user_id	Single column	None possible	Satisfied ✓
STAFF	staff_id	Single column	None possible	Satisfied ✓
GUARDIAN	guardian_id	Single column	None possible	Satisfied ✓
PASSWORD_RESET_TOKEN	token_id	Single column	None possible	Satisfied ✓
ANNOUNCEMENT	ann_id	Single column	None possible	Satisfied ✓
NOTIFICATION	notif_id	Single column	None possible	Satisfied ✓
LEAVE	leave_id	Single column	None possible	Satisfied ✓

LATE_ACCESS	la_id	Single column	None possible	Satisfied ✓
COMPLAINT	complaint_id	Single column	None possible	Satisfied ✓
COMPLAINT_UPDATE	update_no	Single column	None possible	Satisfied ✓
COMPLAINT_IMAGE	image_no	Single column	None possible	Satisfied ✓
SCAN_DEVICE	device_id	Single column	None possible	Satisfied ✓
ATTENDANCE_LOG	log_id	Single column	None possible	Satisfied ✓

Table 4: Second Normal Form (2NF)

- All entities use single-column PKs. No composite keys exist, therefore 2NF is fully satisfied across the entire schema.

2.2.4 Third Normal Form (3NF) — Transitive Dependency Analysis

- Six transitive dependencies were identified by examining the SRS data requirements and the ER diagram. Each was resolved by extracting a separate entity.

#	Transitive Dependency	Dependency Chain	Resolution	SRS Reference
1	Block attributes inside ROOM	room_id → block_id → name, gender	Extracted to BLOCK entity (PK: block_id). ROOM holds only block_id as FK. BLOCK contains ROOM (1:N). Block name and gender never duplicated in ROOM.	FR018
2	Room attributes inside BED	bed_no → room_id → capacity, block_id	Extracted to ROOM entity (PK: room_id). BED holds only room_id as FK. ROOM contains BED (1:N). Room capacity and block reference remain exclusively in ROOM.	FR018, FR019
3	User details inside STUDENT	student_id → user_id → role, email	Extracted to USER entity (PK: user_id). STUDENT holds only user_id as FK (is relationship). USER is STUDENT (1:1). User credentials never duplicated in STUDENT.	FR001

4	User details inside STAFF	staff_id → user_id → role, email	Extracted to USER entity (PK: user_id). STAFF holds only user_id as FK (is relationship). USER is STAFF (1:1). User credentials never duplicated in STAFF.	FR001, FR003
5	Guardian details inside STUDENT	student_id → guardian_id → name, relationship	Extracted to GUARDIAN entity (PK: guardian_id). STUDENT has GUARDIAN (1:N). Guardian name and relationship never stored inside STUDENT row.	FR007
6	Token details inside USER	user_id → token_id → token_hash, expires_at	Extracted to PASSWORD_RESET_TOKEN entity (PK: token_id). USER has PASSWORD_RESET_TOKEN (1:N). A user may have multiple active reset tokens; each is stored as a separate row.	FR004 , UC002

Table 5: Third Normal Form (3NF) — Transitive Dependency Analysis

- No Controlled Denormalization exists in this schema. All foreign key references are resolved cleanly through joins without intentional redundancy.

2.2.5 Boyce-Codd Normal Form (BCNF)

- BCNF requires that for every functional dependency X → Y, X must be a superkey. All 17 entities use single-column primary keys, so every functional dependency trivially satisfies this. Entities with additional candidate keys are verified below.

Entity	Primary Key	Candidate Keys (beside PK)	BCNF Check
USER	user_id	email (UNIQUE)	email → all attributes. email is a superkey. ✓
STUDENT	student_id	index_no (UNIQUE)	index_no → all student attributes. index_no is a superkey. ✓
PASSWORD_RESET_TOKEN	token_id	token_hash (UNIQUE)	token_hash → all token attributes. token_hash is a superkey. ✓

BLOCK	block_id	No additional candidate keys	All FDs have PK on the left. ✓
ROOM	room_id	No additional candidate keys	All FDs have PK on the left. ✓
BED	bed_no	No additional candidate keys	All FDs have PK on the left. ✓
STAFF	staff_id	No additional candidate keys	All FDs have PK on the left. ✓
GUARDIAN	guardian_id	No additional candidate keys	All FDs have PK on the left. ✓
ANNOUNCEMENT	ann_id	No additional candidate keys	All FDs have PK on the left. ✓
NOTIFICATION	notif_id	No additional candidate keys	All FDs have PK on the left. ✓

LEAVE	leave_id	No additional candidate keys	All FDs have PK on the left. ✓
LATE_ACCESS	la_id	No additional candidate keys	All FDs have PK on the left. ✓
COMPLAINT	complaint_id	No additional candidate keys	All FDs have PK on the left. ✓
COMPLAINT_UPDATE	update_no	No additional candidate keys	All FDs have PK on the left. ✓
COMPLAINT_IMAGE	image_no	No additional candidate keys	All FDs have PK on the left. ✓
SCAN_DEVICE	device_id	No additional candidate keys	All FDs have PK on the left. ✓
ATTENDANCE_LOG	log_id	No additional candidate keys	All FDs have PK on the left. ✓

Table 6: Boyce-Codd Normal Form (BCNF)

- No BCNF violations exist in the HostelHive schema.

2.2.6 Anomaly Elimination

Anomaly Type	Definition	HostelHive Example
Insertion Anomaly	Cannot insert data without unrelated data being present.	A new BLOCK can be added before any ROOM is assigned to it. A new ROOM can be added before any BED or STUDENT is associated. A COMPLAINT can be filed before any COMPLAINT_UPDATE or COMPLAINT_IMAGE is attached. A SCAN_DEVICE can be registered before any ATTENDANCE_LOG is recorded. An ANNOUNCEMENT can be posted before any NOTIFICATION is triggered.
Update Anomaly	Changing one fact requires modifying multiple rows.	Updating a BLOCK name requires changing only one BLOCK row — not every ROOM, BED, or STUDENT row. Updating a USER's email requires changing only one USER row — not the STUDENT or STAFF rows that reference it. Changing a COMPLAINT's status updates one COMPLAINT row; all COMPLAINT_UPDATE and COMPLAINT_IMAGE records remain unaffected.

Deletion Anomaly	Deleting one record accidentally destroys unrelated information.	Deleting a BED record does not delete the ROOM or BLOCK it belongs to. Revoking a LEAVE record does not erase the STUDENT profile. Deleting a COMPLAINT_IMAGE does not remove the parent COMPLAINT. Deactivating a USER account does not erase the associated STUDENT or STAFF record, their ATTENDANCE_LOG history, or their COMPLAINT submissions. Expiring a PASSWORD_RESET_TOKEN does not affect the USER account itself.

Table 7: Anomaly Elimination

2.2.7 Normalized Schema Summary

Entity	Primary Key	1NF	2NF	3NF/BCNF	Key Notes
BLOCK	block_id	✓	✓	✓	Resolves Transitive Dep. #1; stores block name and gender

ROOM	room_id	✓	✓	✓	Resolves Transitive Dep. #2; FK to BLOCK via block_id
BED	bed_no	✓	✓	✓	FK to ROOM via room_id; stores bed status
STUDENT	student_id	✓	✓	✓	Resolves Transitive Dep. #3; FK to USER via user_id; candidate key: index_no
USER	user_id	✓	✓	✓	Candidate key: email; stores role and credentials for all user types

STAFF	staff_id	✓	✓	✓	Resolves Transitive Dep. #4; FK to USER via user_id; stores designation
GUARDIAN	guardian_id	✓	✓	✓	Resolves Transitive Dep. #5; FK to STUDENT via student_id; stores name and relationship
PASSWORD_RESET_TOKEN	token_id	✓	✓	✓	Resolves Transitive Dep. #6; FK to USER via user_id; candidate key: token_hash
ANNOUNCEMENT	ann_id	✓	✓	✓	FK to USER via posts relationship;

					stores title and priority
NOTIFICATION	notif_id	✓	✓	✓	FK to USER via receives relationship; stores message and is_read
LEAVE	leave_id	✓	✓	✓	FK to STUDENT via requests relationship; stores from_date and status
LATE_ACCESS	la_id	✓	✓	✓	FK to STUDENT via requests relationship; stores status and until_time

COMPLAINT	complaint_id	✓	✓	✓	FK to STUDENT via files relationship; stores category and status
COMPLAINT_UPDATE	update_no	✓	✓	✓	FK to COMPLAINT via has relationship; stores author_id and text
COMPLAINT_IMAGE	image_no	✓	✓	✓	FK to COMPLAINT via has relationship; stores url and uploaded_at
SCAN_DEVICE	device_id	✓	✓	✓	FK implicit via records relationship; stores location and status

ATTENDANCE_LOG	log_id	✓	✓	✓	FK to SCAN_DEVICE via records relationship; stores timestamp and direction

Table 8: Normalized Schema Summary

CHAPTER 3: CLASS DIAGRAM

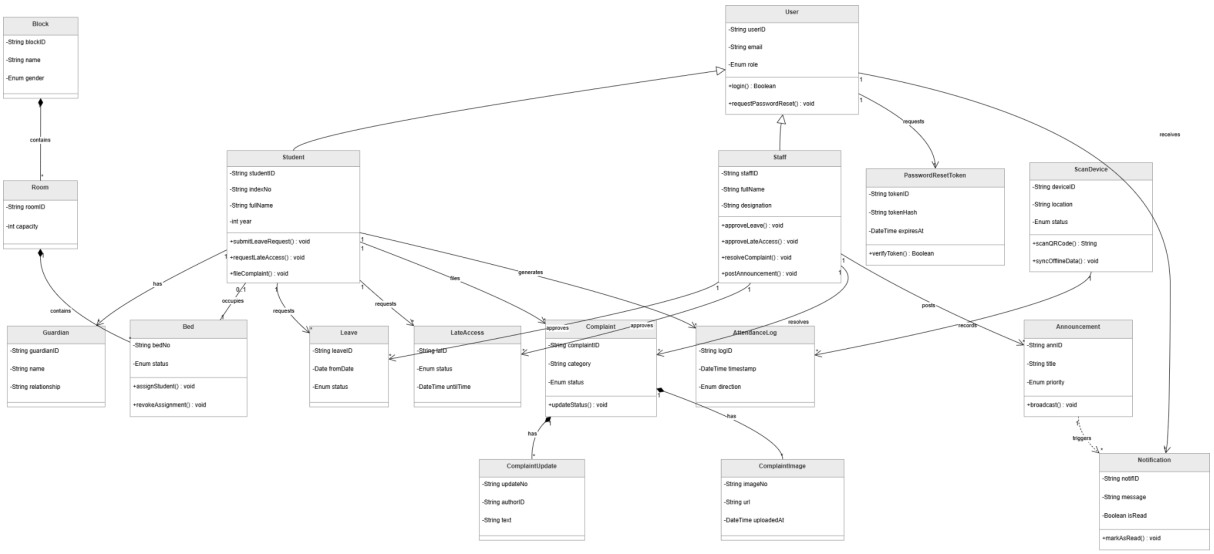


Figure 4: Class Diagram

CHAPTER 4: SEQUENCE DIAGRAMS

4.1 UC001 - Login /Authentication

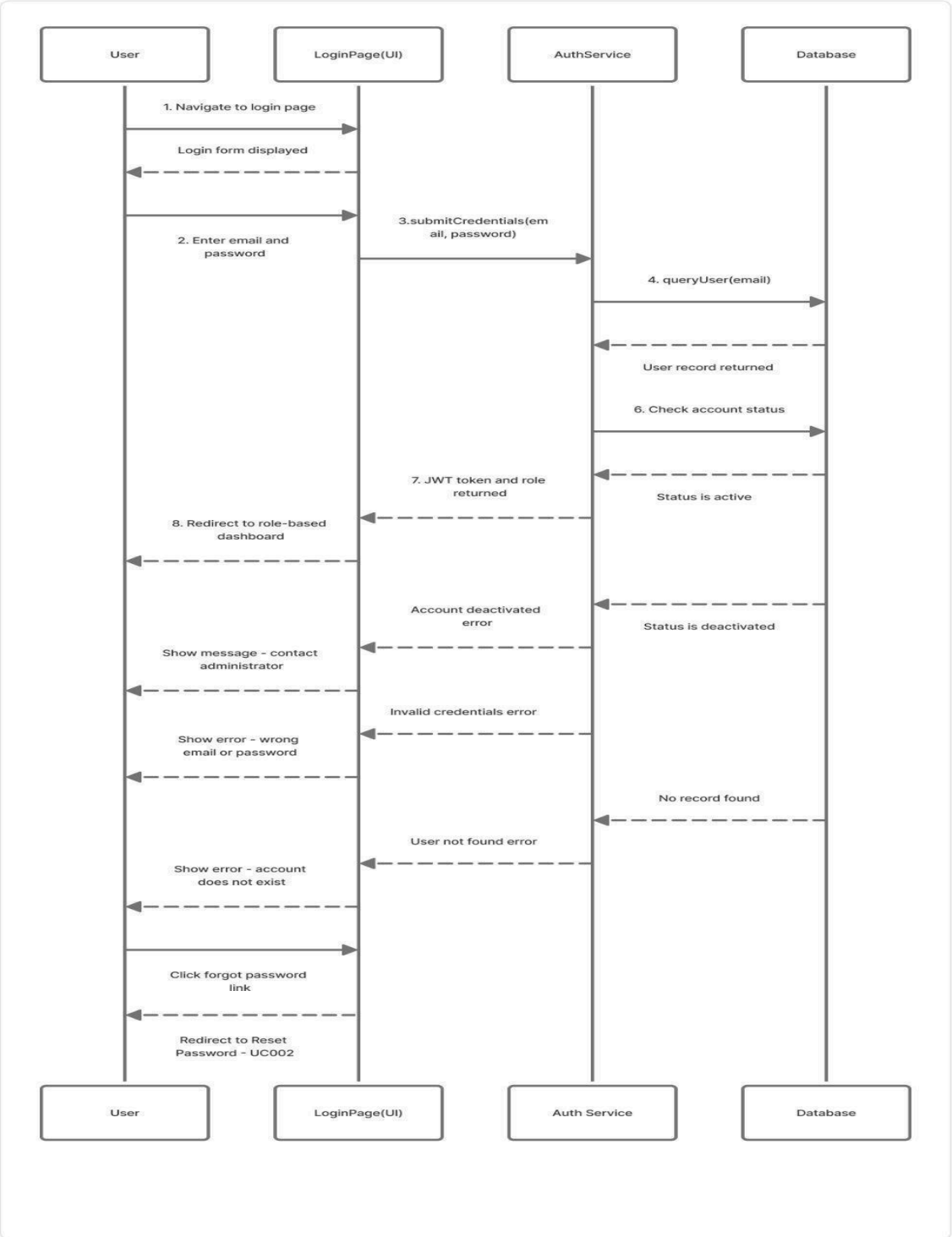


Figure 5: Login /Authentication

4.2 UC002 - Reset Password

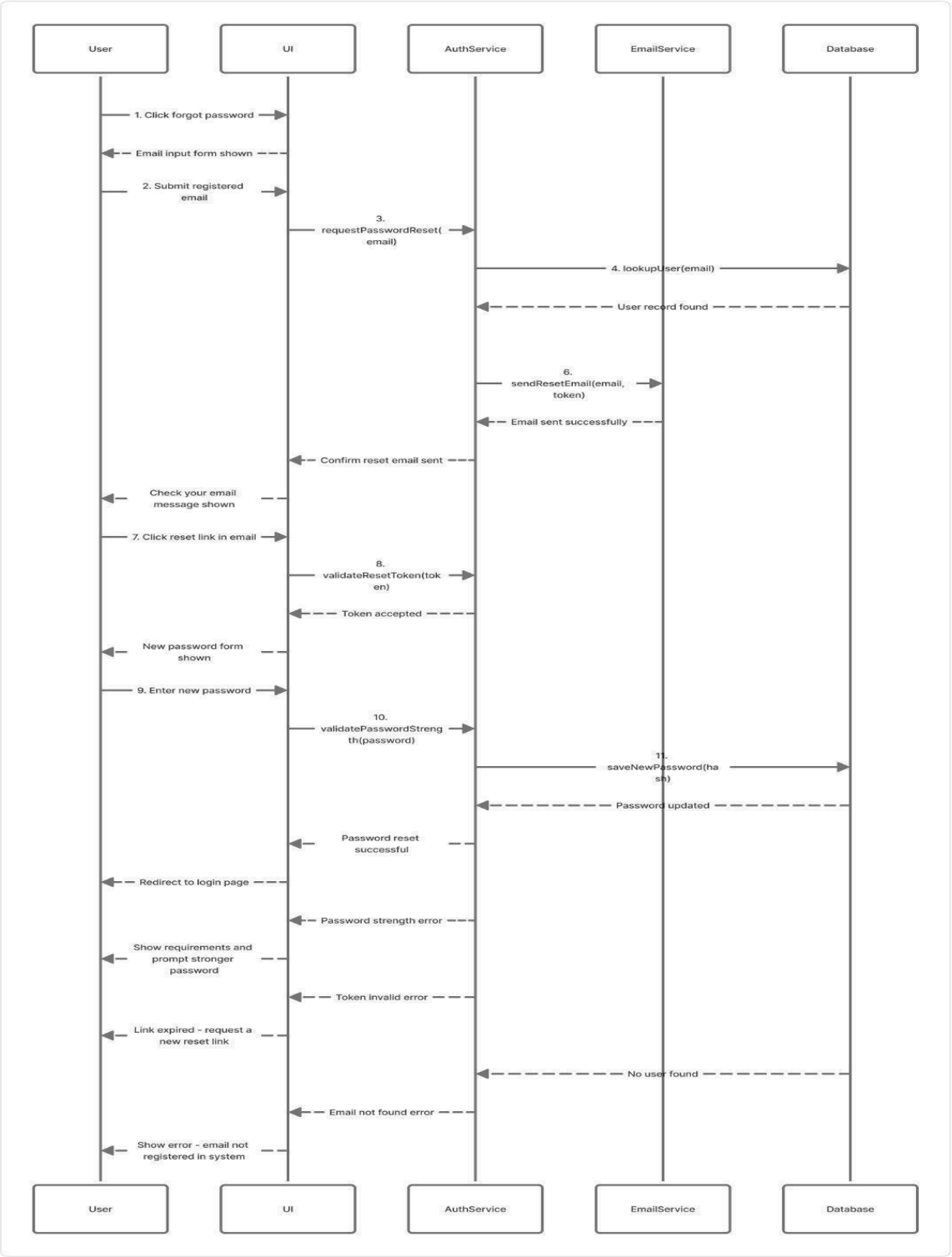


Figure 6: Reset Password

4.3 UC003 - Manage Student Profiles

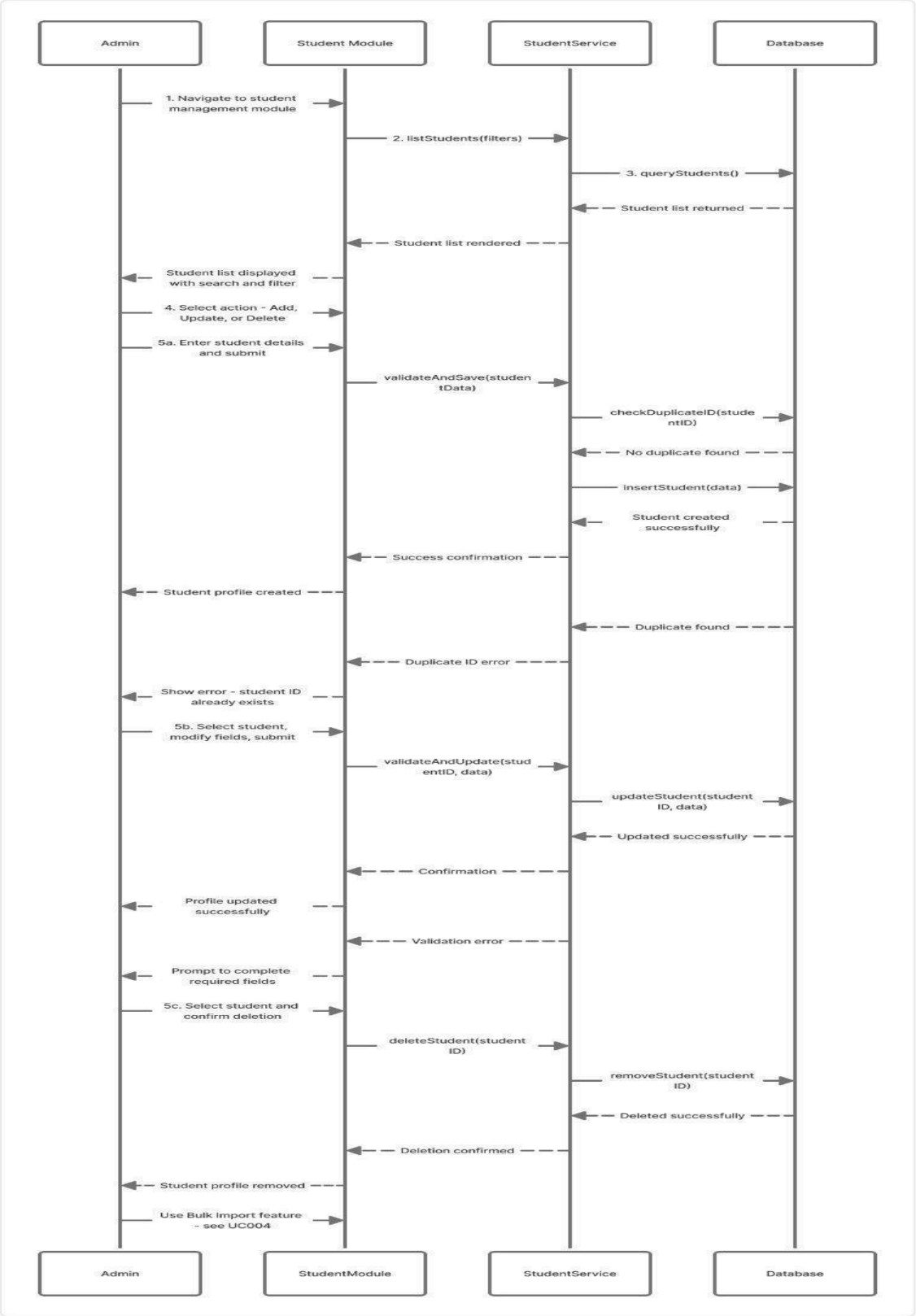


Figure 7: Manage Student Profiles

4.4 UC004 - Bulk Import Students (CSV)

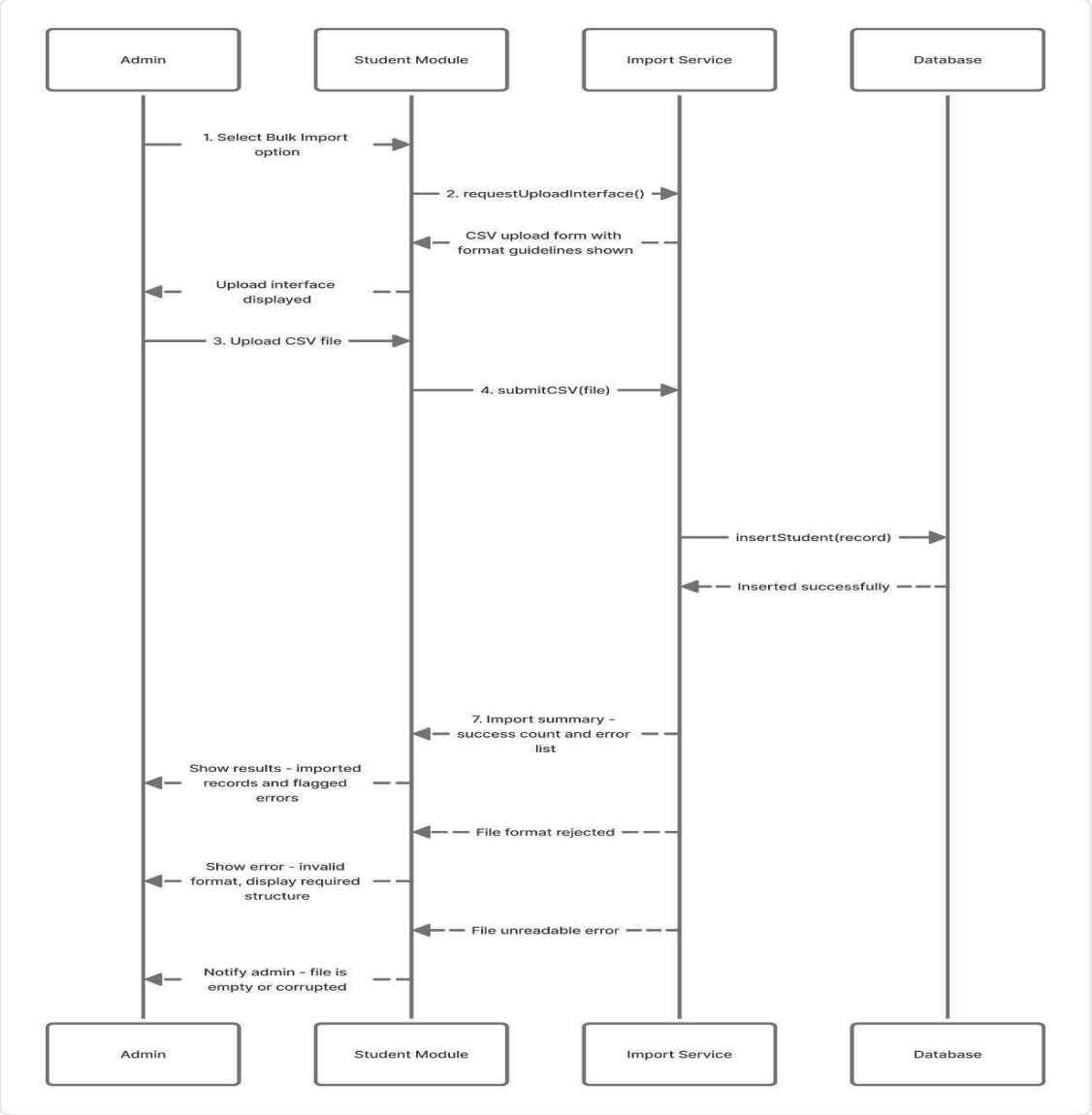


Figure 8: Bulk Import Students (CSV)

4.5 UC005 - Scan QR Code

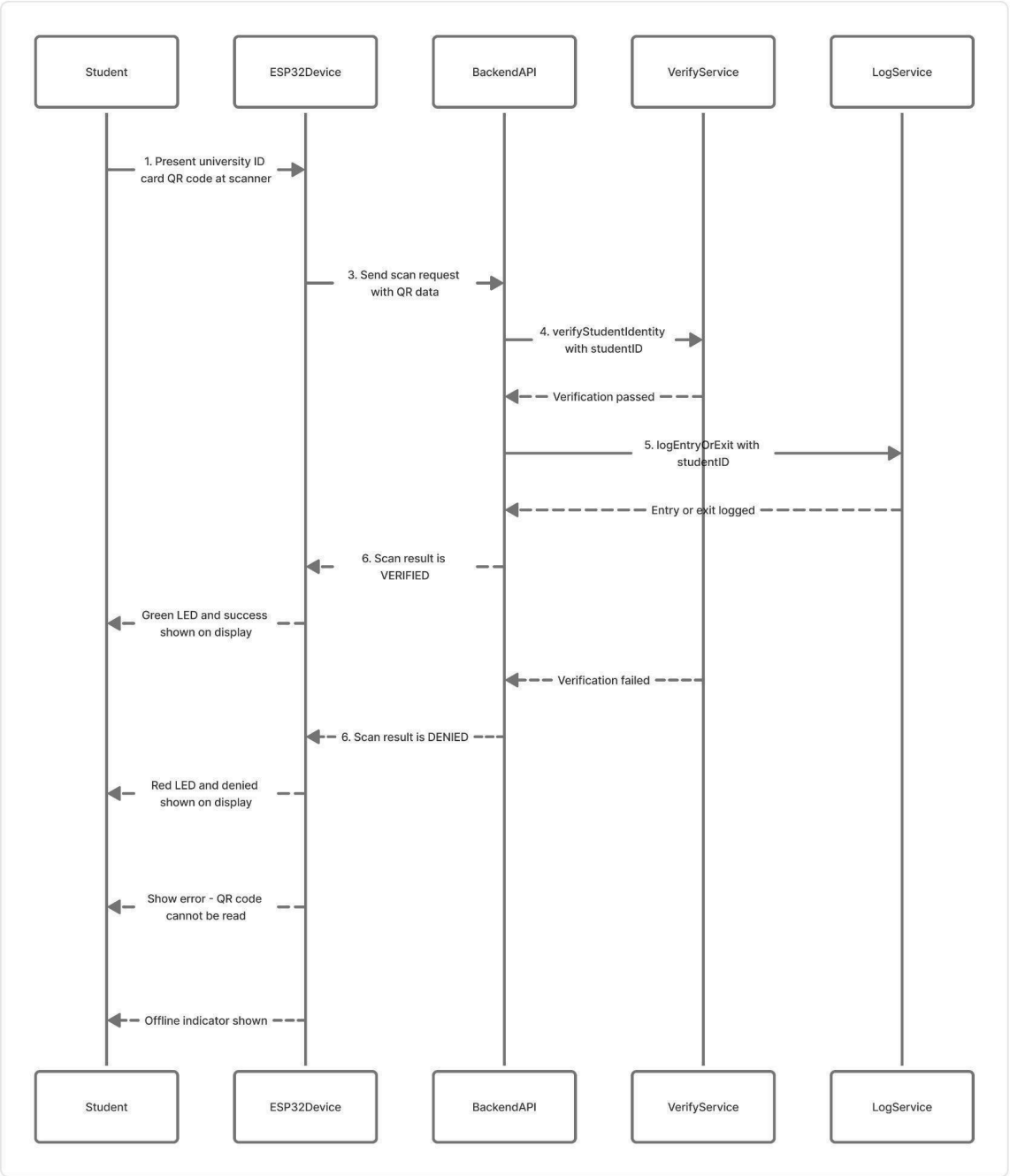


Figure 9: Scan QR Code

4.6 UC006 - Verify Student Identity

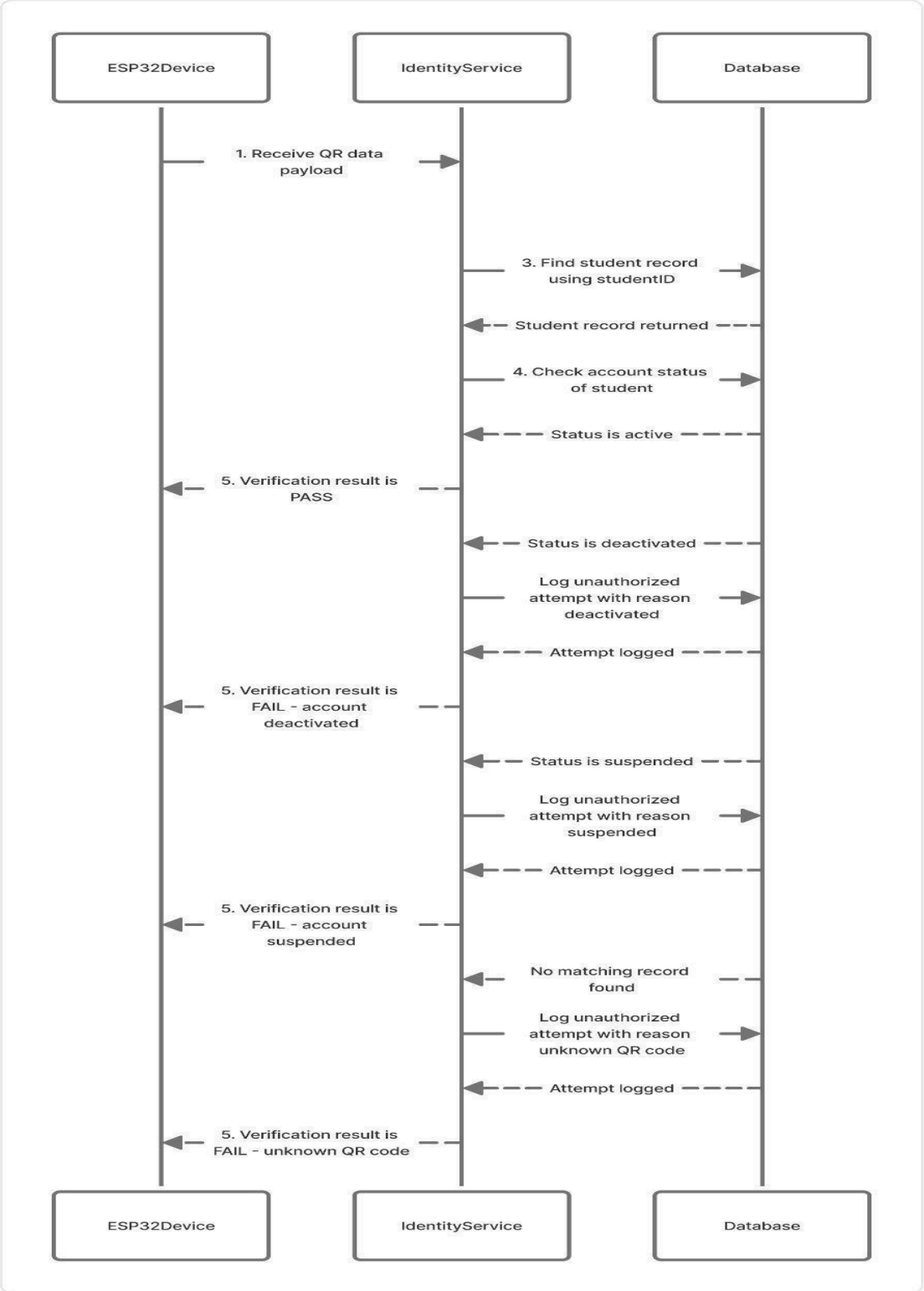


Figure 10: Verify Student Identity

4.7 UC007 - Log Entry/Exit

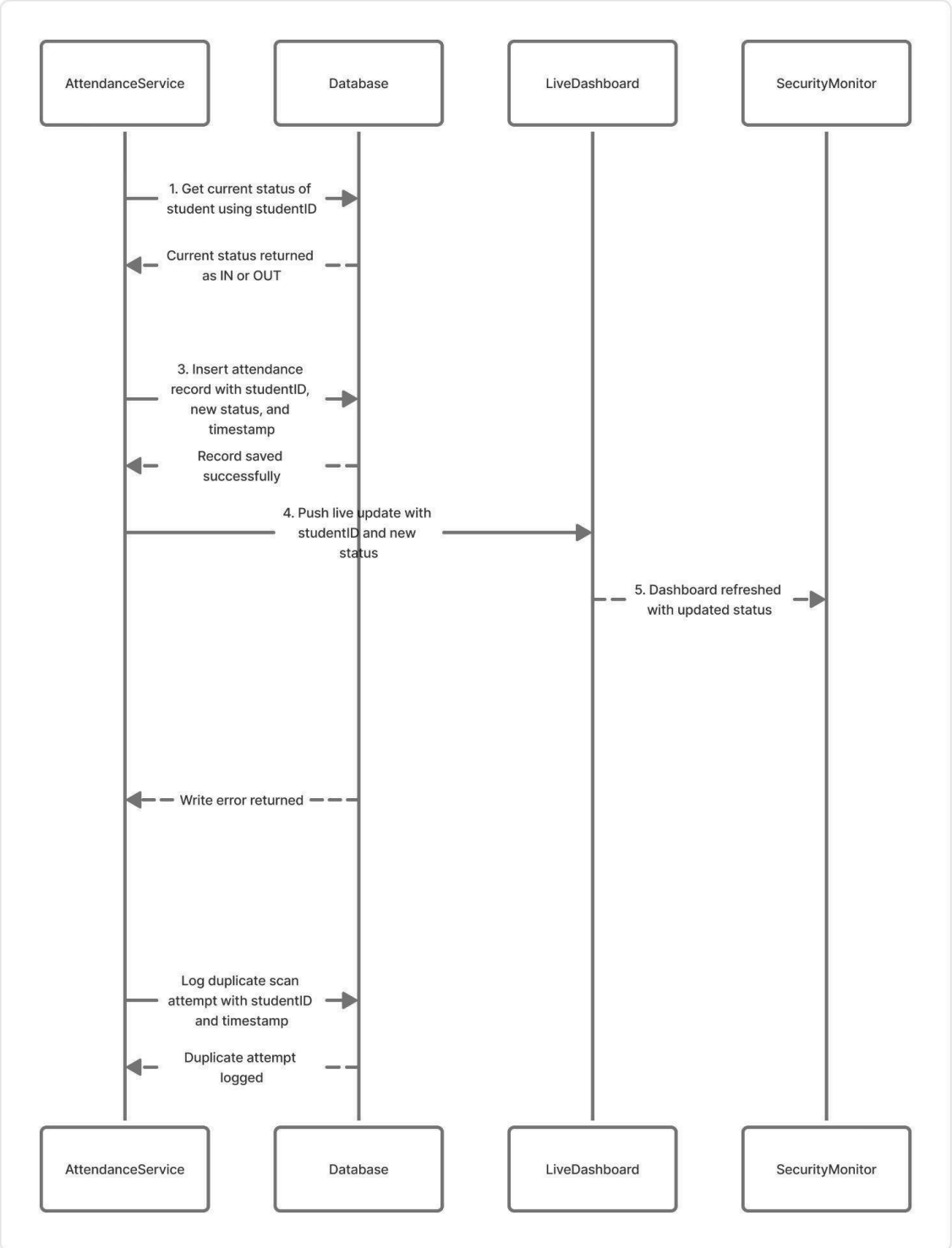


Figure 11: Log Entry/Exit

4.8 UC008 - Detect Unapproved Late Entry

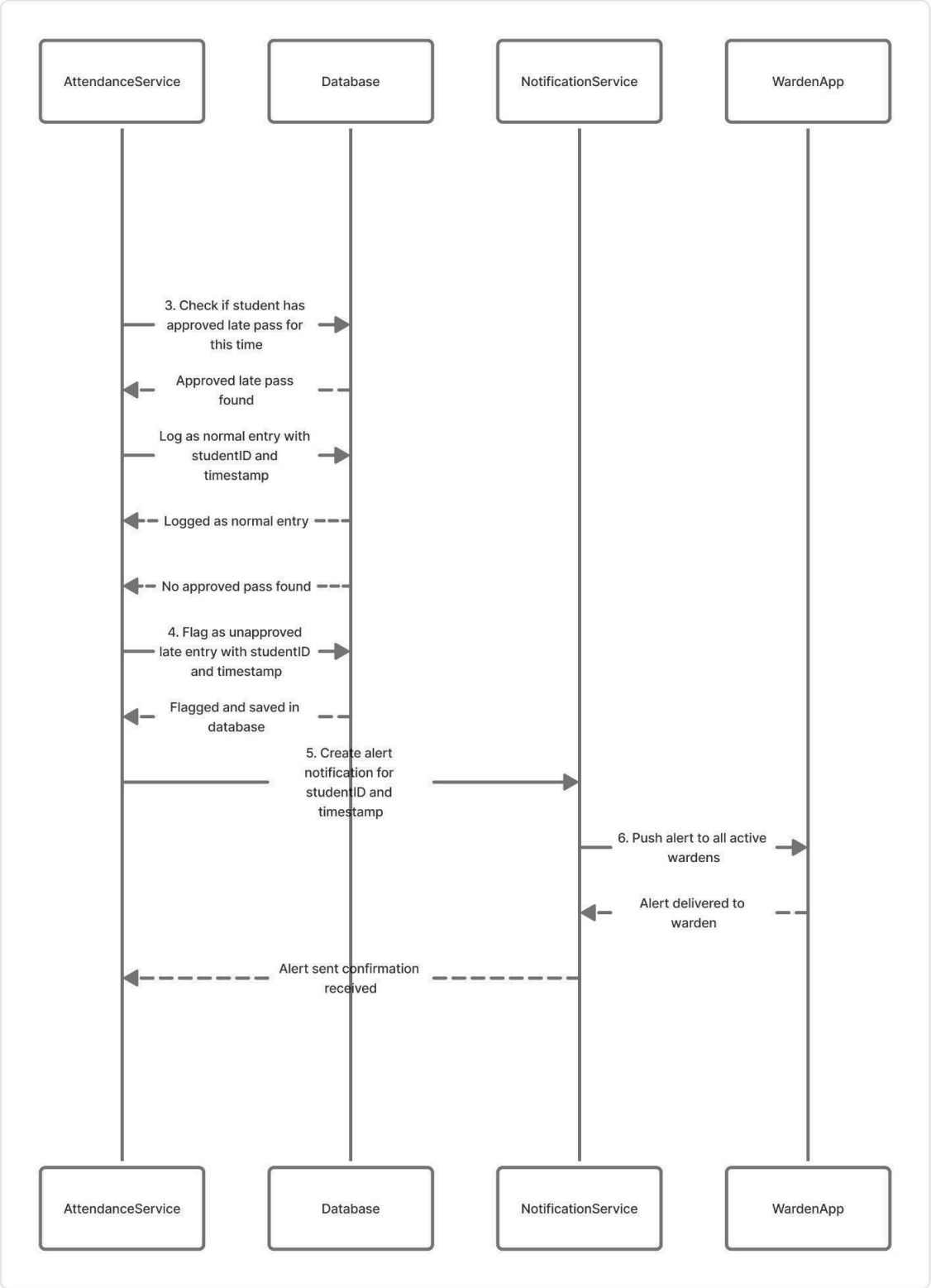


Figure 12: Detect Unapproved Late Entry

4.9 UC009 - Record Leave Status

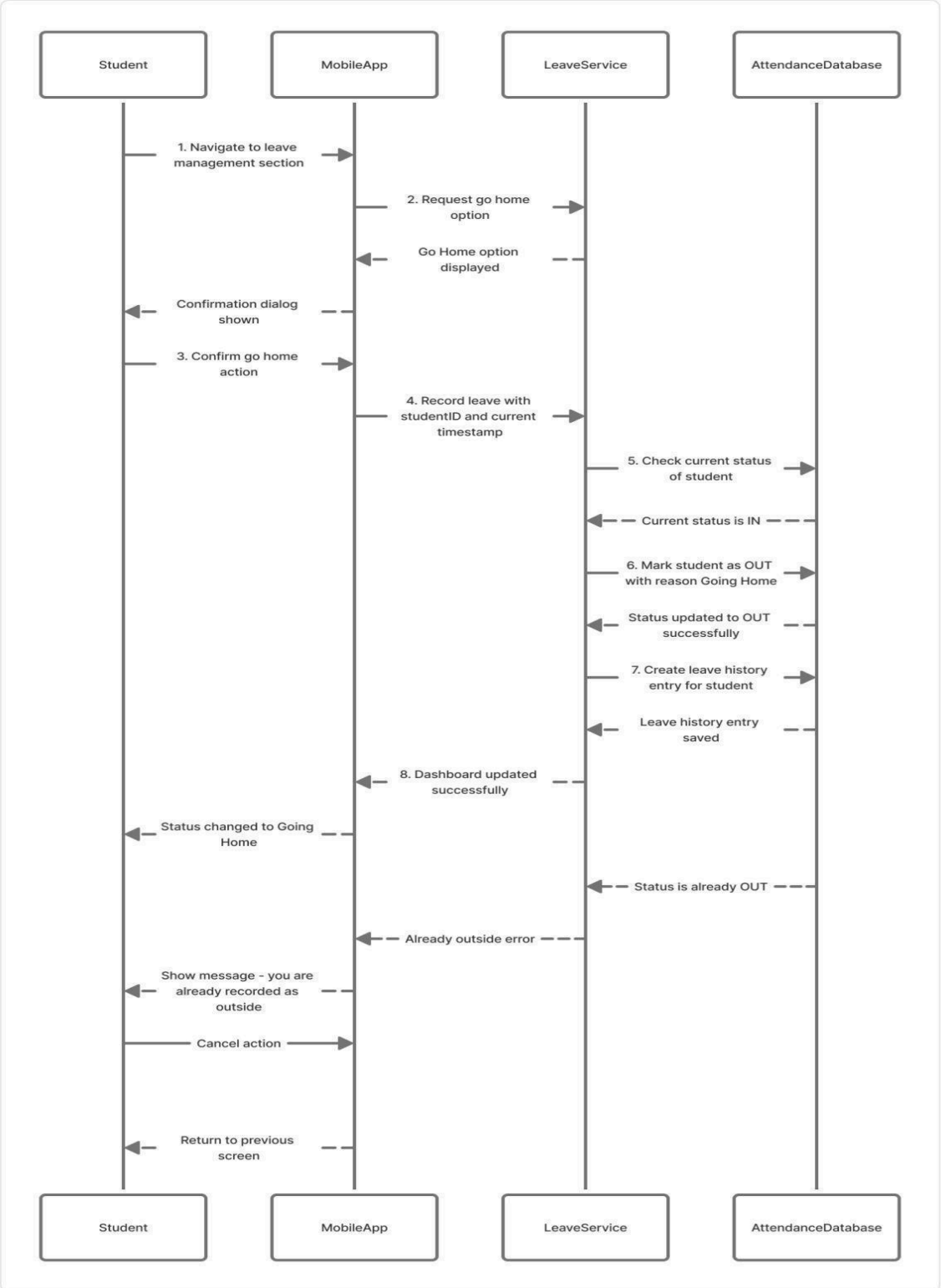


Figure 13: Record Leave Status

4.10 U010 - Request Late Entry/Exit

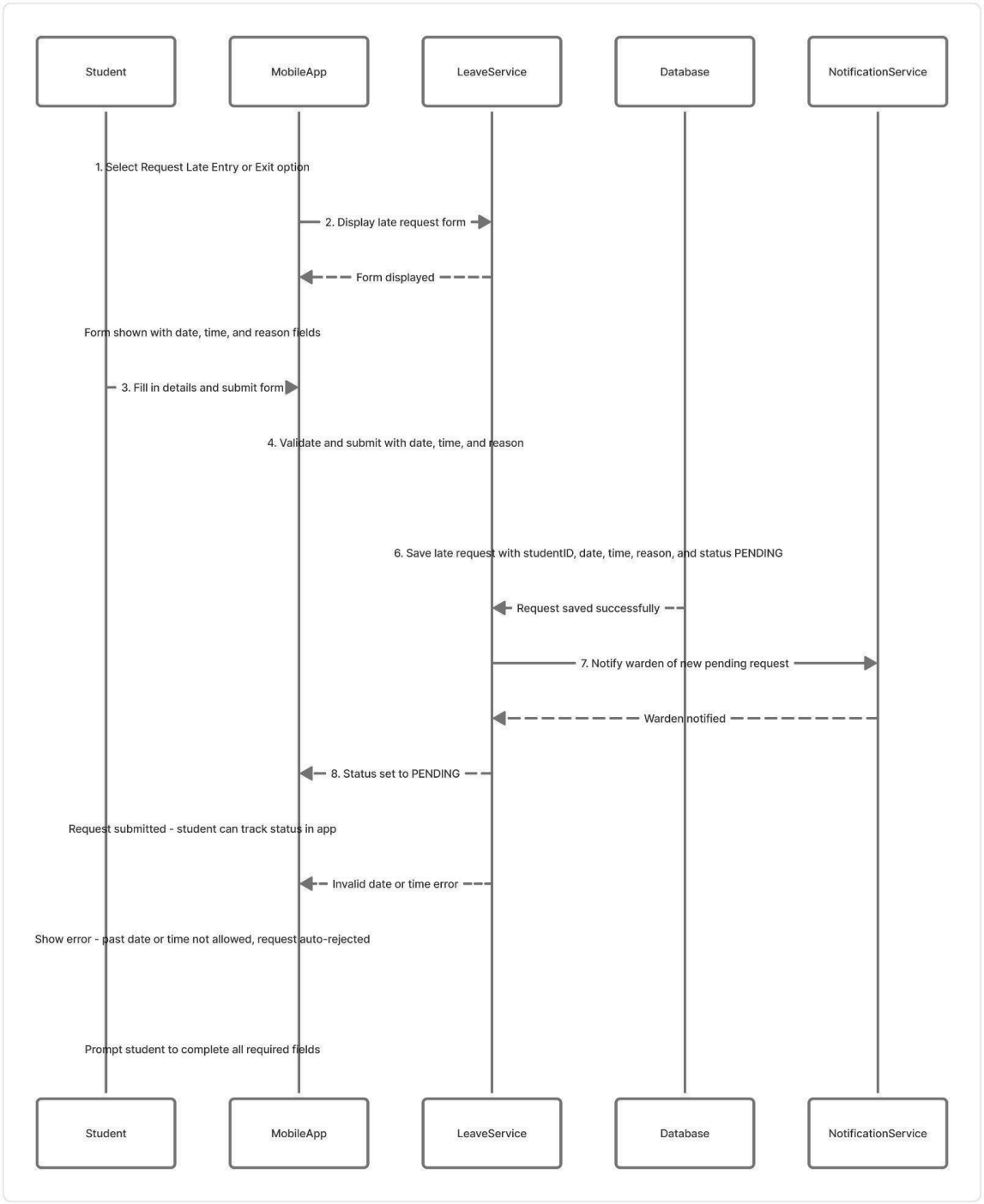


Figure 14: Request Late Entry/Exit

4.11 UC011 - Approve/Reject Late Access Request

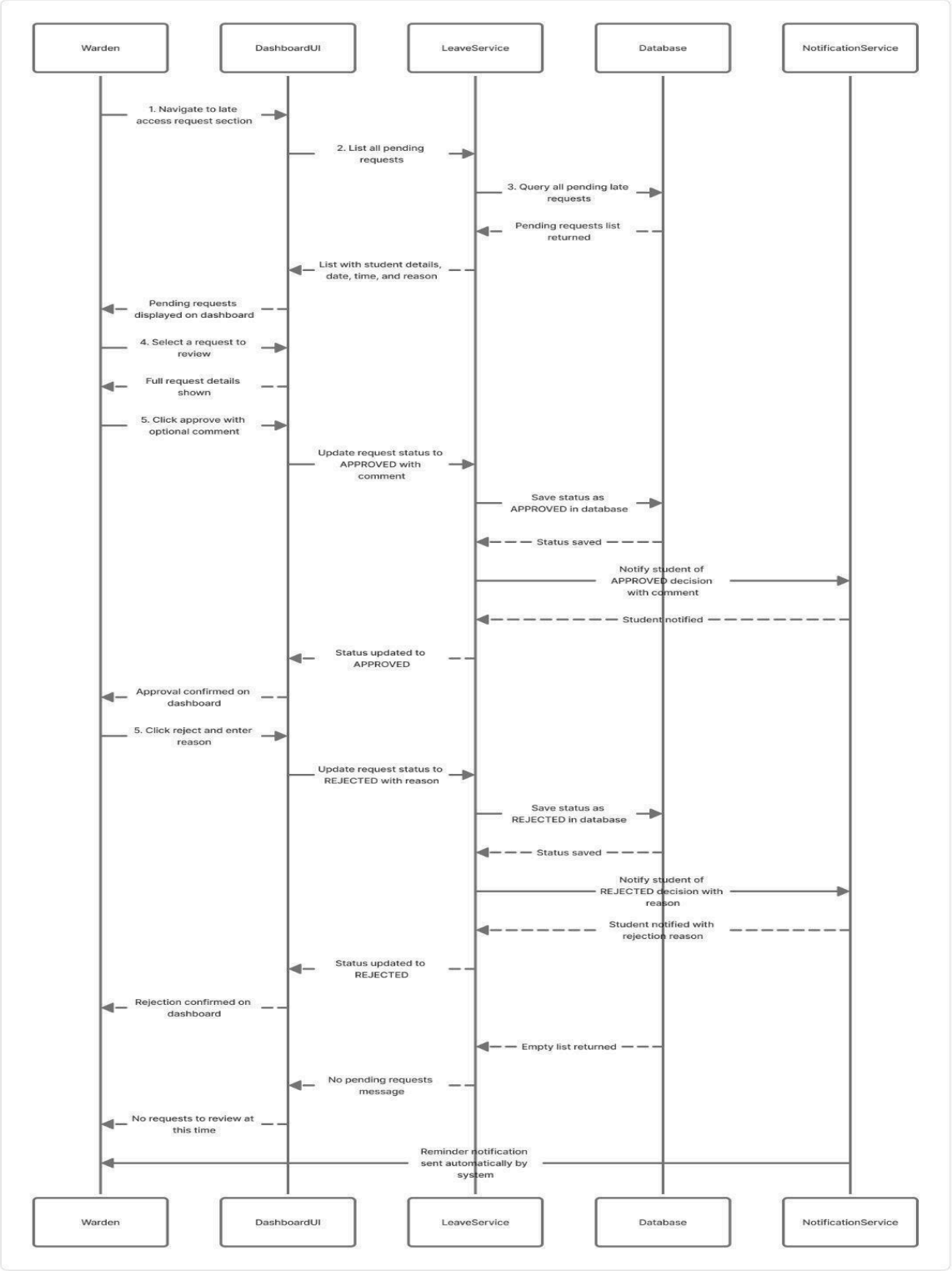


Figure 15: Approve/Reject Late Access Request

4.12 UC012 - Submit Complaint

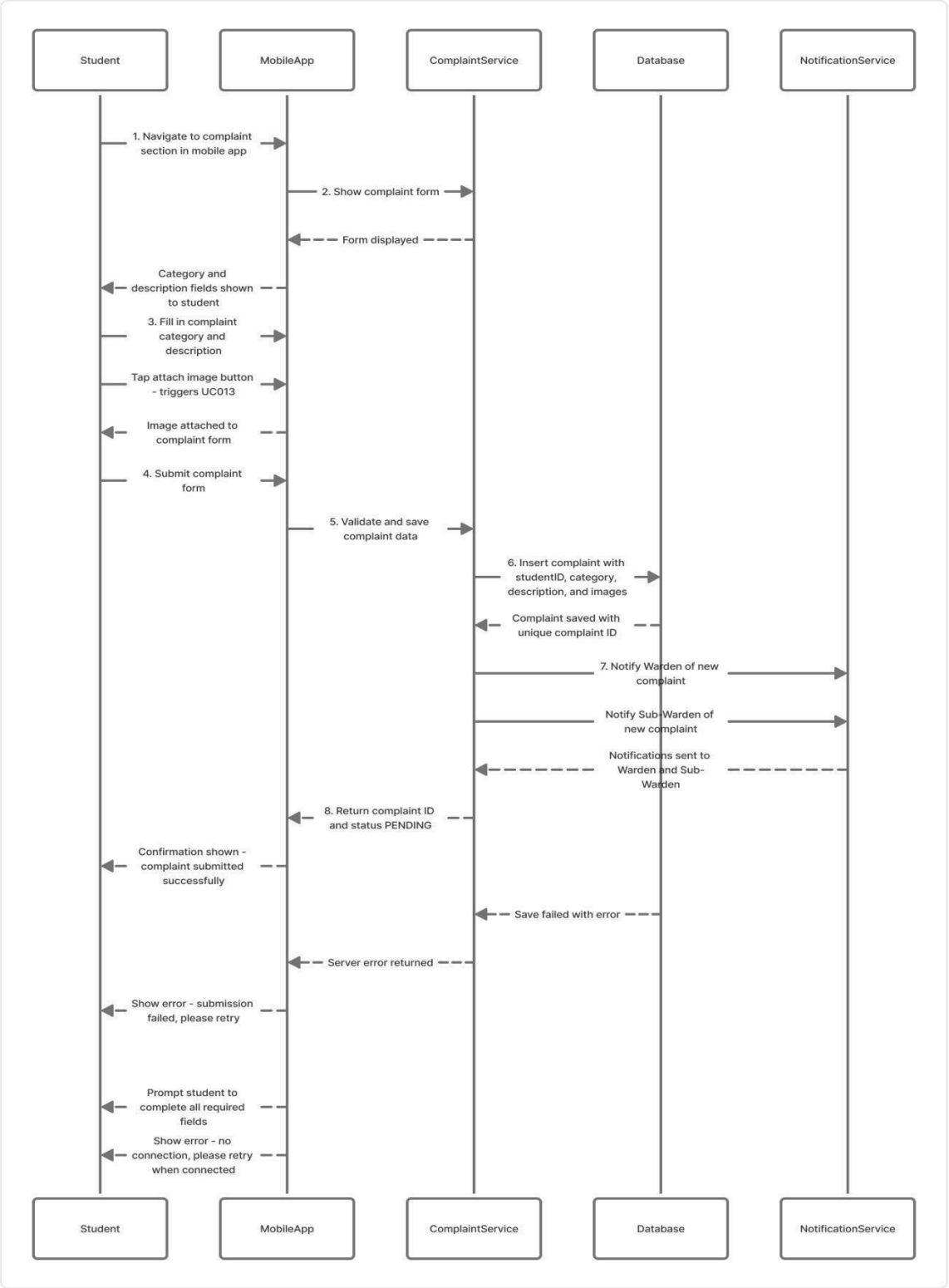


Figure 16: Submit Complaint

4.13 UC013 - Attach Images

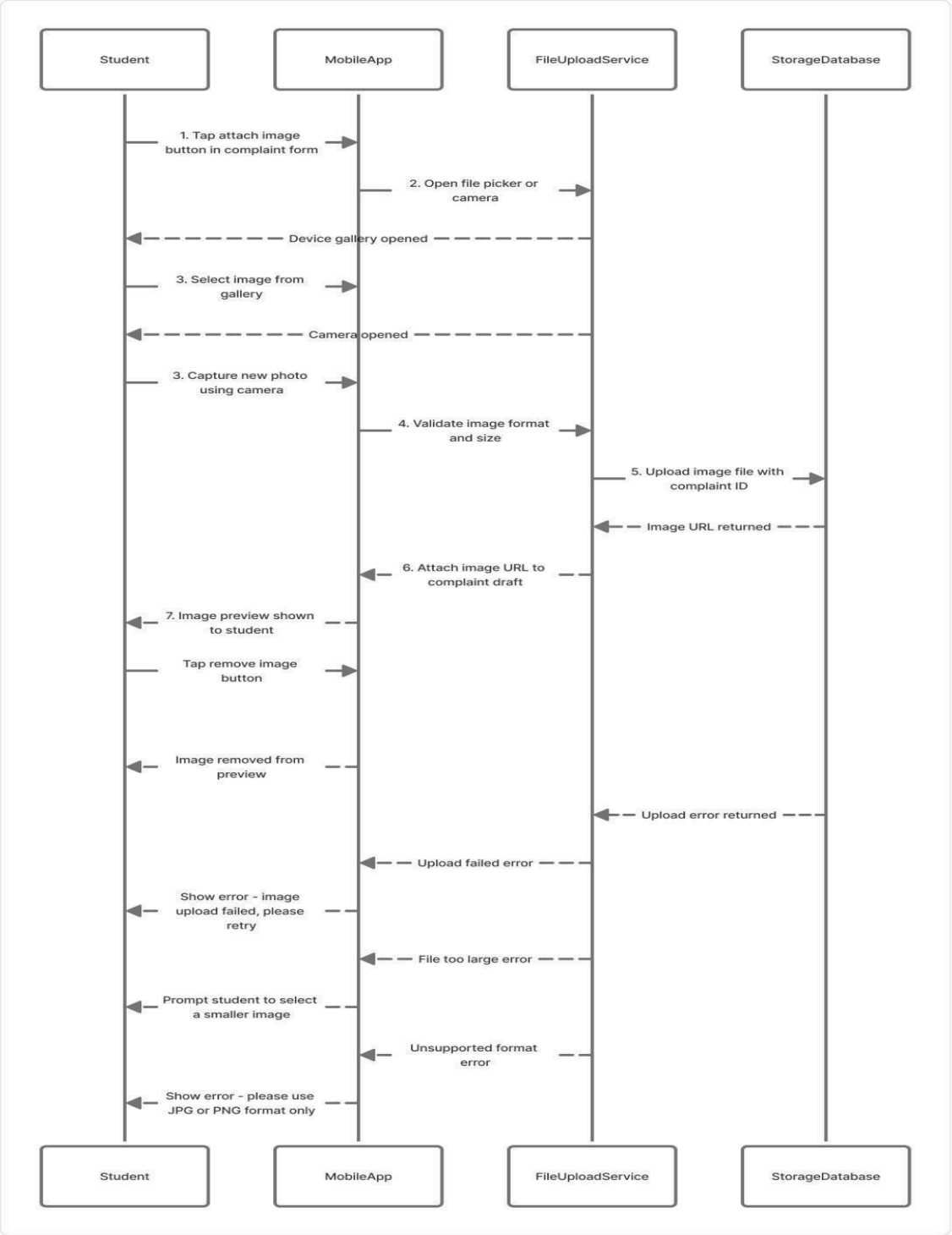


Figure 17: Attach Images

4.14 UC014 - Receive Notifications

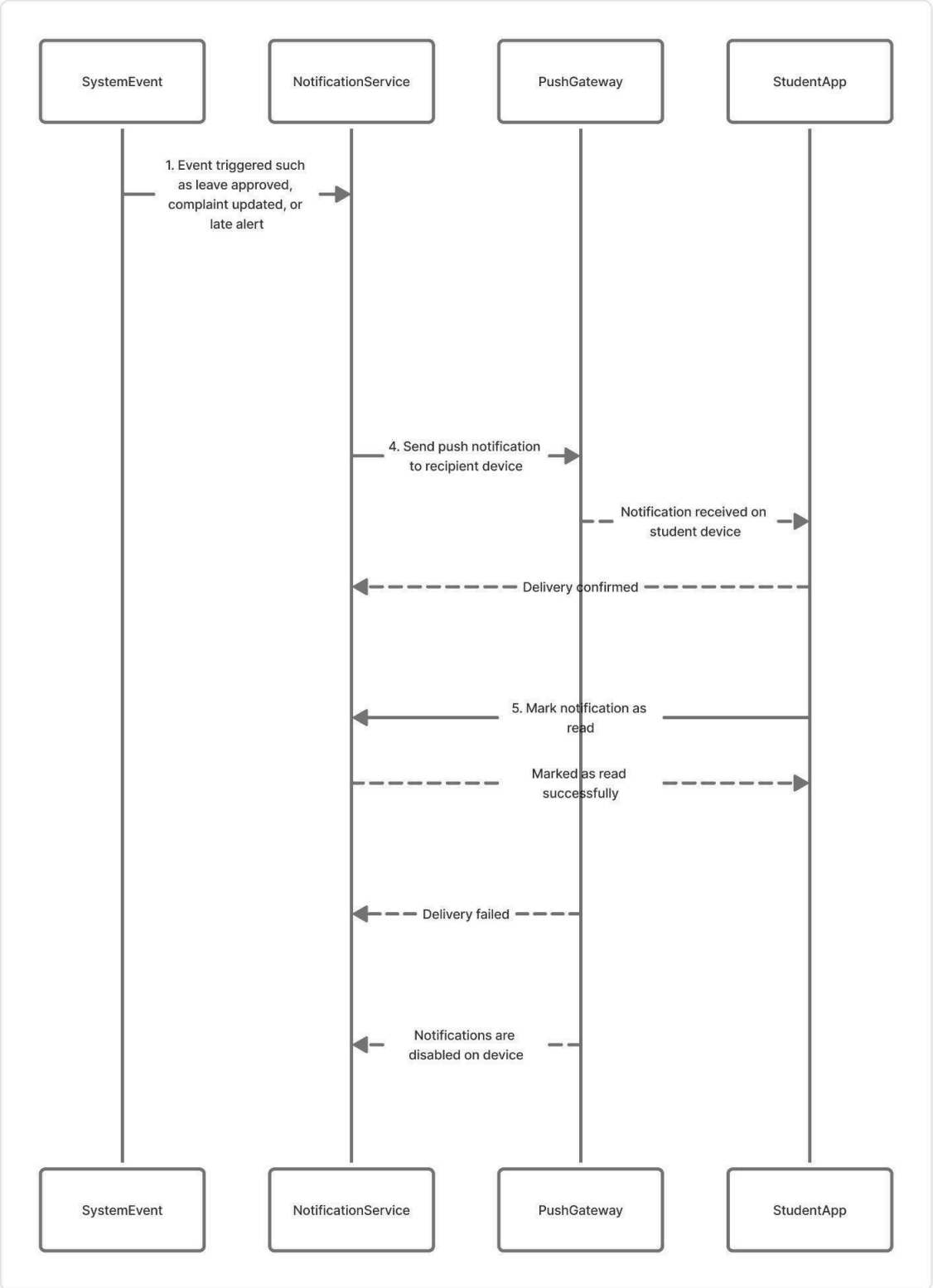


Figure 18: Receive Notifications

4.15 UC015 - Monitor & View Attendance Dashboard

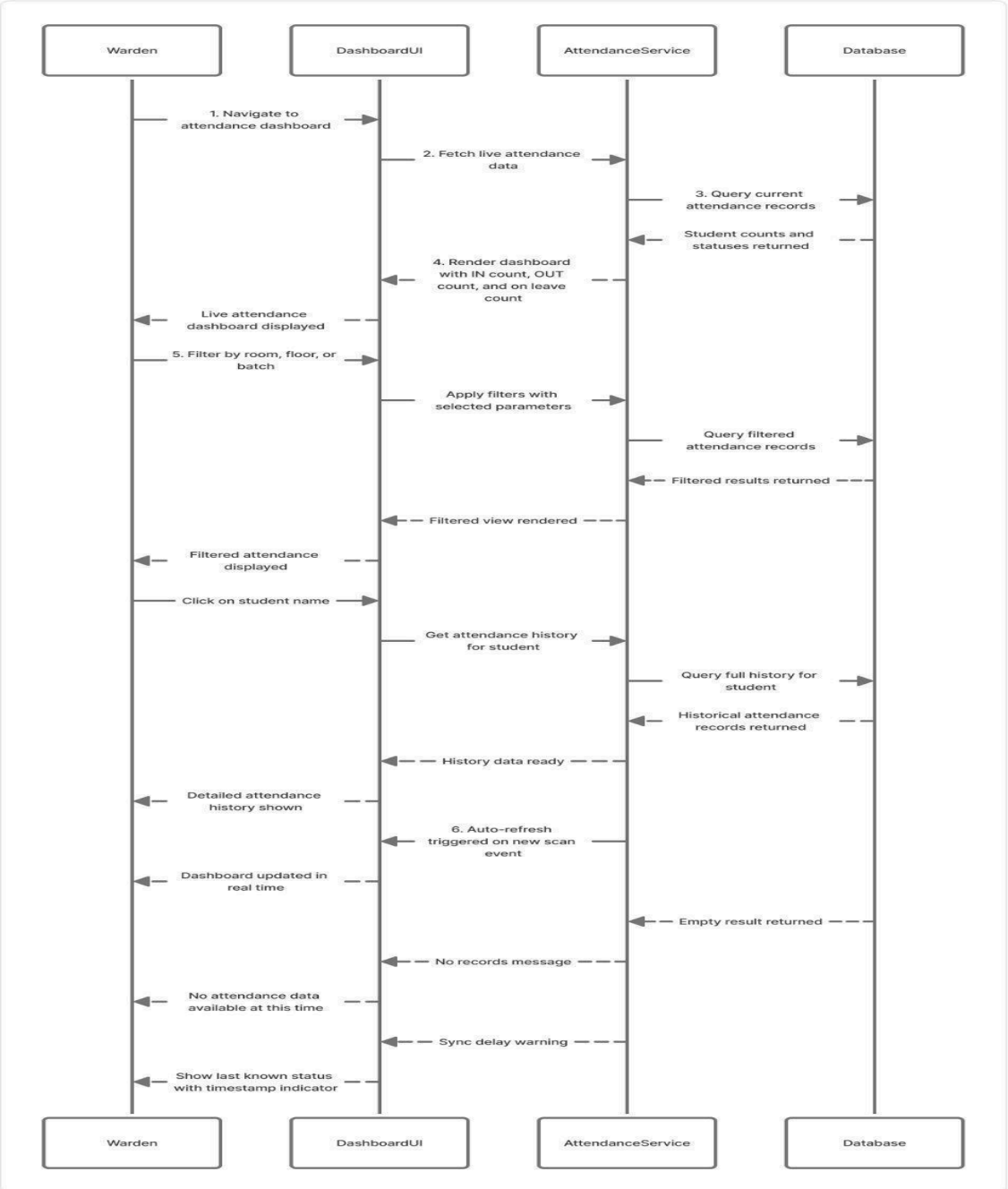


Figure 19: Monitor & View Attendance Dashboard

4.16 UC016 - Resolve Maintenance Issues

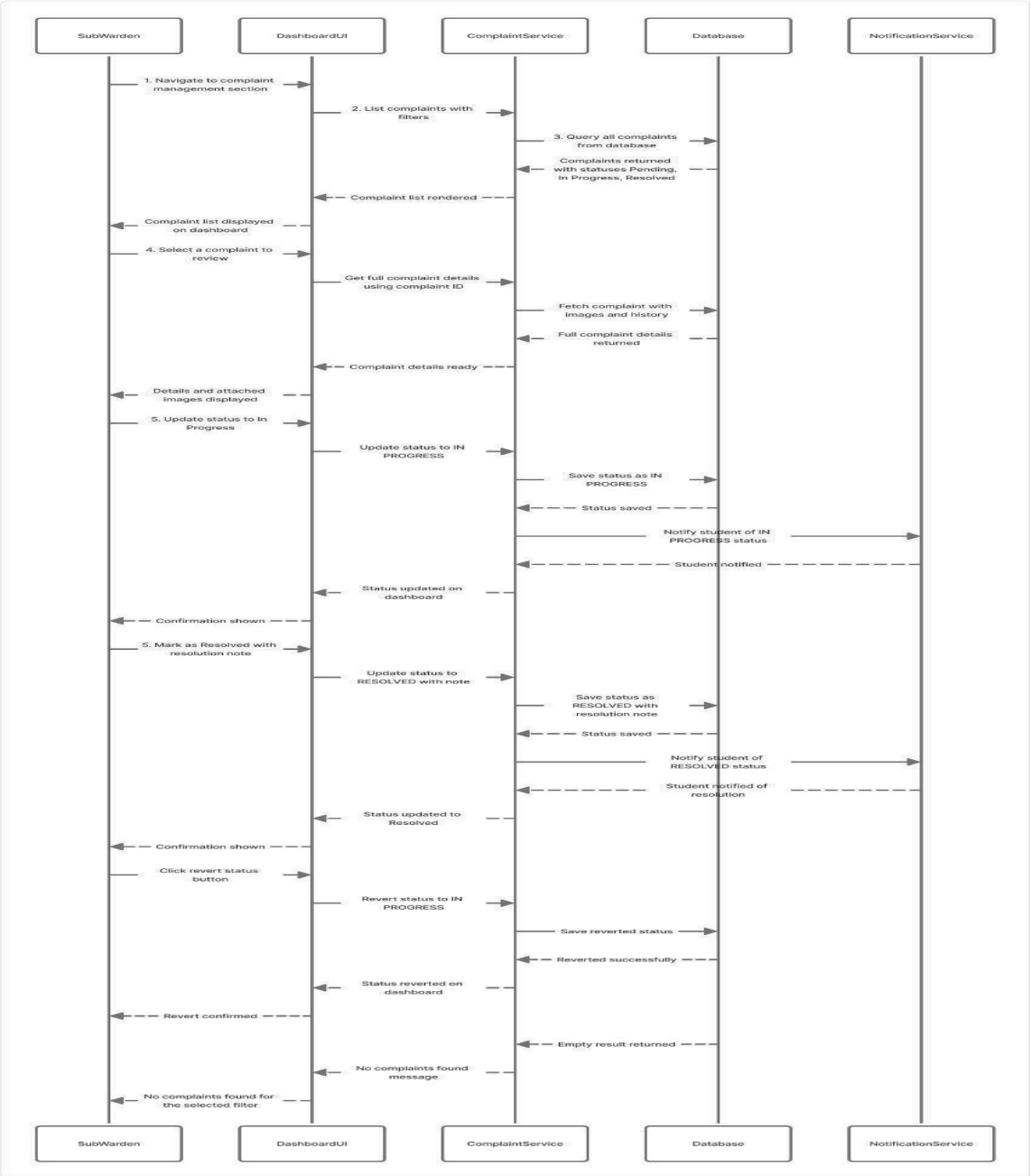


Figure 20: Resolve Maintenance Issues

4.17 UC017 - Generate Analytics & Reports

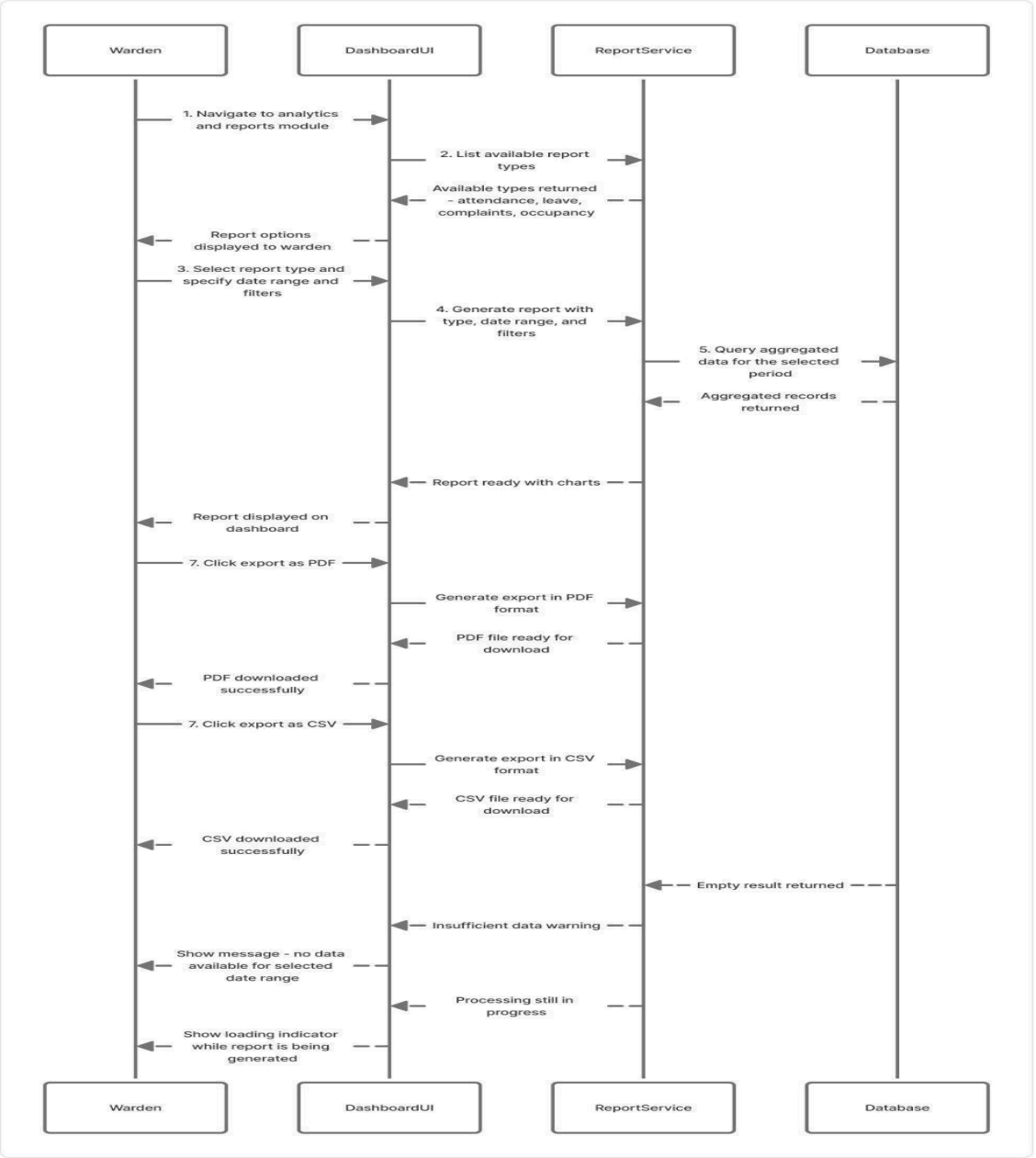


Figure 21: Generate Analytics & Reports

4.18 UC018 - Allocate Room/Bed

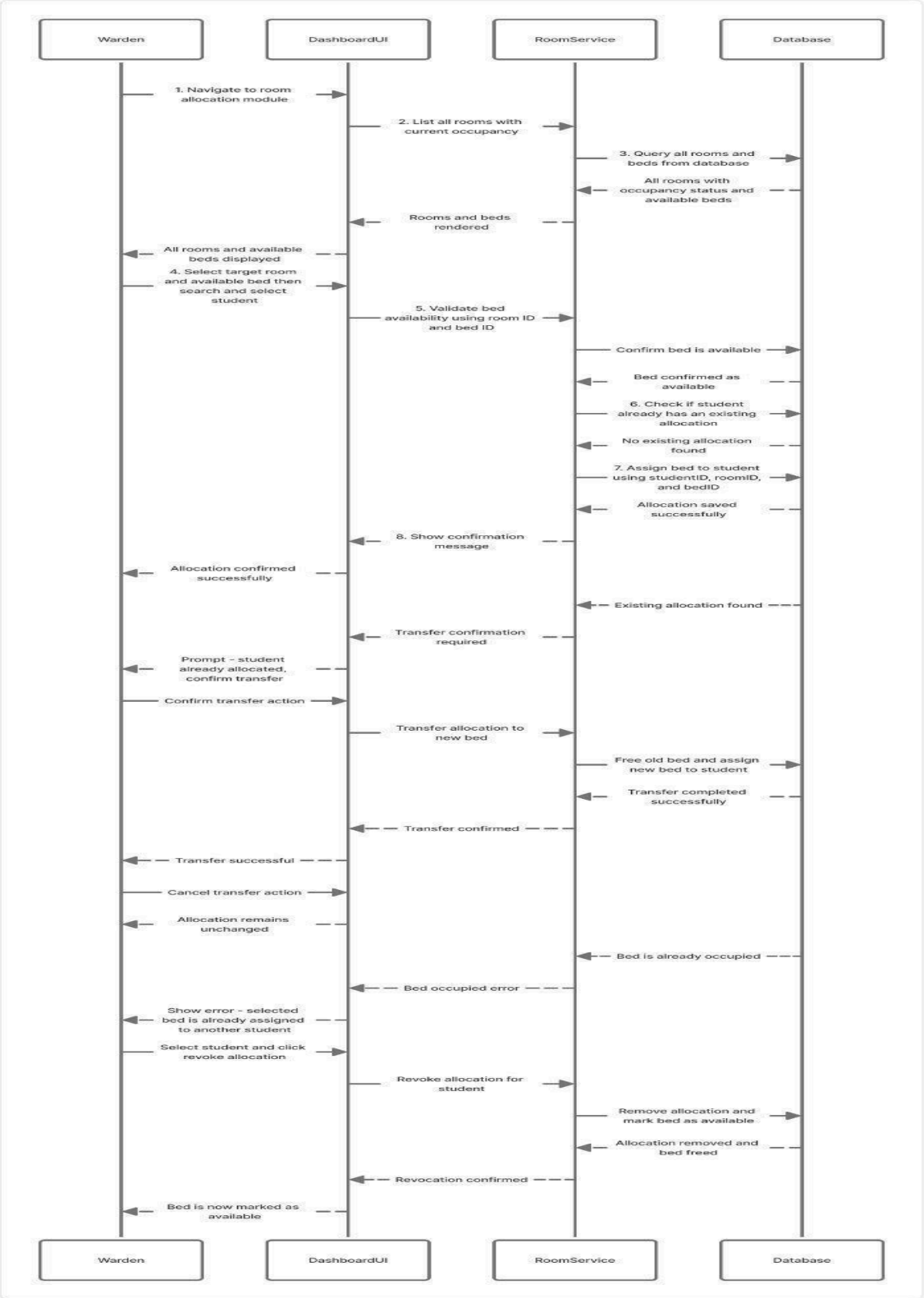


Figure 22: Allocate Room/Bed

4.19 UC019 - Monitor System Display

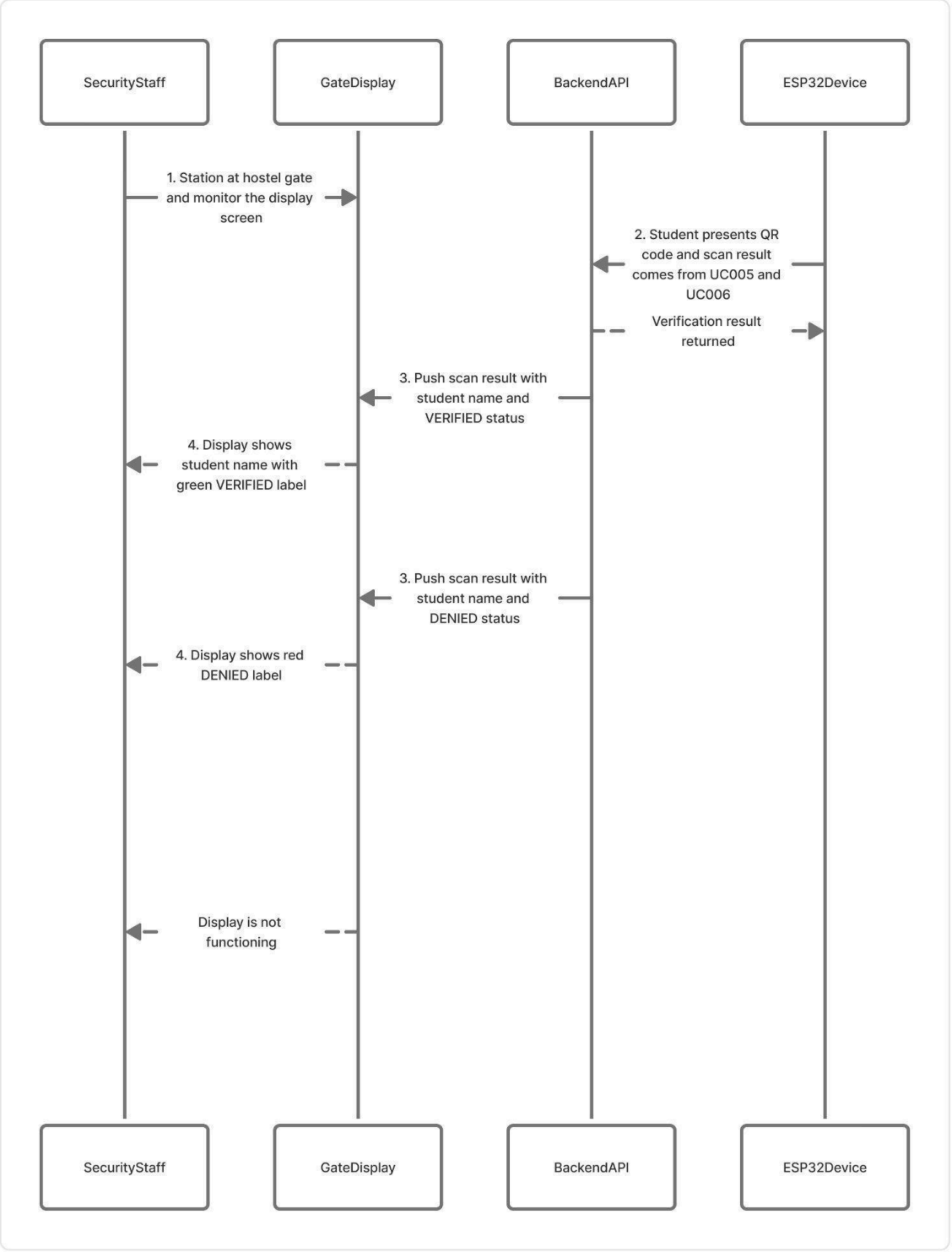


Figure 23: Monitor System Display

4.20 UC020 - Maintain Physical Records

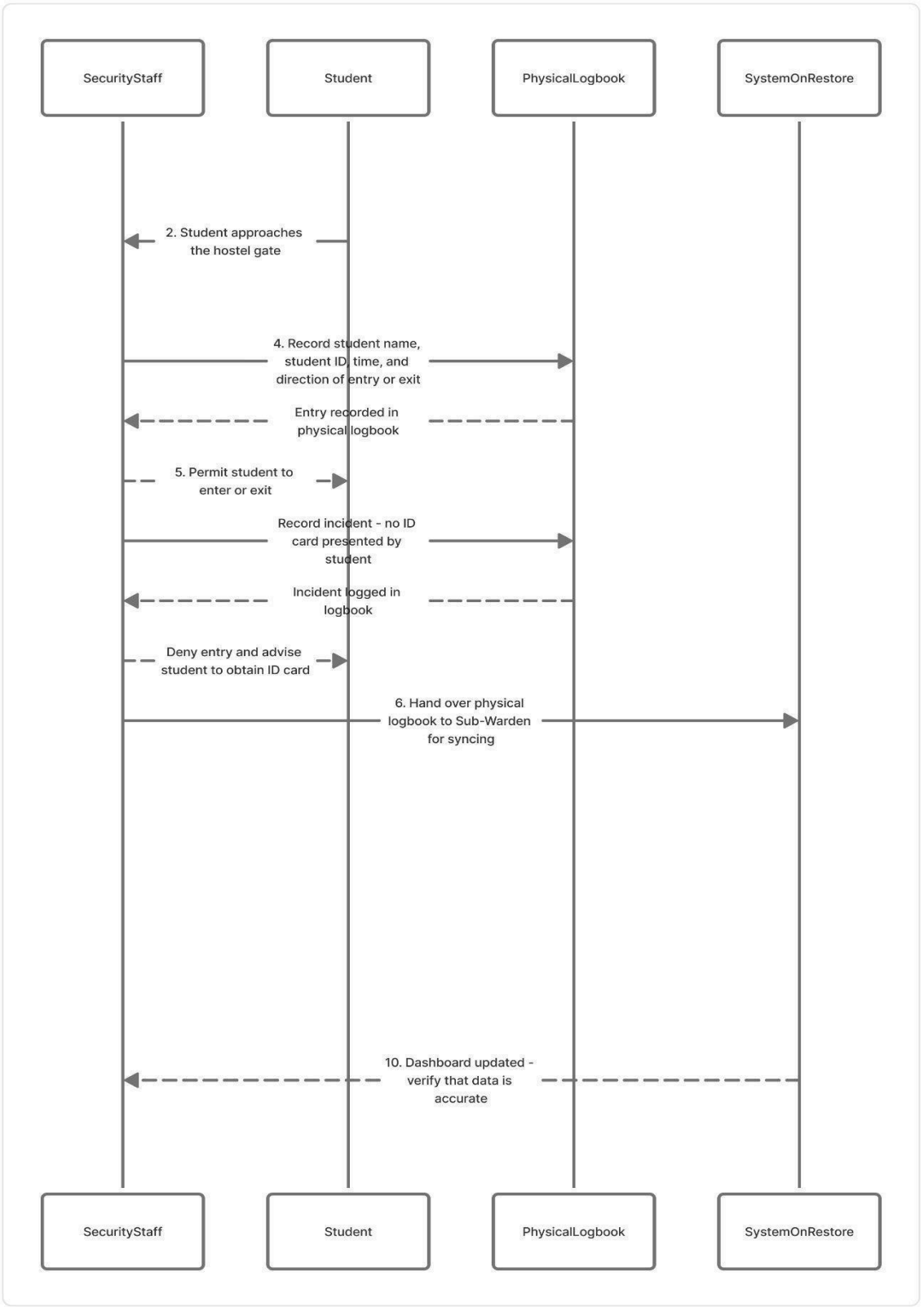


Figure 24: Maintain Physical Records

CHAPTER 5: UI/UX DESIGN

5.1 Wireframes / Screen Mock-ups for Key User Interfaces

The user interface of the **Hostel Hive System** has been designed using **Figma** to ensure a consistent, user-friendly, and responsive experience across all user roles. The design follows a role-based approach, where each stakeholder interacts with a customized interface based on their responsibilities.

All major subsystems identified in the SRS are included in the mockups, covering both web and mobile platforms.

Included UI Subsystems

The following key user interfaces have been designed:

1) Student Mobile Application (Flutter App)

The student-facing mobile application is the primary interface for hostel residents.

Key Screens:

- **Login & Authentication Screen**
    - Secure login using credentials
    - QR code generation after authentication
- **QR Code Display Screen**
    - Displays unique QR code for entry/exit scanning
    - Shows status (Active / Invalidated)
- **Attendance History Screen**
    - View past entry/exit logs
    - Timestamp and direction (IN/OUT)
- **Leave Request Screen**
    - Submit late entry or leave requests
    - Input fields: date, reason
- **Complaint Submission Screen**

    - Submit maintenance or service complaints
    - Upload images (linked to COMPLAINT_IMAGE entity)
- **Announcements Screen**
    - View notices published by the warden

2) Warden Web Dashboard (React Application)

The main administrative interface used by the **Warden and Sub-Warden**.

Key Screens:

- **Dashboard Overview**
    - Real-time student presence (IN/OUT)
    - Quick statistics (occupancy, complaints, requests)
- **Attendance Monitoring Panel**
    - Live attendance logs from ATTENDANCE_RECORD
    - Filter by student, time, or status
- **Leave Request Management**
    - Approve / Reject student requests
    - Add comments (linked to LATE_ENTRY_REQUEST)
- **Room & Bed Allocation Module**
    - Assign students to rooms (BED_ALLOCATION)
    - View room capacity and occupancy
- **Complaint Management System**
    - View complaints submitted by students
    - Assign complaints to staff
    - Update resolution status
- **Announcement Management**
    - Create and broadcast announcements
    - Target specific student groups

3) Security Staff Interface (Web / Tablet UI)

Used by security personnel at hostel entry points.

Key Screens:

- **QR Scan Verification Panel**
    - Displays scan results from ESP32 device
    - Shows student identity and status
- **Real-time Entry Log**
    - Immediate updates of student movements
- **Alert Notifications**
    - Invalid QR code alerts
    - Unauthorized access attempts

4) System Administrator Panel

Used for system-level configuration and maintenance.

Key Screens:

- **User Management**
    - Activate / deactivate users
    - Assign roles (Student, Warden, Security)
- **System Monitoring Dashboard**
    - View system health and logs
- **QR Code Management**
    - Regenerate or invalidate QR codes

5) IoT Device Interaction Interface (ESP32 Integration Layer)

Although not a traditional UI, the system includes an interface layer for hardware communication.

Functional Views:

- QR scan data transmission
- Offline data queue visualization (for debugging/admin)
- Sync status monitoring

Figma Design Access

The complete set of wireframes and high-fidelity mockups for all subsystems can be accessed via the following link:

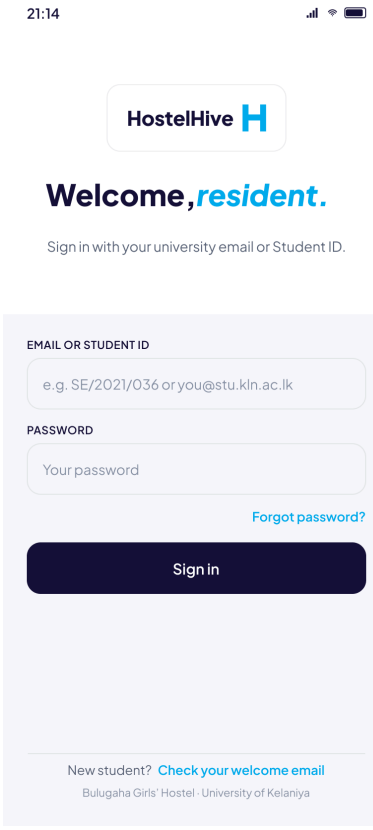
Figma Design: Figma ui link

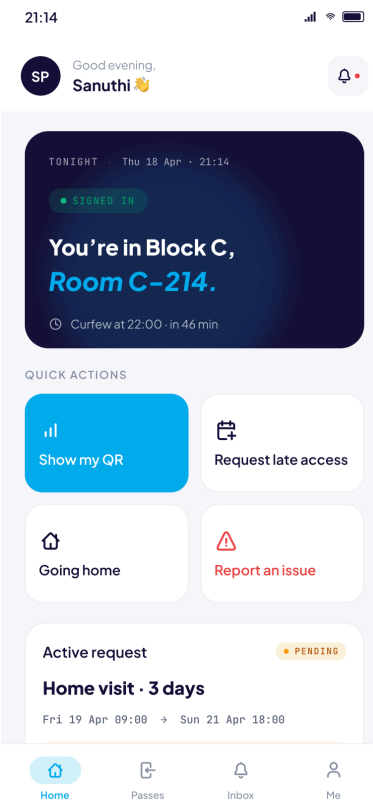
Design Considerations

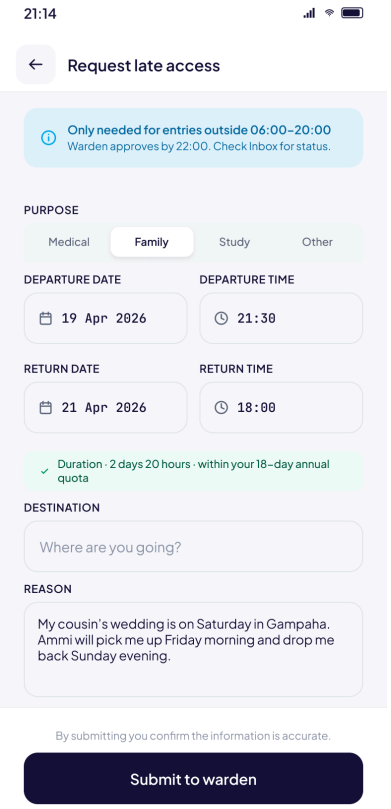
The UI design follows these principles:

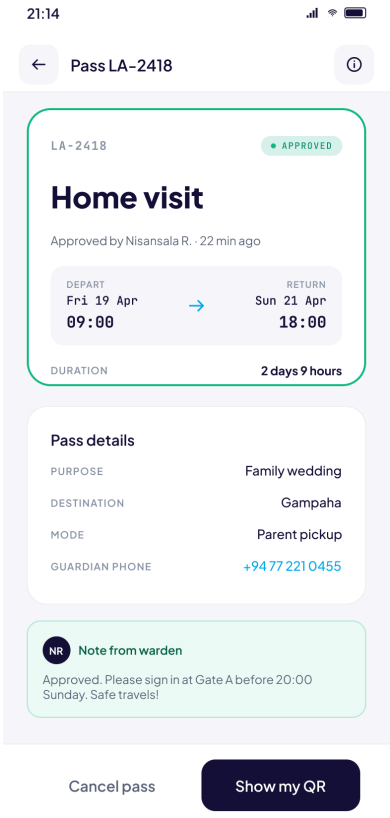
- **Role-Based Access Control (RBAC):**
  Each interface is tailored to specific stakeholders (Student, Warden, Security, Admin)
- **Usability & Simplicity:**
  Minimal steps for critical actions like QR scanning and complaint submission
- **Real-Time Feedback:**
  Instant updates for attendance and system alerts
- **Mobile-First Approach:**
  Student app optimized for quick QR access and interaction
- **Scalability:**
  UI components designed to support future feature extensions

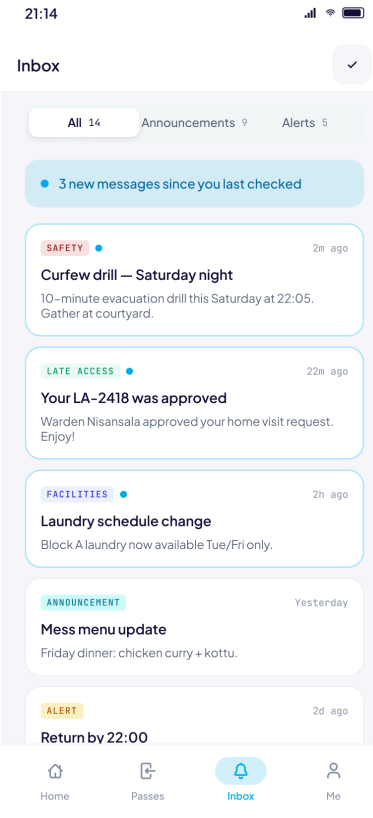
Student Screens

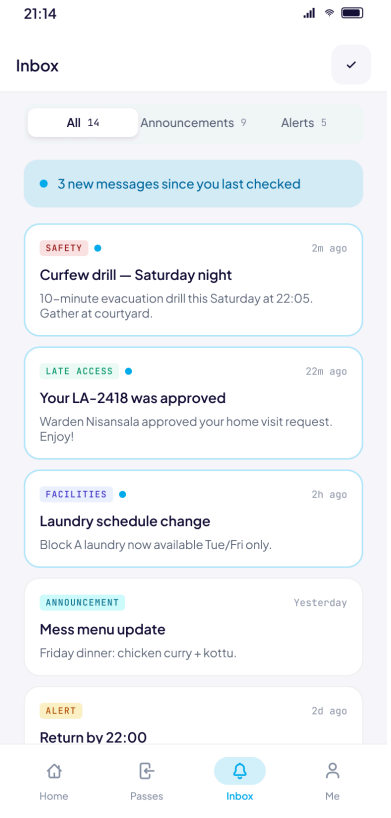




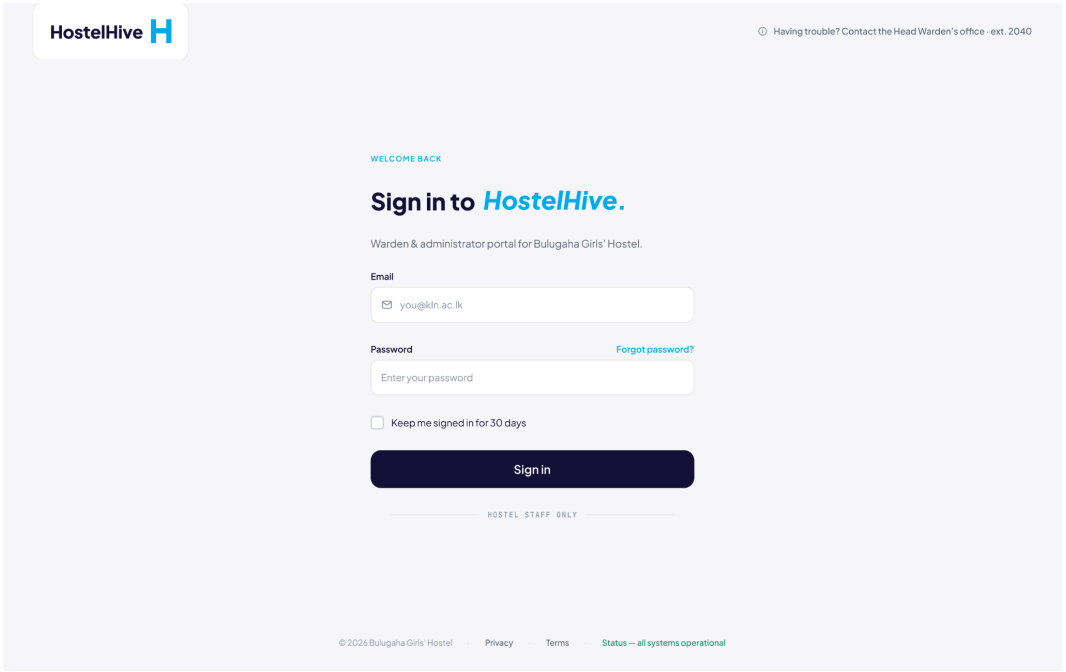


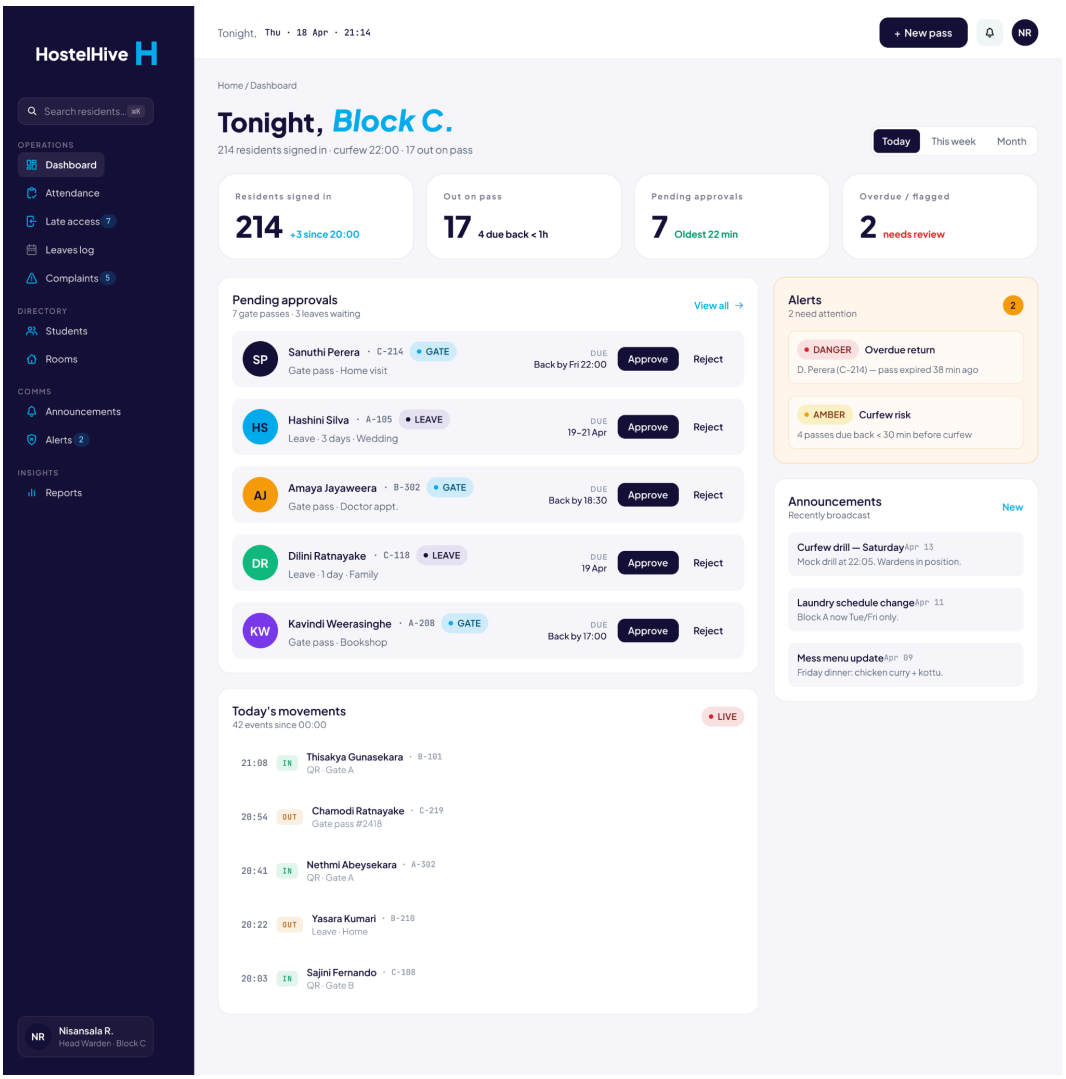


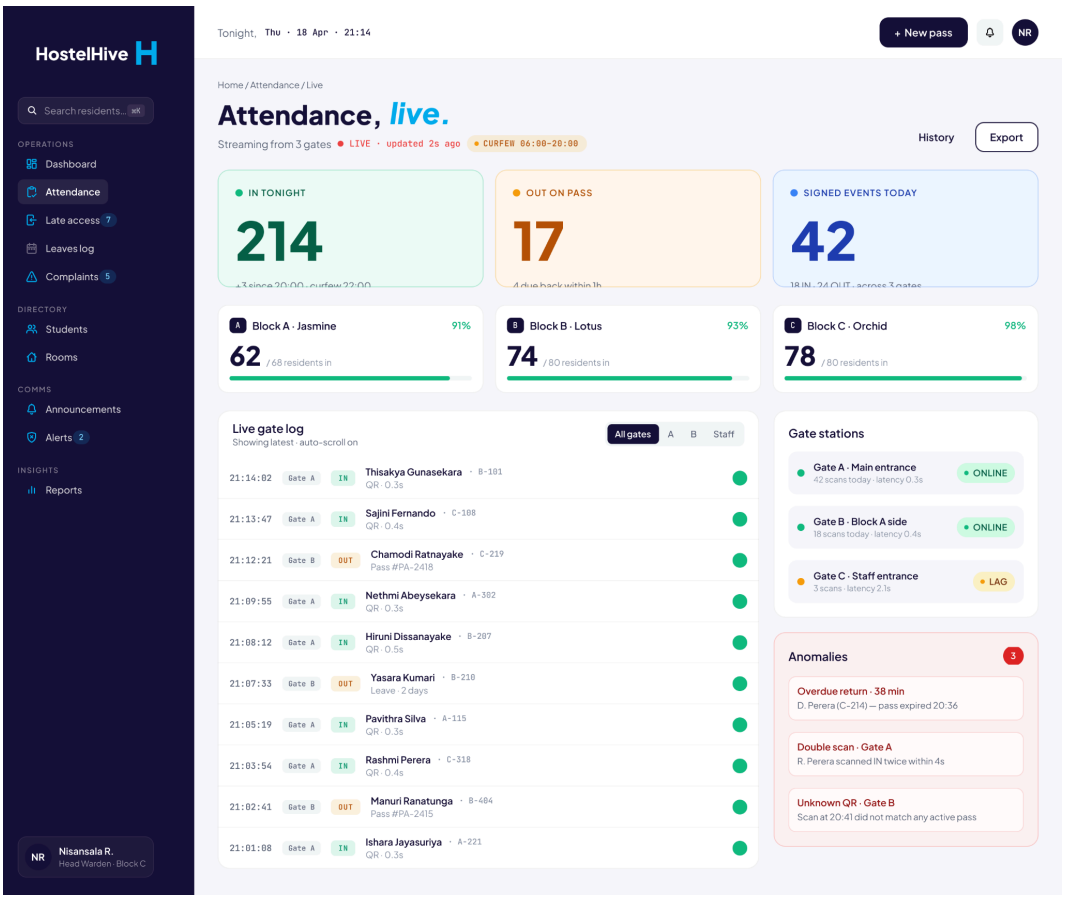


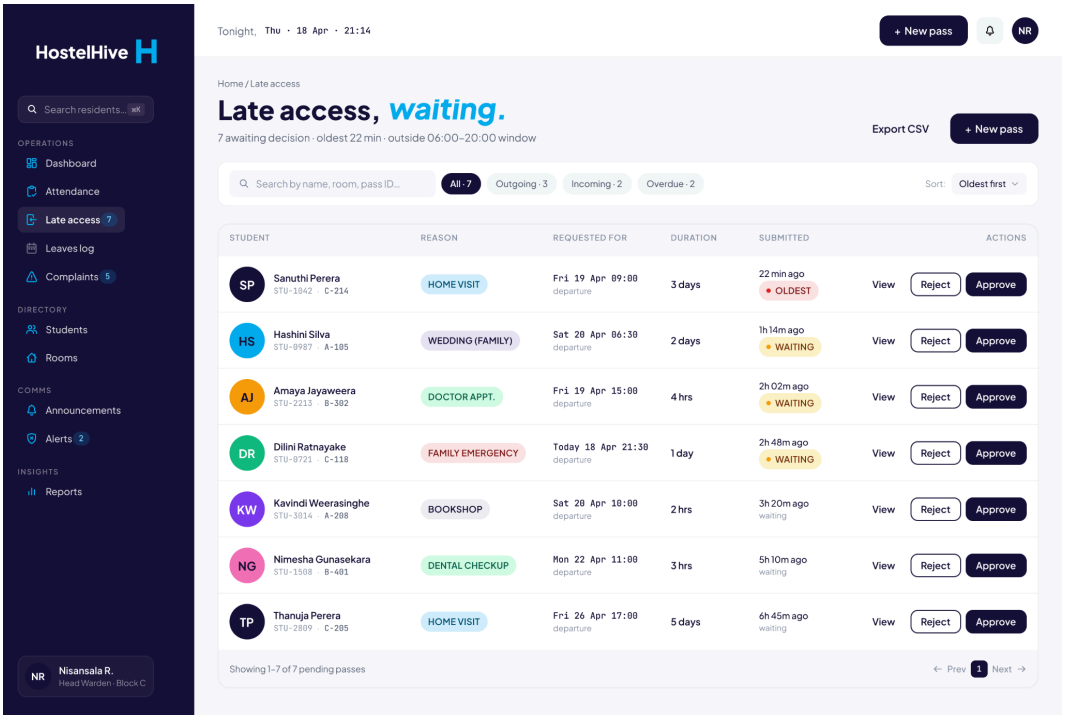


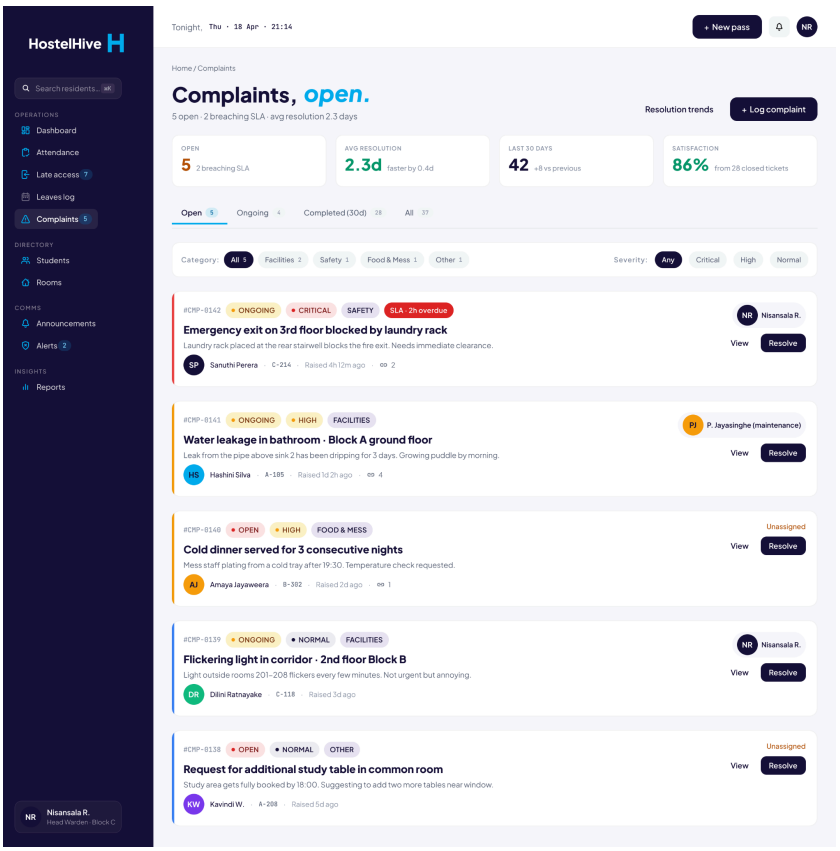
Warden/Subwarden

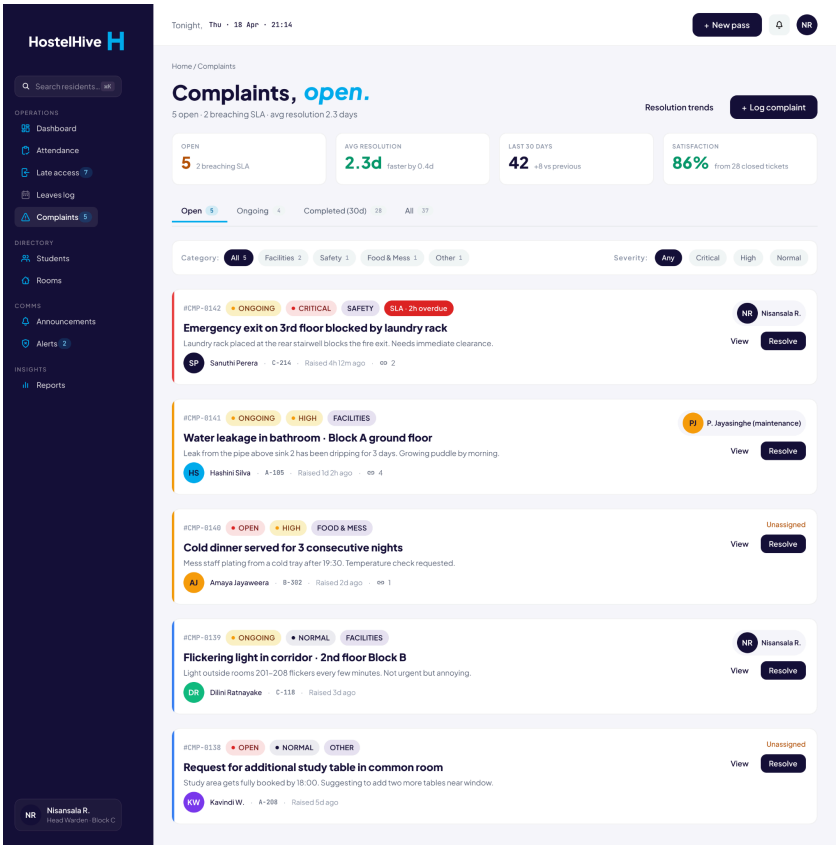


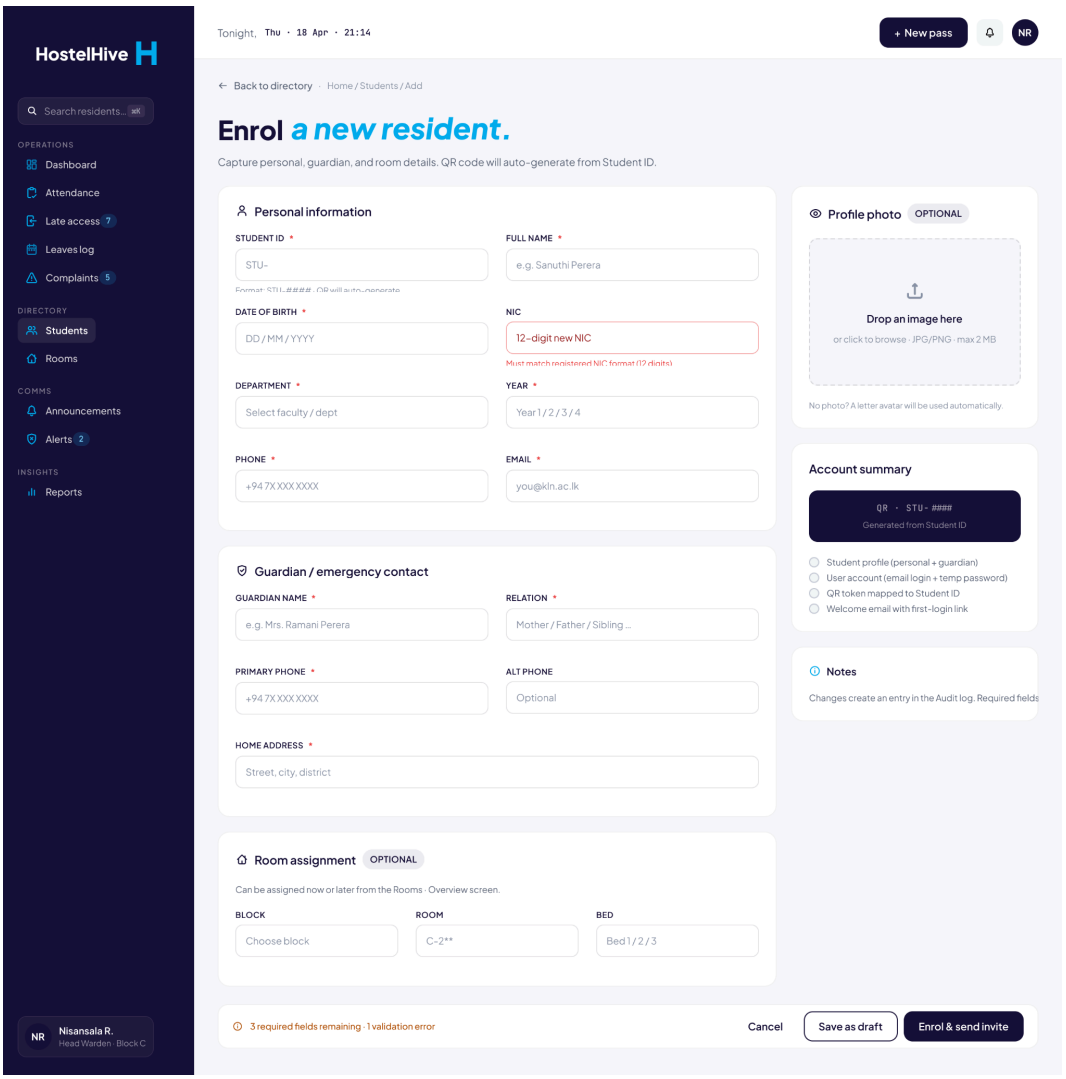

HostelHive H
Search residents... ⌘K
OPERATIONS
Dashboard
Attendance
Late access 7
Leaves log
Complaints 5
DIRECTORY
Students
Rooms
COMMS
Announcements
Alerts 2
INSIGHTS
Reports
NR
Nisansala R.
Head Warden · Block C
Tonight, Thu · 18 Apr · 21:14
+ New pass
NR
Home / Dashboard
Tonight, Block C.
214 residents signed in · curfew 22:00 · 17 out on pass
Today
This week
Month
Residents signed in
214 +3 since 20:00
Out on pass
17 4 due back < 1h
Pending approvals
7 Oldest 22 min
Overdue / flagged
2 needs review
Pending approvals
7 gate passes · 3 leaves waiting
View all →
SP
Sanuthi Perera · C-214 GATE
Gate pass · Home visit
DUE
Back by Fri 22:00
Approve
Reject
HS
Hashini Silva · A-105 LEAVE
Leave · 3 days · Wedding
DUE
19–21 Apr
Approve
Reject
AJ
Amaya Jayaweera · B-302 GATE
Gate pass · Doctor appt.
DUE
Back by 18:30
Approve
Reject
DR
Dilini Ratnayake · C-118 LEAVE
Leave · 1 day · Family
DUE
19 Apr
Approve
Reject
KW
Kavindi Weerasinghe · A-208 GATE
Gate pass · Bookshop
DUE
Back by 17:00
Approve
Reject
Today's movements
42 events since 00:00
LIVE
21:08 IN Thisakya Gunasekara · B-101
QR · Gate A
20:54 OUT Chamodi Ratnayake · C-219
Gate pass #2418
20:41 IN Nethmi Abeysekara · A-302
QR · Gate A
20:22 OUT Yasara Kumari · B-210
Leave · Home
20:03 IN Sajini Fernando · C-108
QR · Gate B
Alerts
2 need attention
2
DANGER Overdue return
D. Perera (C-214) — pass expired 38 min ago
AMBER Curfew risk
4 passes due back < 30 min before curfew
Announcements
Recently broadcast
New
Curfew drill — Saturday Apr 13
Mock drill at 22:05. Wardens in position.
Laundry schedule change Apr 11
Block A now Tue/Fri only.
Mess menu update Apr 09
Friday dinner: chicken curry + kottu.

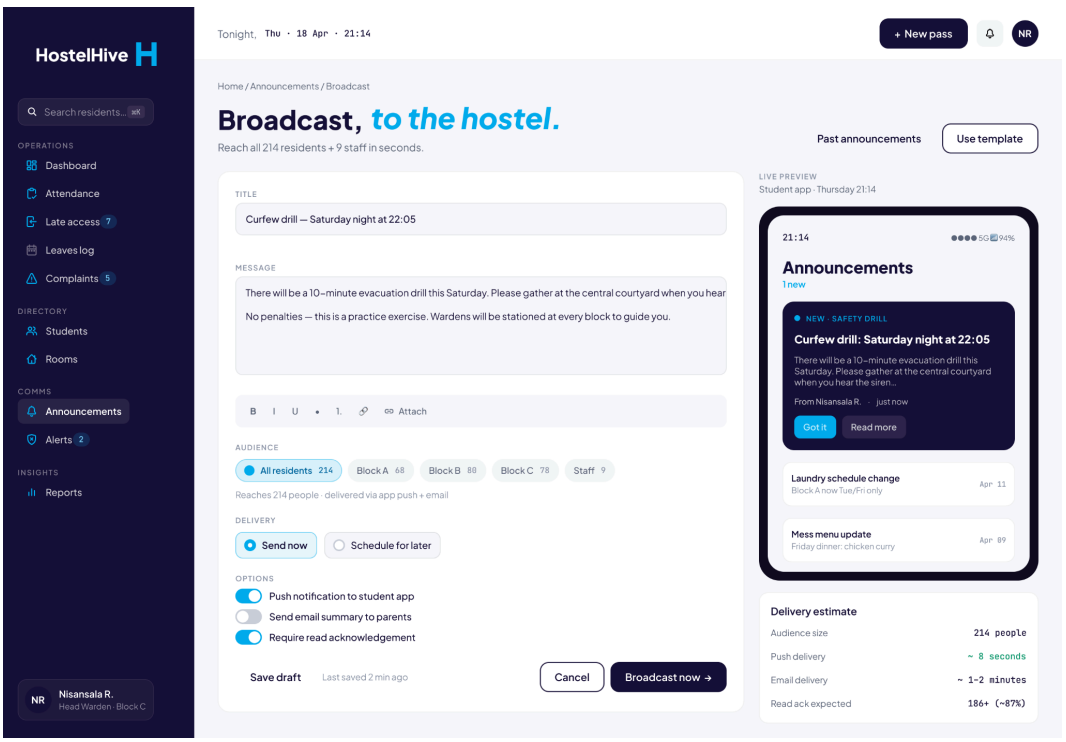

HostelHive H
Search residents... ⌘K
OPERATIONS
Dashboard
Attendance
Late access 7
Leaves log
Complaints 5
DIRECTORY
Students
Rooms
COMMS
Announcements
Alerts 2
INSIGHTS
Reports
NR Nisansala R. Head Warden · Block C
Tonight, Thu · 18 Apr · 21:14
+ New pass
NR
Home / Attendance / Live
Attendance, live.
Streaming from 3 gates LIVE · updated 2s ago CURFEW 06:00–20:00
History
Export
IN TONIGHT
214
+3 since 20:00 · curfew 22:00
OUT ON PASS
17
4 due back within 1h
SIGNED EVENTS TODAY
42
18 IN · 24 OUT · across 3 gates
A Block A · Jasmine 91%
62 / 68 residents in
B Block B · Lotus 93%
74 / 80 residents in
C Block C · Orchid 98%
78 / 80 residents in
Live gate log
Showing latest · auto-scroll on
All gates A B Staff
21:14:02 Gate A IN Thisakya Gunasekara · B-101 QR · 0.3s
21:13:47 Gate A IN Sajini Fernando · C-108 QR · 0.4s
21:12:21 Gate B OUT Chamodi Ratnayake · C-219 Pass #PA-2418
21:09:55 Gate A IN Nethmi Abeysekara · A-302 QR · 0.3s
21:08:12 Gate A IN Hiruni Dissanayake · B-207 QR · 0.5s
21:07:33 Gate B OUT Yasara Kumari · B-210 Leave · 2 days
21:05:19 Gate A IN Pavithra Silva · A-115 QR · 0.3s
21:03:54 Gate A IN Rashmi Perera · C-318 QR · 0.4s
21:02:41 Gate B OUT Manuri Ranatunga · B-404 Pass #PA-2415
21:01:08 Gate A IN Ishara Jayasuriya · A-221 QR · 0.3s
Gate stations
Gate A · Main entrance 42 scans today · latency 0.3s ONLINE
Gate B · Block A side 18 scans today · latency 0.4s ONLINE
Gate C · Staff entrance 3 scans · latency 2.1s LAG
Anomalies 3
Overdue return · 38 min
D. Perera (C-214) — pass expired 20:36
Double scan · Gate A
R. Perera scanned IN twice within 4s
Unknown QR · Gate B
Scan at 20:41 did not match any active pass


HostelHive H
Search residents... ⌘K
OPERATIONS
Dashboard
Attendance
Late access 7
Leaves log
Complaints 5
DIRECTORY
Students
Rooms
COMMS
Announcements
Alerts 2
INSIGHTS
Reports
NR Nisansala R. Head Warden · Block C
Tonight, Thu · 18 Apr · 21:14
+ New pass
NR
Home / Late access
Late access, waiting.
7 awaiting decision · oldest 22 min · outside 06:00–20:00 window
Export CSV
+ New pass
Search by name, room, pass ID...
All · 7 Outgoing · 3 Incoming · 2 Overdue · 2
Sort: Oldest first
STUDENT REASON REQUESTED FOR DURATION SUBMITTED ACTIONS
SP Sanuthi Perera STU-1042 · C-214 HOME VISIT Fri 19 Apr 09:00 departure 3 days 22 min ago OLDEST View Reject Approve
HS Hashini Silva STU-0987 · A-105 WEDDING (FAMILY) Sat 20 Apr 06:30 departure 2 days 1h 14m ago WAITING View Reject Approve
AJ Amaya Jayaweera STU-2213 · B-302 DOCTOR APPT. Fri 19 Apr 15:00 departure 4 hrs 2h 02m ago WAITING View Reject Approve
DR Dilini Ratnayake STU-0721 · C-118 FAMILY EMERGENCY Today 18 Apr 21:30 departure 1 day 2h 48m ago WAITING View Reject Approve
KW Kavindi Weerasinghe STU-3014 · A-208 BOOKSHOP Sat 20 Apr 10:00 departure 2 hrs 3h 20m ago waiting View Reject Approve
NG Nimesha Gunasekara STU-1508 · B-401 DENTAL CHECKUP Mon 22 Apr 11:00 departure 3 hrs 5h 10m ago waiting View Reject Approve
TP Thanuja Perera STU-2809 · C-205 HOME VISIT Fri 26 Apr 17:00 departure 5 days 6h 45m ago waiting View Reject Approve
Showing 1–7 of 7 pending passes
← Prev 1 Next →

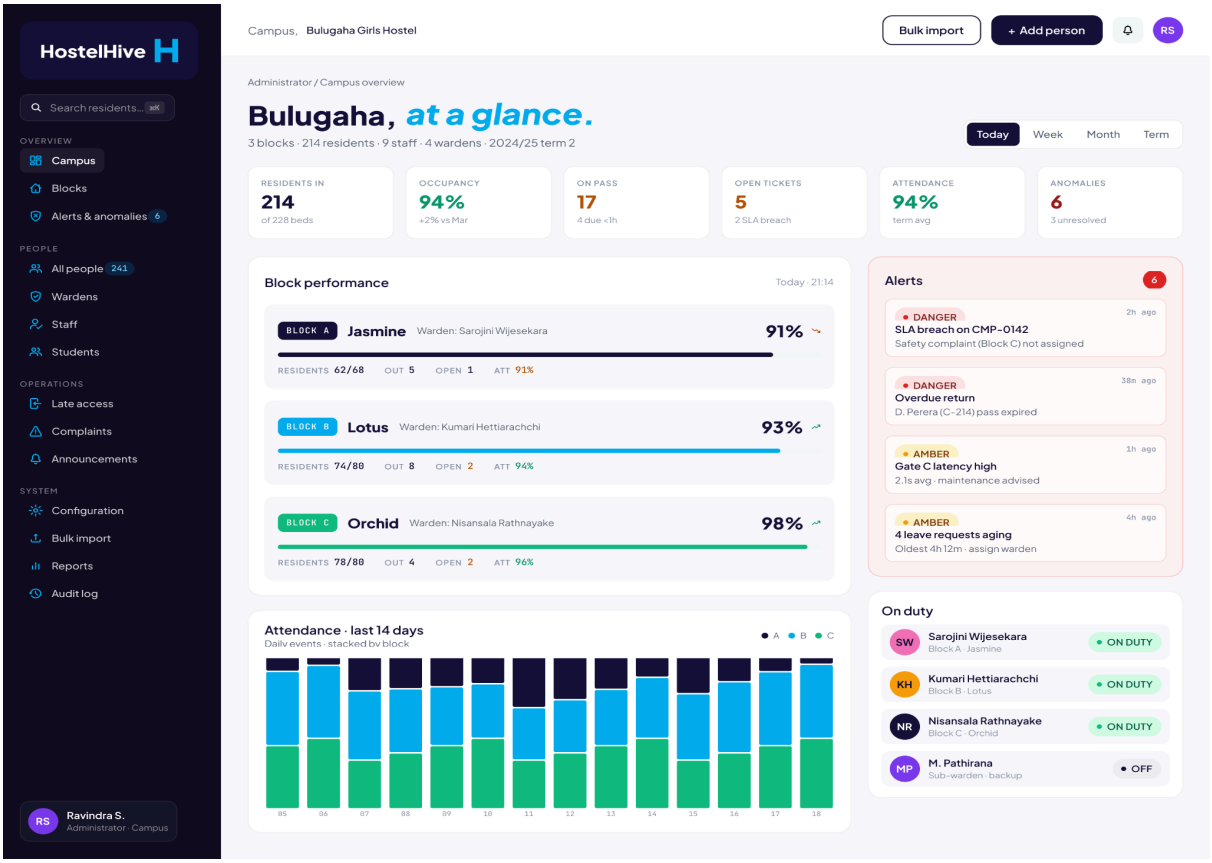

HostelHive
Search residents... ⌘K
OPERATIONS
Dashboard
Attendance
Late access 7
Leaves log
Complaints 5
DIRECTORY
Students
Rooms
COMMS
Announcements
Alerts 2
INSIGHTS
Reports
NR Nisansala R. Head Warden · Block C
Tonight. Thu · 18 Apr · 21:14
+ New pass
NR
Home / Complaints
Complaints, open.
5 open · 2 breaching SLA · avg resolution 2.3 days
Resolution trends
+ Log complaint
OPEN 5 2 breaching SLA
AVG RESOLUTION 2.3d faster by 0.4d
LAST 30 DAYS 42 +8 vs previous
SATISFACTION 86% from 28 closed tickets
Open 5 Ongoing 4 Completed (30d) 28 All 37
Category: All 5 Facilities 2 Safety 1 Food & Mess 1 Other 1
Severity: Any Critical High Normal
#CMP-0142 ONGOING CRITICAL SAFETY SLA -2h overdue
Emergency exit on 3rd floor blocked by laundry rack
Laundry rack placed at the rear stairwell blocks the fire exit. Needs immediate clearance.
SP Sanuthi Perera · C-214 · Raised 4h 12m ago · 2
NR Nisansala R.
View Resolve
#CMP-0141 ONGOING HIGH FACILITIES
Water leakage in bathroom · Block A ground floor
Leak from the pipe above sink 2 has been dripping for 3 days. Growing puddle by morning.
HS Hashini Silva · A-105 · Raised 1d 2h ago · 4
PJ P. Jayasinghe (maintenance)
View Resolve
#CMP-0140 OPEN HIGH FOOD & MESS
Cold dinner served for 3 consecutive nights
Mess staff plating from a cold tray after 19:30. Temperature check requested.
AJ Amaya Jayaweera · B-302 · Raised 2d ago · 1
Unassigned
View Resolve
#CMP-0139 ONGOING NORMAL FACILITIES
Flickering light in corridor · 2nd floor Block B
Light outside rooms 201–208 flickers every few minutes. Not urgent but annoying.
DR Dilini Ratnayake · C-118 · Raised 3d ago
NR Nisansala R.
View Resolve
#CMP-0138 OPEN NORMAL OTHER
Request for additional study table in common room
Study area gets fully booked by 18:00. Suggesting to add two more tables near window.
KW Kavindi W. · A-208 · Raised 5d ago
Unassigned
View Resolve

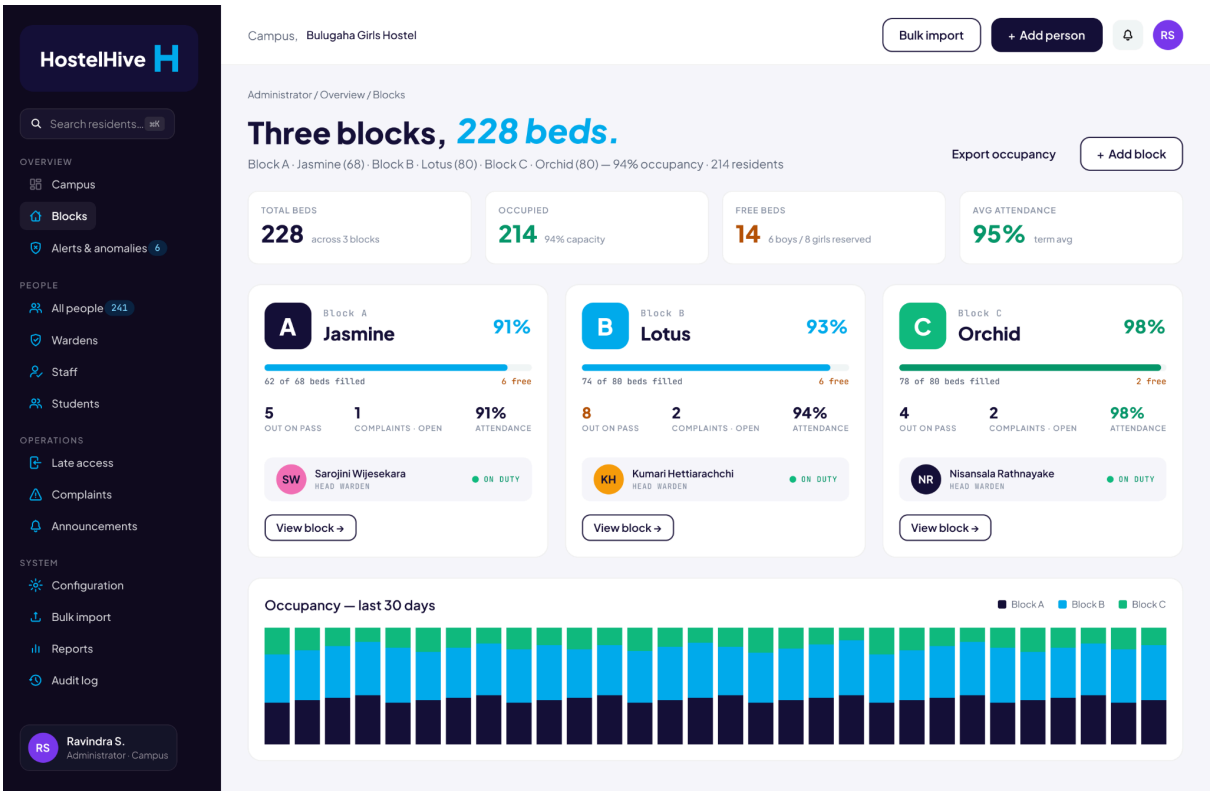


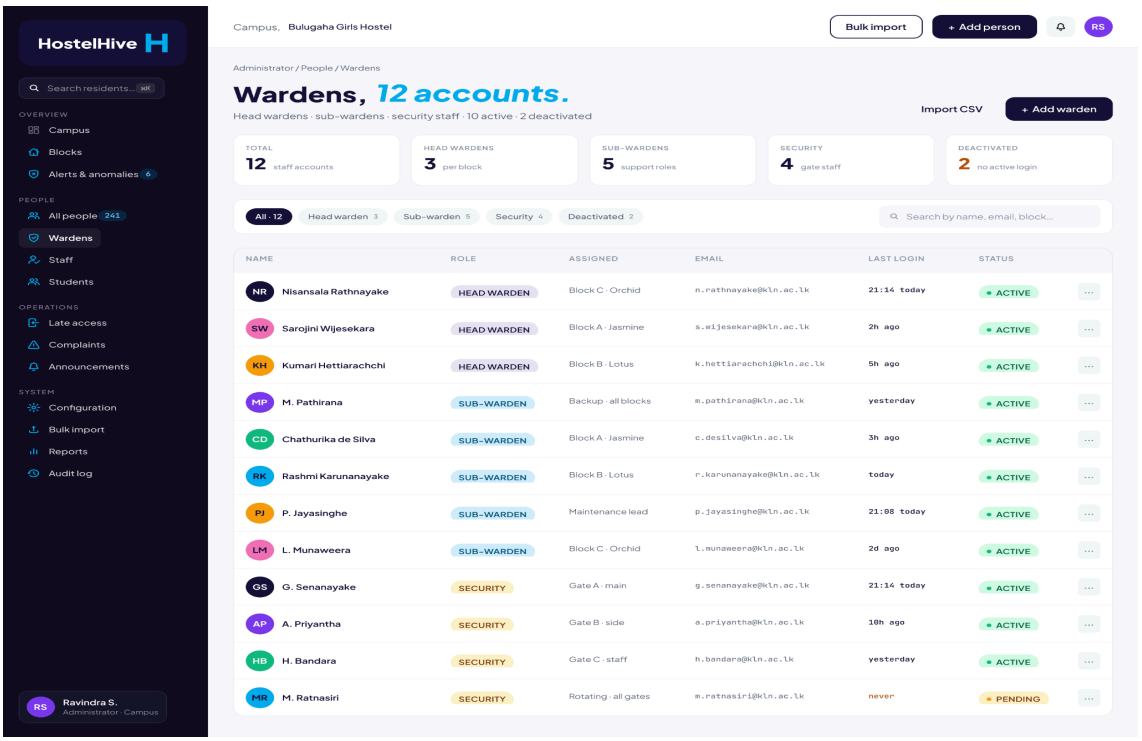

HostelHive
Search residents... ⌘K
OPERATIONS
Dashboard
Attendance
Late access 7
Leaves log
Complaints 5
DIRECTORY
Students
Rooms
COMMS
Announcements
Alerts 2
INSIGHTS
Reports
NR
Nisansala R.
Head Warden · Block C
Tonight, Thu · 18 Apr · 21:14
+ New pass
NR
Back to directory · Home / Students / Add
Enrol a new resident.
Capture personal, guardian, and room details. QR code will auto-generate from Student ID.
Personal information
STUDENT ID *
STU-
Format: STU-####. QR will auto-generate.
FULL NAME *
e.g. Sanuthi Perera
DATE OF BIRTH *
DD / MM / YYYY
NIC
12-digit new NIC
Must match registered NIC format (12 digits)
DEPARTMENT *
Select faculty / dept
YEAR *
Year 1 / 2 / 3 / 4
PHONE *
+94 7X XXX XXXX
EMAIL *
you@kln.ac.lk
Guardian / emergency contact
GUARDIAN NAME *
e.g. Mrs. Ramani Perera
RELATION *
Mother / Father / Sibling ...
PRIMARY PHONE *
+94 7X XXX XXXX
ALT PHONE
Optional
HOME ADDRESS *
Street, city, district
Room assignment OPTIONAL
Can be assigned now or later from the Rooms · Overview screen.
BLOCK
Choose block
ROOM
C-2**
BED
Bed 1 / 2 / 3
Profile photo OPTIONAL
Drop an image here
or click to browse · JPG/PNG · max 2 MB
No photo? A letter avatar will be used automatically.
Account summary
QR · STU-####
Generated from Student ID
Student profile (personal + guardian)
User account (email login + temp password)
QR token mapped to Student ID
Welcome email with first-login link
Notes
Changes create an entry in the Audit log. Required fields
3 required fields remaining · 1 validation error
Cancel
Save as draft
Enrol & send invite

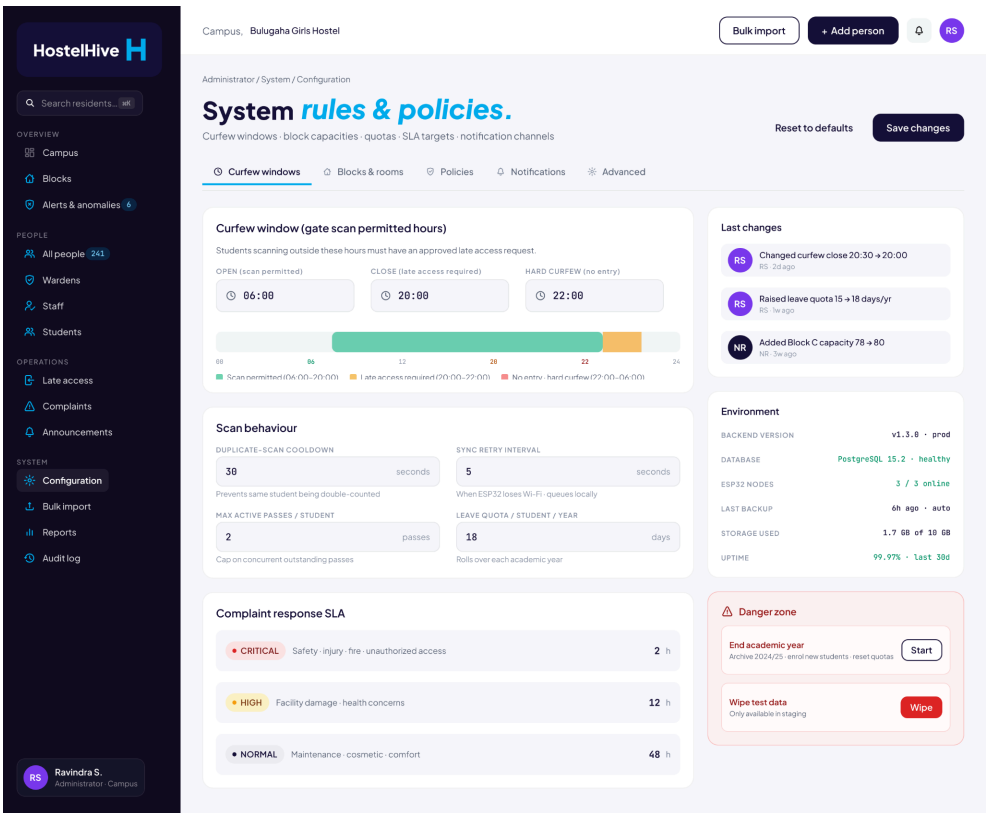

HostelHive
Search residents... ⌘K
OPERATIONS
Dashboard
Attendance
Late access 7
Leaves log
Complaints 5
DIRECTORY
Students
Rooms
COMMS
Announcements
Alerts 2
INSIGHTS
Reports
NR
Nisansala R.
Head Warden · Block C
Tonight, Thu · 18 Apr · 21:14
+ New pass
NR
Home / Announcements / Broadcast
Broadcast, to the hostel.
Reach all 214 residents + 9 staff in seconds.
Past announcements
Use template
TITLE
Curfew drill — Saturday night at 22:05
MESSAGE
There will be a 10-minute evacuation drill this Saturday. Please gather at the central courtyard when you hear
No penalties — this is a practice exercise. Wardens will be stationed at every block to guide you.
B I U • 1. Attach
AUDIENCE
All residents 214
Block A 68
Block B 80
Block C 78
Staff 9
Reaches 214 people · delivered via app push + email
DELIVERY
Send now
Schedule for later
OPTIONS
Push notification to student app
Send email summary to parents
Require read acknowledgement
Save draft
Last saved 2 min ago
Cancel
Broadcast now →
LIVE PREVIEW
Student app · Thursday 21:14
21:14
Announcements
1 new
NEW · SAFETY DRILL
Curfew drill: Saturday night at 22:05
There will be a 10-minute evacuation drill this Saturday. Please gather at the central courtyard when you hear the siren...
From Nisansala R. · just now
Got it
Read more
Laundry schedule change
Block A now Tue/Fri only
Apr 11
Mess menu update
Friday dinner: chicken curry
Apr 09
Delivery estimate
Audience size 214 people
Push delivery ~ 8 seconds
Email delivery ~ 1-2 minutes
Read ack expected 186+ (~87%)

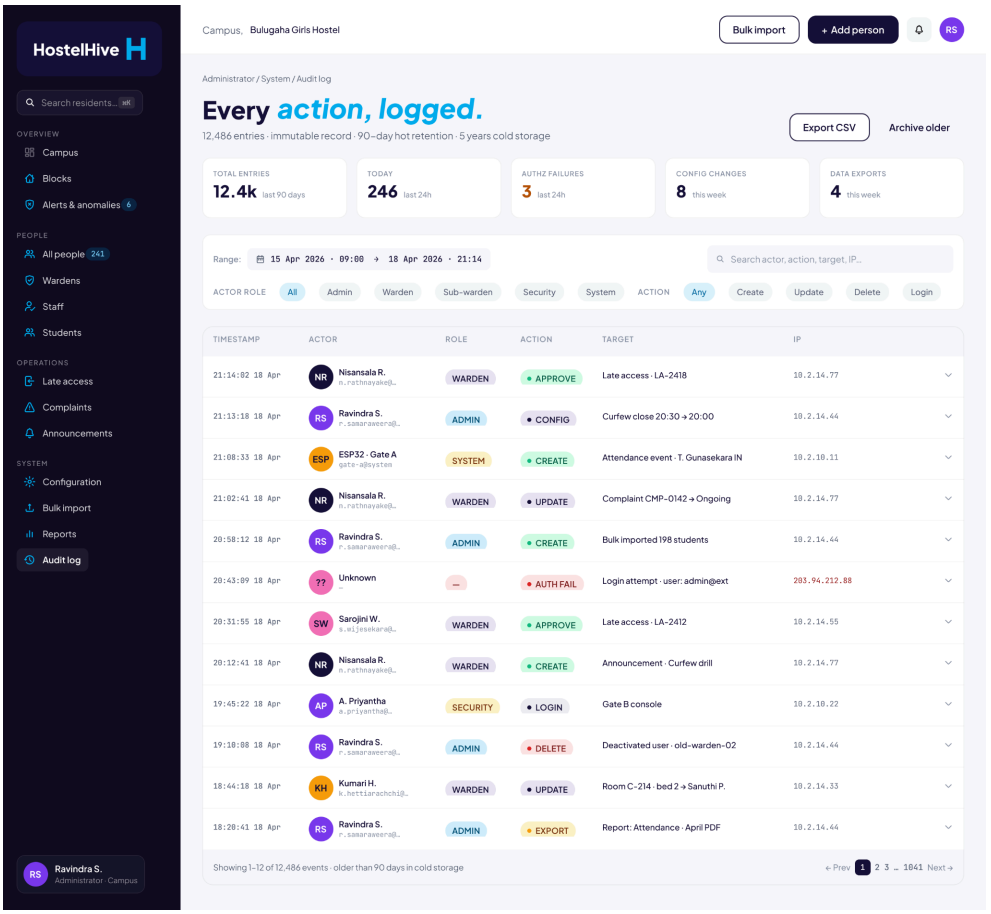
Administrator



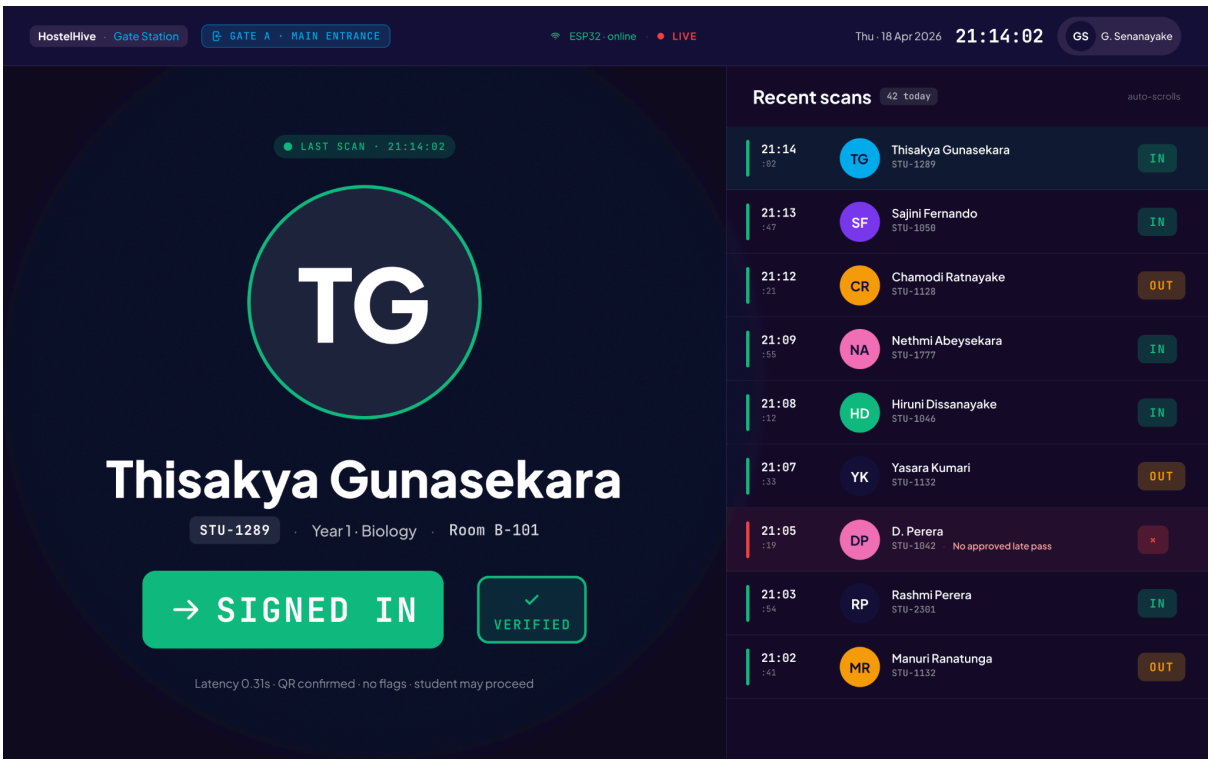



HostelHive
Search residents... ⌘K
OVERVIEW
Campus
Blocks
Alerts & anomalies 6
PEOPLE
All people 241
Wardens
Staff
Students
OPERATIONS
Late access
Complaints
Announcements
SYSTEM
Configuration
Bulk import
Reports
Audit log
RS Ravindra S. Administrator · Campus
Campus, Bulugaha Girls Hostel
Bulk import
+ Add person
RS
Administrator / People / Wardens
Wardens, 12 accounts.
Head wardens · sub-wardens · security staff · 10 active · 2 deactivated
Import CSV
+ Add warden
TOTAL 12 staff accounts
HEAD WARDENS 3 per block
SUB-WARDENS 5 support roles
SECURITY 4 gate staff
DEACTIVATED 2 no active login
All · 12
Head warden 3
Sub-warden 5
Security 4
Deactivated 2
Search by name, email, block...
NAME ROLE ASSIGNED EMAIL LAST LOGIN STATUS
NR Nisansala Rathnayake HEAD WARDEN Block C · Orchid n.rathnayake@kln.ac.lk 21:14 today ACTIVE
SW Sarojini Wijesekara HEAD WARDEN Block A · Jasmine s.wijesekara@kln.ac.lk 2h ago ACTIVE
KH Kumari Hettiarachchi HEAD WARDEN Block B · Lotus k.hettiarachchi@kln.ac.lk 5h ago ACTIVE
MP M. Pathirana SUB-WARDEN Backup · all blocks m.pathirana@kln.ac.lk yesterday ACTIVE
CD Chathurika de Silva SUB-WARDEN Block A · Jasmine c.desilva@kln.ac.lk 3h ago ACTIVE
RK Rashmi Karunanayake SUB-WARDEN Block B · Lotus r.karunanayake@kln.ac.lk today ACTIVE
PJ P. Jayasinghe SUB-WARDEN Maintenance lead p.jayasinghe@kln.ac.lk 21:08 today ACTIVE
LM L. Munaweera SUB-WARDEN Block C · Orchid l.munaweera@kln.ac.lk 2d ago ACTIVE
GS G. Senanayake SECURITY Gate A · main g.senanayake@kln.ac.lk 21:14 today ACTIVE
AP A. Priyantha SECURITY Gate B · side a.priyantha@kln.ac.lk 10h ago ACTIVE
HB H. Bandara SECURITY Gate C · staff h.bandara@kln.ac.lk yesterday ACTIVE
MR M. Ratnasiri SECURITY Rotating · all gates m.ratnasiri@kln.ac.lk never PENDING


HostelHive
Search residents… ⌘K
OVERVIEW
Campus
Blocks
Alerts & anomalies 6
PEOPLE
All people 241
Wardens
Staff
Students
OPERATIONS
Late access
Complaints
Announcements
SYSTEM
Configuration
Bulk import
Reports
Audit log
Ravindra S.
Administrator · Campus
Campus, Bulugaha Girls Hostel
Bulk import
+ Add person
RS
Administrator / System / Configuration
System rules & policies.
Curfew windows · block capacities · quotas · SLA targets · notification channels
Reset to defaults
Save changes
Curfew windows
Blocks & rooms
Policies
Notifications
Advanced
Curfew window (gate scan permitted hours)
Students scanning outside these hours must have an approved late access request.
OPEN (scan permitted)
06:00
CLOSE (late access required)
20:00
HARD CURFEW (no entry)
22:00
00
06
12
20
22
24
Scan permitted (06:00–20:00)
Late access required (20:00–22:00)
No entry · hard curfew (22:00–06:00)
Last changes
Changed curfew close 20:30 → 20:00
RS · 2d ago
Raised leave quota 15 → 18 days/yr
RS · 1w ago
Added Block C capacity 78 → 80
NR · 3w ago
Scan behaviour
DUPLICATE-SCAN COOLDOWN
30
seconds
Prevents same student being double-counted
SYNC RETRY INTERVAL
5
seconds
When ESP32 loses Wi-Fi · queues locally
MAX ACTIVE PASSES / STUDENT
2
passes
Cap on concurrent outstanding passes
LEAVE QUOTA / STUDENT / YEAR
18
days
Rolls over each academic year
Environment
BACKEND VERSION
v1.3.0 · prod
DATABASE
PostgreSQL 15.2 · healthy
ESP32 NODES
3 / 3 online
LAST BACKUP
6h ago · auto
STORAGE USED
1.7 GB of 10 GB
UPTIME
99.97% · last 30d
Complaint response SLA
CRITICAL
Safety · injury · fire · unauthorized access
2 h
HIGH
Facility damage · health concerns
12 h
NORMAL
Maintenance · cosmetic · comfort
48 h
Danger zone
End academic year
Archive 2024/25 · enrol new students · reset quotas
Start
Wipe test data
Only available in staging
Wipe


HostelHive
Search residents… ⌘K
OVERVIEW
Campus
Blocks
Alerts & anomalies 6
PEOPLE
All people 241
Wardens
Staff
Students
OPERATIONS
Late access
Complaints
Announcements
SYSTEM
Configuration
Bulk import
Reports
Audit log
Ravindra S.
Administrator · Campus
Campus, Bulugaha Girls Hostel
Bulk import
+ Add person
RS
Administrator / System / Audit log
Every action, logged.
12,486 entries · immutable record · 90-day hot retention · 5 years cold storage
Export CSV
Archive older
TOTAL ENTRIES
12.4k
last 90 days
TODAY
246
last 24h
AUTHZ FAILURES
3
last 24h
CONFIG CHANGES
8
this week
DATA EXPORTS
4
this week
Range: 15 Apr 2026 · 09:00 → 18 Apr 2026 · 21:14
Search actor, action, target, IP…
ACTOR ROLE
All
Admin
Warden
Sub-warden
Security
System
ACTION
Any
Create
Update
Delete
Login
TIMESTAMP
ACTOR
ROLE
ACTION
TARGET
IP
21:14:02 18 Apr
Nisansala R.
WARDEN
APPROVE
Late access · LA-2418
10.2.14.77
21:13:18 18 Apr
Ravindra S.
ADMIN
CONFIG
Curfew close 20:30 → 20:00
10.2.14.44
21:08:33 18 Apr
ESP32 · Gate A
SYSTEM
CREATE
Attendance event · T. Gunasekara IN
10.2.10.11
21:02:41 18 Apr
Nisansala R.
WARDEN
UPDATE
Complaint CMP-0142 → Ongoing
10.2.14.77
20:58:12 18 Apr
Ravindra S.
ADMIN
CREATE
Bulk imported 198 students
10.2.14.44
20:43:09 18 Apr
Unknown
AUTH FAIL
Login attempt · user: admin@ext
203.94.212.88
20:31:55 18 Apr
Sarojini W.
WARDEN
APPROVE
Late access · LA-2412
10.2.14.55
20:12:41 18 Apr
Nisansala R.
WARDEN
CREATE
Announcement · Curfew drill
10.2.14.77
19:45:22 18 Apr
A. Priyantha
SECURITY
LOGIN
Gate B console
10.2.10.22
19:10:08 18 Apr
Ravindra S.
ADMIN
DELETE
Deactivated user · old-warden-02
10.2.14.44
18:44:18 18 Apr
Kumari H.
WARDEN
UPDATE
Room C-214 · bed 2 → Sanuthi P.
10.2.14.33
18:20:41 18 Apr
Ravindra S.
ADMIN
EXPORT
Report: Attendance · April PDF
10.2.14.44
Showing 1–12 of 12,486 events · older than 90 days in cold storage
← Prev
1
2
3
… 1041
Next →

Security Staff



User Flows

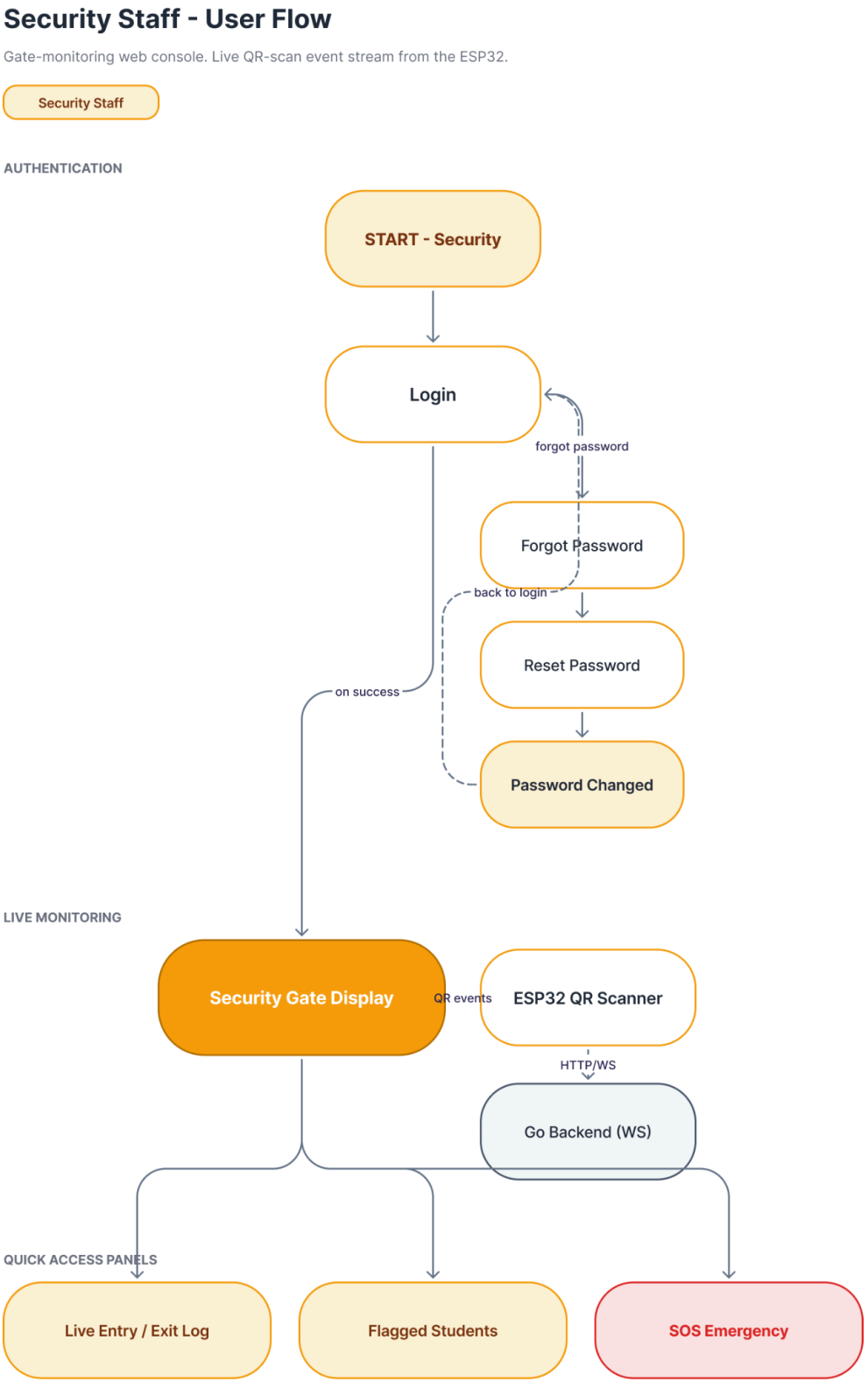
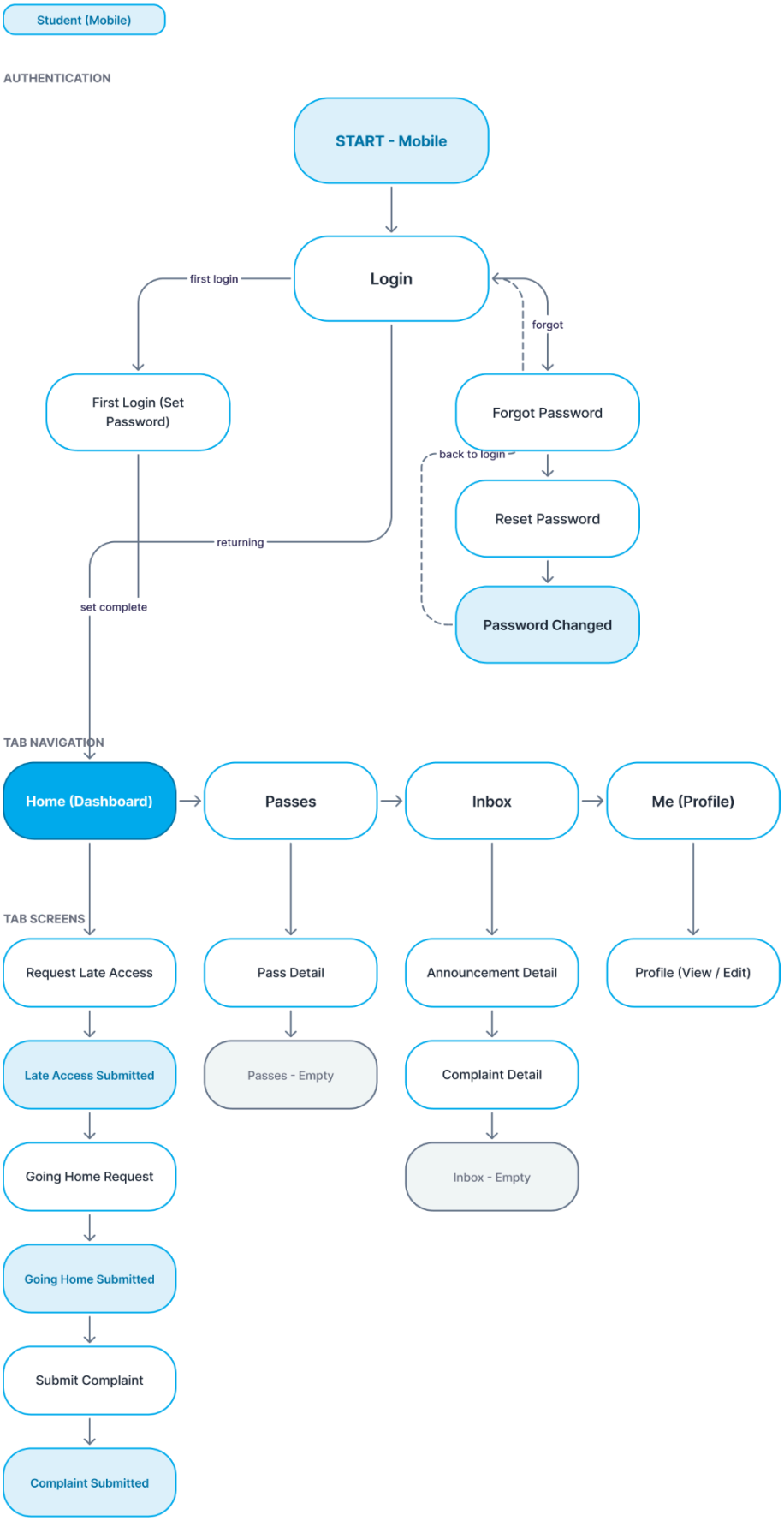


Figure 25 : Security Staff - User Flow

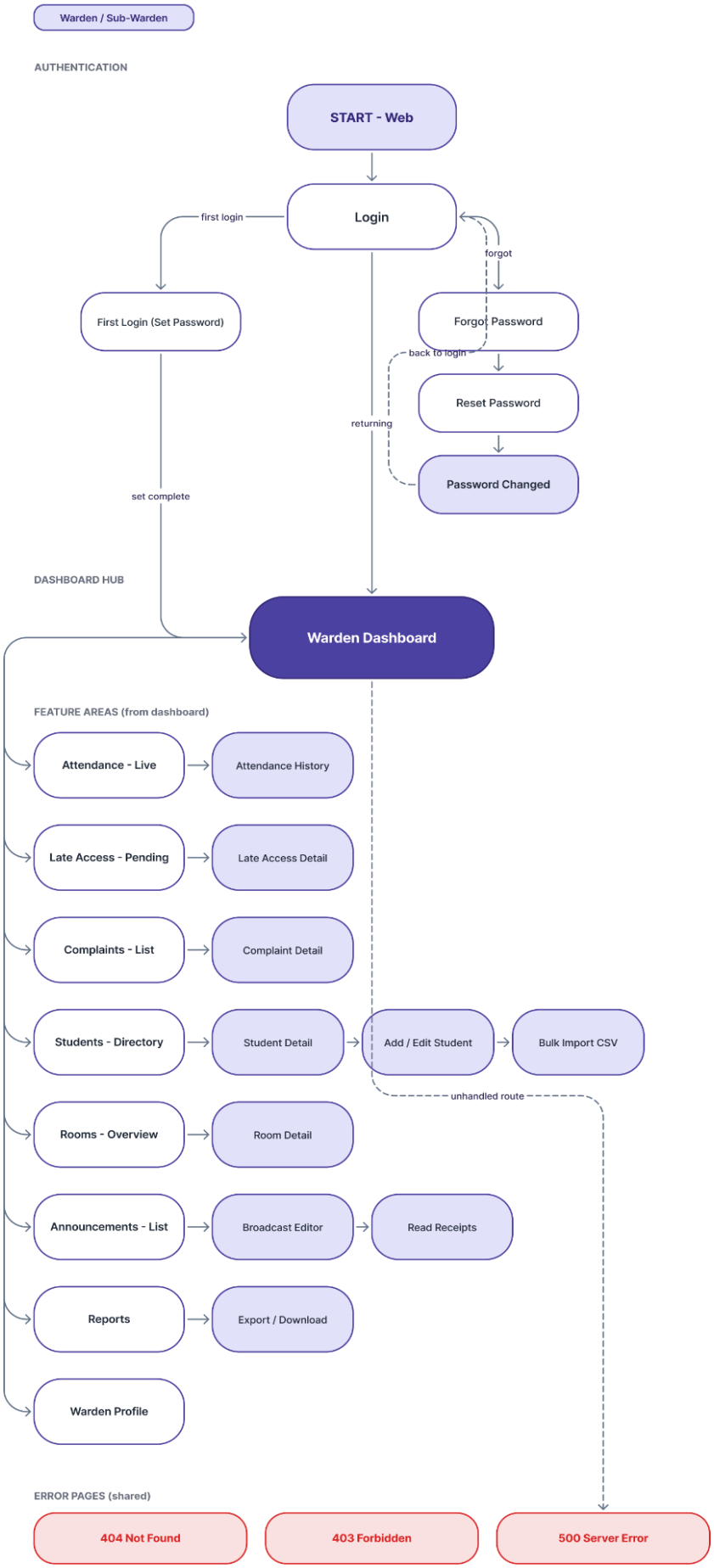
Student - Mobile App User Flow

React Native app. Bottom tab navigation: Home, Passes, Inbox, Me.



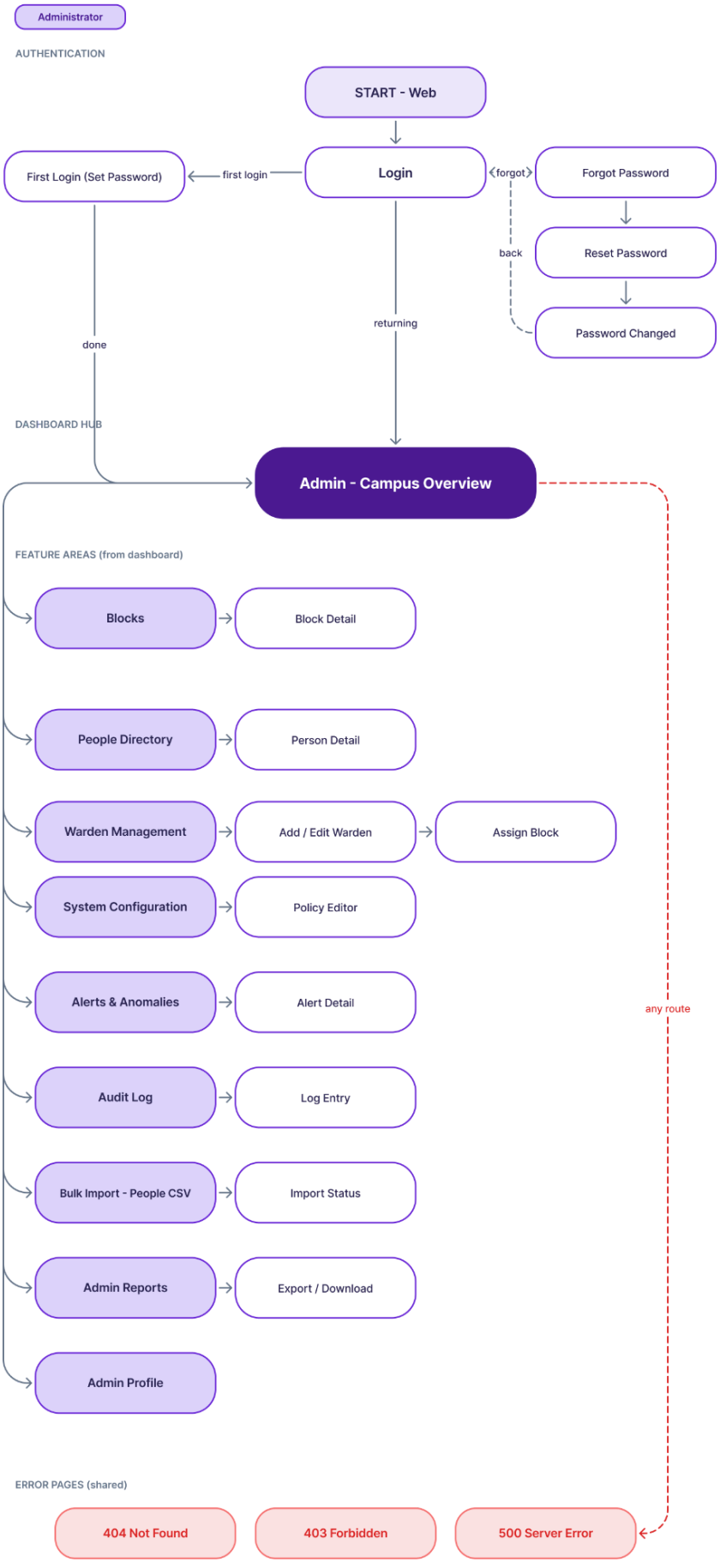
Warden / Sub-Warden - Web User Flow

Central dashboard. Eight feature chains: attendance, leave, complaints, students, rooms, announcements, reports, profile.



Administrator - Web User Flow

Campus-wide control. Manage blocks, people, wardens, system config, alerts, audit.



CHAPTER 6: SECURITY DESIGN

6.1 Authentication and Authorisation Flow

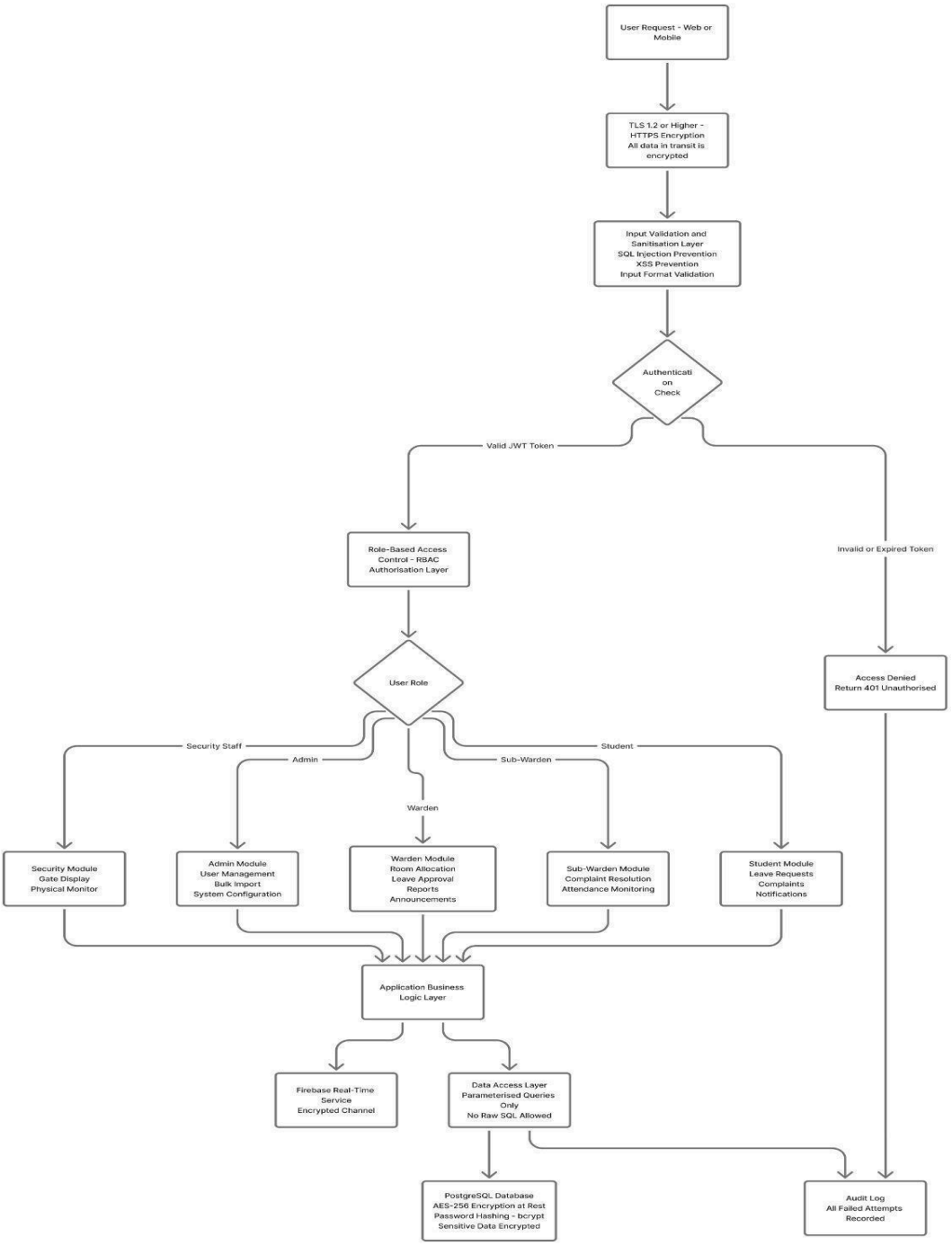


Figure 29: Authentication and Authorisation Flow

6.2 Full Security Components

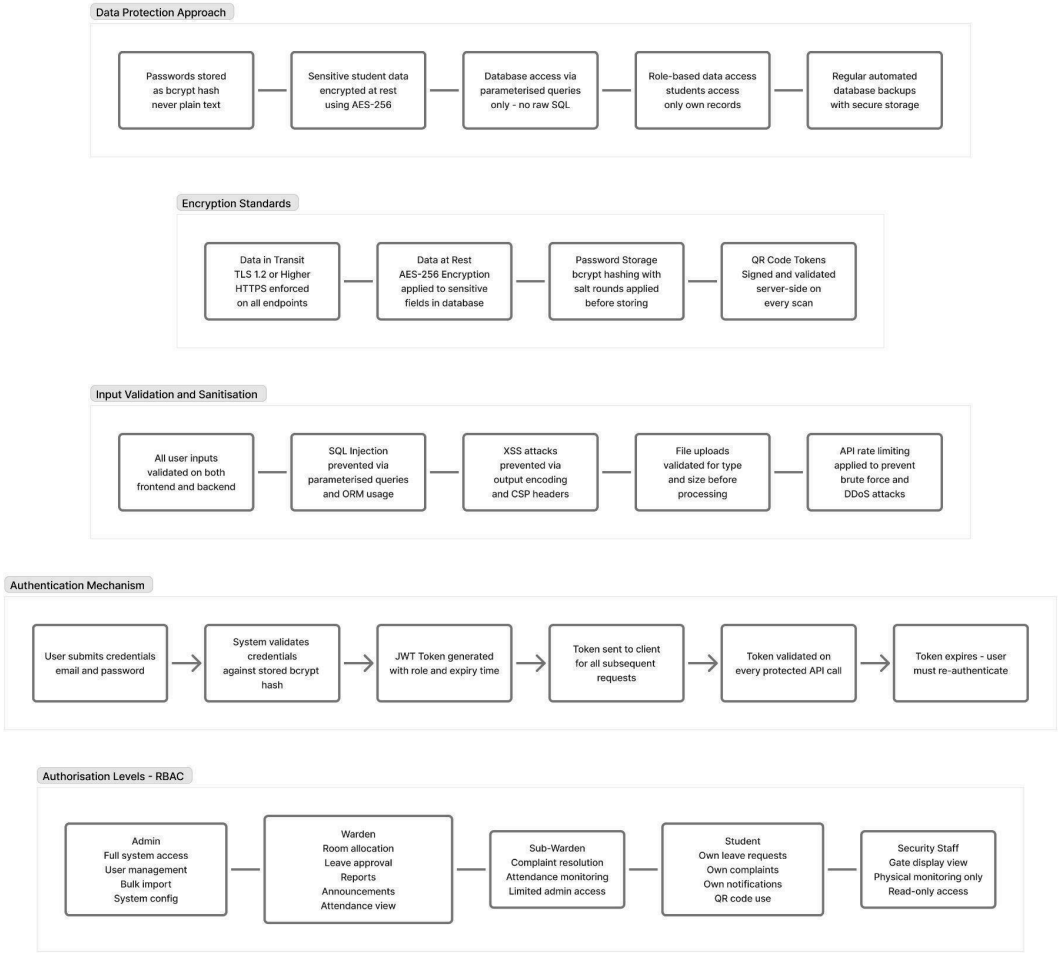


Figure 30: Full Security Components

6.3 Hardware and Session Security

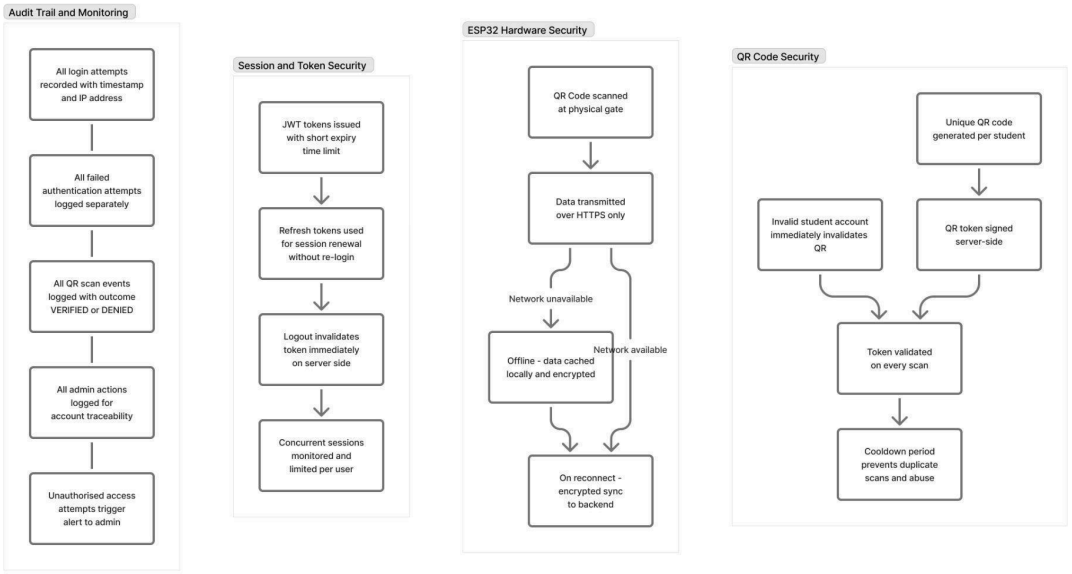


Figure 31: Hardware and Session Security

CHAPTER 7: DEPLOYMENT DESIGN

The deployment design of the HostelHive system defines the hosting environment, hardware specifications, network topology, environment separation, and continuous integration and delivery pipeline. All decisions are aligned with the Modular Monolithic architecture described in Chapter 1 and the constraints documented in Section 2.4 of the SRS. The deployment strategy prioritises simplicity, cost-effectiveness, and reliability appropriate to a single-client institutional deployment.

7.1 Hosting Plan

The HostelHive system is hosted on a Linux-based Virtual Private Server (VPS) provided by a commercial cloud infrastructure provider. All services are packaged as Docker containers and orchestrated using Docker Compose. This approach ensures environment parity across development, staging, and production, and avoids the operational complexity of a full Kubernetes cluster for a single-institution deployment. The Go backend, PostgreSQL database, and Redis cache are all containerised on the same VPS, while Firebase services (Authentication, Realtime Database, and FCM) are consumed as managed Google Cloud services to avoid the burden of maintaining a separate real-time messaging and authentication infrastructure.

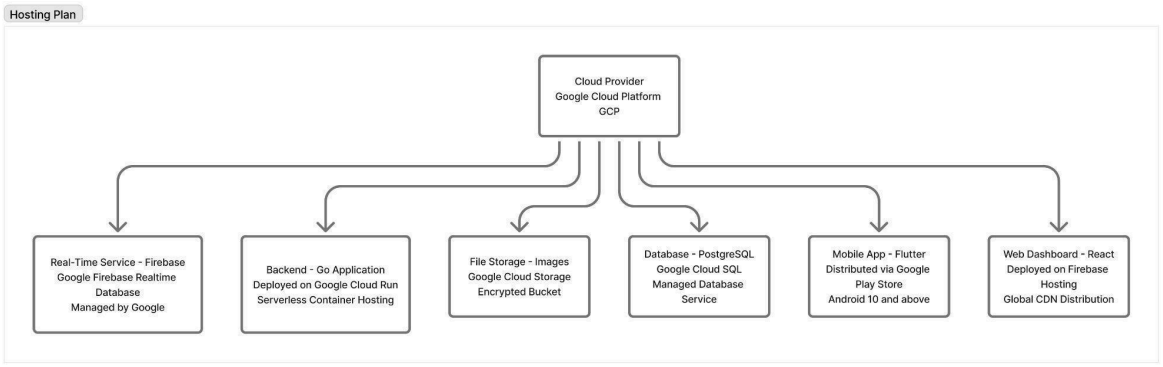


Figure 32: Hosting plan

Production VPS configuration:

- Provider: DigitalOcean Droplet (or equivalent, e.g., Linode, AWS Lightsail)
- Region: Singapore ap-southeast-1 (lowest latency for Sri Lanka)
- Operating system: Ubuntu 22.04 LTS
- Docker Engine: 24.x or later
- Docker Compose: 2.x

Component	Hosting Approach	Justification
API Gateway / Nginx	Docker container on VPS	TLS termination, reverse proxy, and centralised request routing to internal services
Go Backend	Docker container on VPS	Handles concurrent IoT attendance POST requests and REST/WebSocket API for client applications
PostgreSQL Database	Docker container with volume mount	Persistent relational data storage with nightly backup capability via volume snapshot
Redis Cache	Docker container on VPS	Session token caching and rate-limit enforcement for QR scan deduplication (FR014)

Firebase Realtime DB	Managed – Google Cloud	Real-time attendance state synchronisation pushed to warden dashboard without polling
Firebase Auth / FCM	Managed – Google Cloud	JWT-based authentication and push notification delivery to Flutter mobile app (FR032)
SSL/TLS Certificates	Let's Encrypt via Certbot	Zero-cost trusted certificates with automatic renewal; satisfies encryption-in-transit requirement

Table 9: Hosting Plan Summary

7.2 Hardware Requirements

The following hardware specifications apply to the production VPS hosting the HostelHive system. On-premise hardware is limited to the ESP32 QR scanner nodes installed at hostel entry points, student smartphones running the Flutter mobile application, and warden workstations accessing the React dashboard via a web browser.



Figure 33: Hardware Requirements

7.2.1 Server Hardware (VPS)

Component	Minimum Requirement	Recommended
CPU	2 x86-64 cores	4 cores
RAM	4 GB	8 GB
Storage	30 GB SSD	60 GB SSD
Network bandwidth	100 Mbps	1 Gbps
Public IPv4 address	1 (static)	1 (static)

Table 10: Production Server Minimum Specifications

Rationale: The 2-core / 4 GB configuration is sufficient to support up to 500 concurrent student connections and process real-time QR scan events with sub-500 ms response times. Storage accommodates Docker container images, PostgreSQL data volumes, and 12 months of structured JSON logs. The recommended 4-core / 8 GB configuration provides headroom for growth in student population and future feature additions.

7.2.2 On-Premises Hardware

Device	Qty	Interface	Protocol	Purpose
ESP32 Microcontroller (QR Scanner)	2	Wi-Fi 802.11b/g/n	HTTPS POST	Entry and exit QR code scanning; queues scans locally when offline (FR011, FR015)
Warden Workstation / PC	1–2	Ethernet / Wi-Fi	HTTPS / WSS	Browser-based React dashboard for real-time monitoring, leave approval, and room management
Security Staff Tablet / PC	1	Wi-Fi	HTTPS	Verification of student entry/exit and attendance status at the gate
Student Smartphones	N/A	Wi-Fi / Mobile data	HTTPS / FCM	Flutter app for QR display, leave submission, complaints, and announcement viewing
Hostel Wi-Fi Access Point	1+	IEEE 802.11	—	Provides connectivity for ESP32 nodes, staff devices, and warden workstations within the hostel

Table 11: On-Premises Hardware Requirements

ESP32 connectivity: Each ESP32 node must maintain a stable Wi-Fi connection to the hostel's internal network to transmit scan payloads to the backend API in real time. In the event of a Wi-Fi outage, scanned QR data is queued locally in the ESP32's SPIFFS flash storage and is automatically flushed to the PostgreSQL database upon network restoration (FR015). Each node must be assigned a static IP address on the hostel LAN and powered via a continuous supply through a wall adapter or UPS.

7.3 Network Diagram

The network architecture separates public-facing traffic from internal service communication. All external requests are terminated at the API Gateway (Nginx), which proxies requests to the Go backend over the Docker bridge network. Firebase services are accessed from the backend over secure HTTPS connections to Google Cloud. ESP32 nodes communicate with the API Gateway via HTTPS POST requests over the hostel Wi-Fi network.

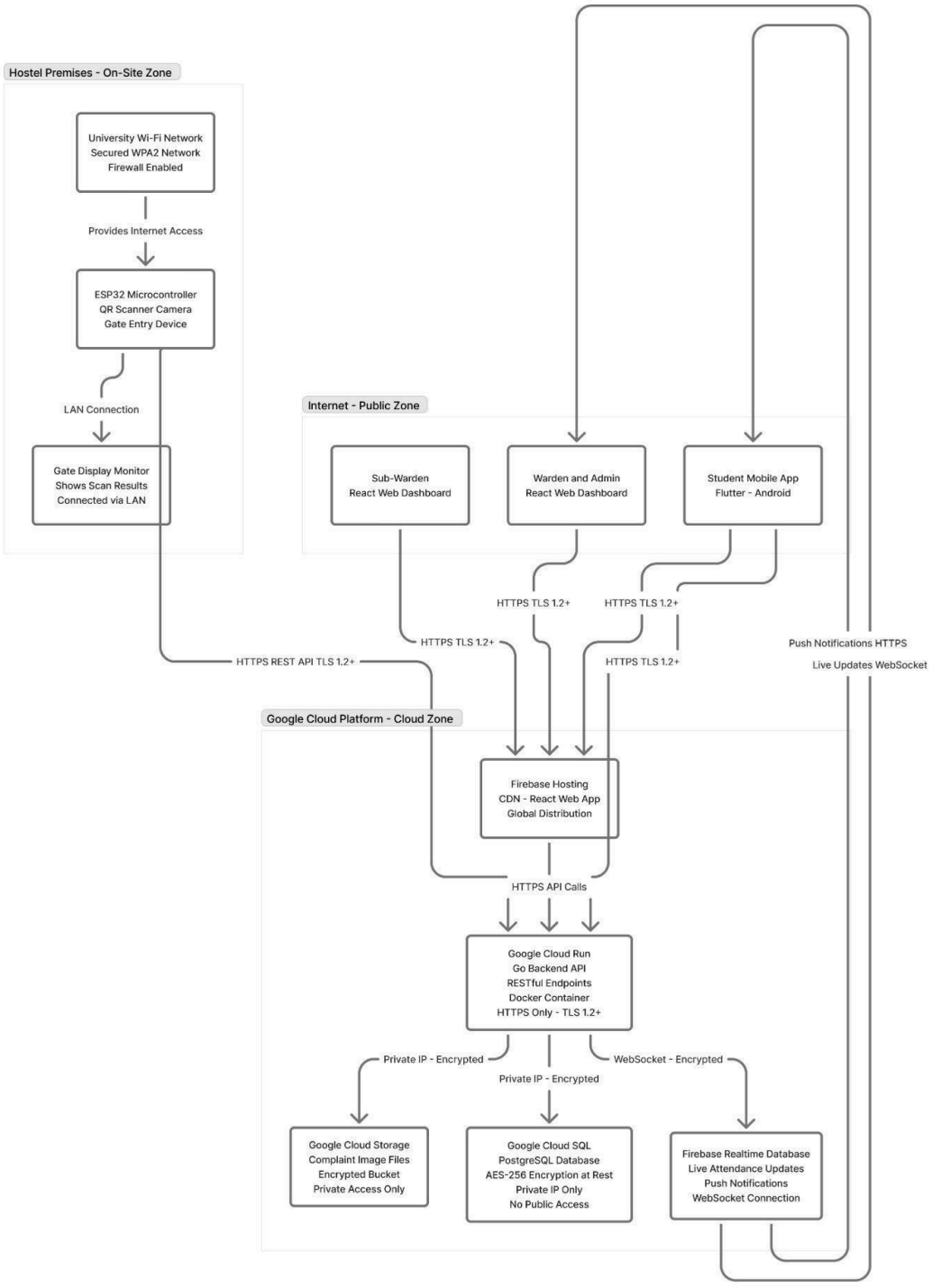


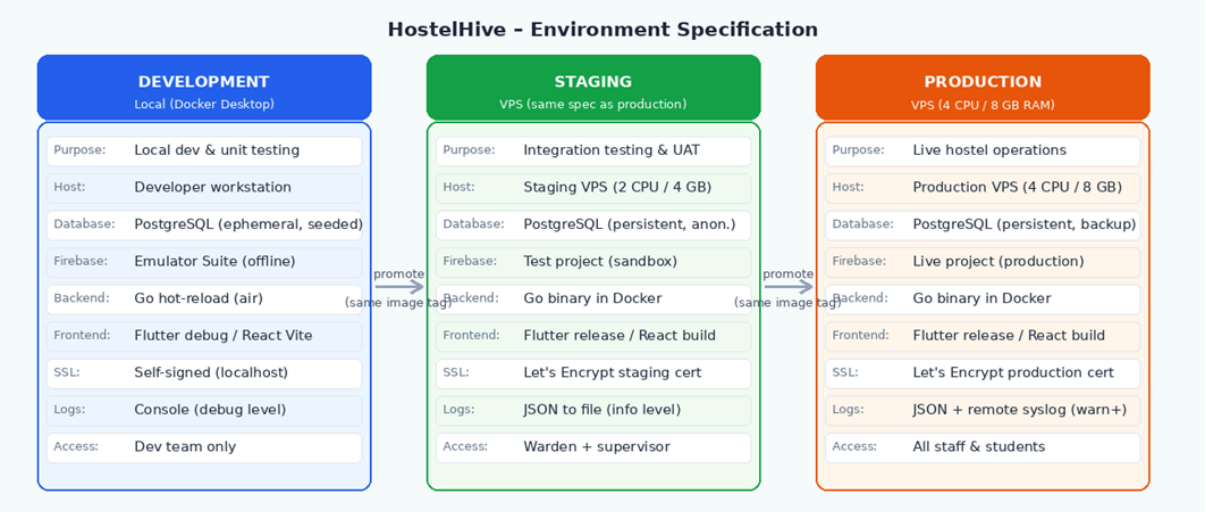
Figure 34: Network Diagram

Key network security controls:

- Only the API Gateway (Nginx) exposes port 443 (HTTPS/WSS) to the public internet. No other container port is exposed externally.
- All internal services (Go backend, PostgreSQL, Redis) communicate exclusively over a Docker bridge network with no public routes.
- The warden dashboard and security staff interface are accessible only from the hostel's internal Wi-Fi network. A network-level restriction via the hostel router is recommended in addition to RBAC enforcement in the application.
- ESP32 nodes are restricted to sending requests to a single HTTPS endpoint and carry a device-specific pre-shared API key in the request header for source verification.
- Firebase Authentication tokens are validated by the Go backend on every protected API request, preventing unauthenticated access to attendance or student data.

7.4 Environment Specification

Three distinct environments are maintained throughout the software development lifecycle: Development, Staging, and Production. Each environment runs the same Docker container images, ensuring full environment parity. Configuration differences are managed exclusively through environment variable files (.env) injected at container startup — no code-level changes are permitted between environments.



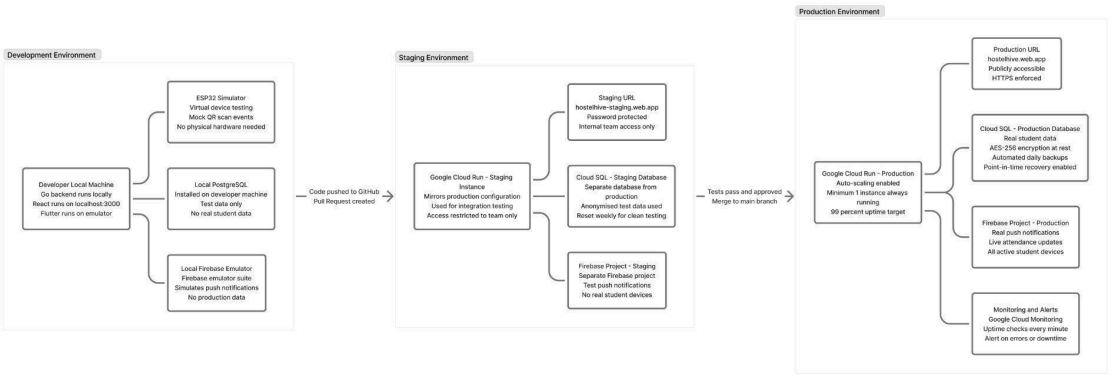


Figure 35: HostelHive Environment Specification Overview

Aspect	Development	Staging	Production
Purpose	Local development and unit testing	Integration testing, UAT, pre-production validation	Live hostel operations
Host	Developer workstation (Docker Desktop)	Staging VPS (same Docker spec)	Production VPS (4 CPU / 8 GB RAM)
Database	PostgreSQL container (ephemeral, seeded with test data)	PostgreSQL container (persistent, anonymised snapshot)	PostgreSQL container (persistent, nightly backup)

Firebase	Firebase Emulator Suite (fully offline)	Test Firebase project (sandbox)	Live Firebase project (production keys)
ESP32 Nodes	Simulated (Postman / curl mock)	Physical ESP32 on test network	Physical ESP32 nodes at hostel gates
SSL/TLS	Self-signed certificate (localhost)	Let's Encrypt staging certificate	Let's Encrypt production certificate
Logging	Console output (debug level)	JSON logs to file (info level)	JSON logs + remote syslog (warn/error)
Access	Development team only	Supervisor and warden team	All staff and students

Table 12: Environment Specifications

Environment promotion workflow: A container image that passes all automated tests and smoke checks in the staging environment is promoted unchanged to production using the same image SHA tag. No code modifications, configuration changes, or file replacements are permitted between the staging and production promotion. The only difference at promotion time is the injection of production environment variables (live Firebase keys, production PostgreSQL credentials, and the Let's Encrypt production certificate).

7.5 CI/CD Pipeline Overview

A Continuous Integration and Continuous Delivery (CI/CD) pipeline is implemented using GitHub Actions. The pipeline automates building, testing, and deployment of the HostelHive system across all environments. The goal is to ensure that every code change is validated automatically, that staging remains an accurate mirror of production, and that production deployments require explicit human approval. Because HostelHive is a Modular Monolith

rather than a microservices architecture, a single unified pipeline handles all service modules. Individual domain modules (identity, attendance, allocation, leave, complaint) may be independently tested but are always deployed as a single composed application image.

7.5.1 Pipeline Stages

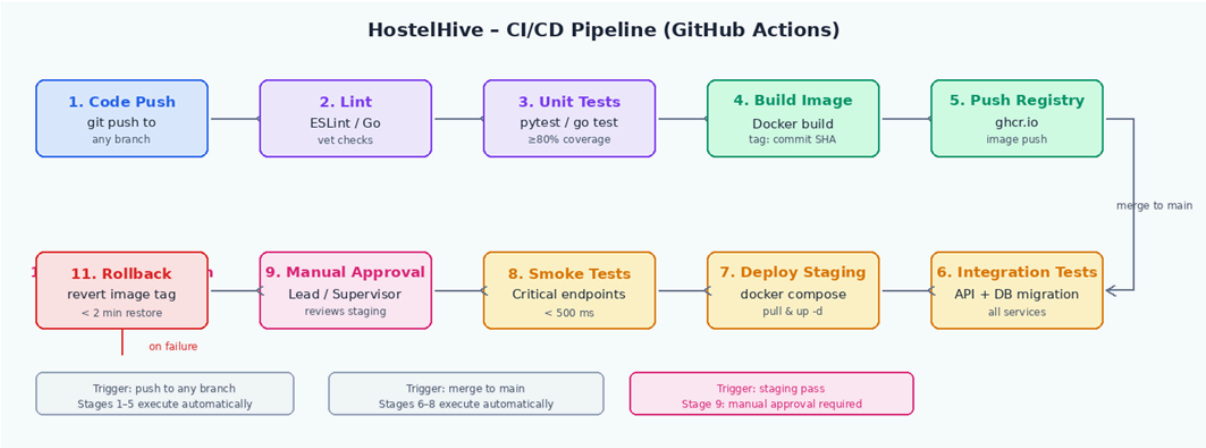


Figure 36: HostelHive CI/CD Pipeline Overview (GitHub Actions)

7.5.2 Pipeline Implementation Details

Stage	Trigger	Actions	Success Criteria
1. Lint	Push to any branch	go vet, staticcheck on Go source; ESLint on React codebase	Zero errors; zero warnings in strict mode
2. Unit Tests	Push to any branch	go test for backend; pytest for any utility scripts; Flutter unit tests	`100% pass rate; coverage ≥ 80%`

3. Build Image	Push to main or release branch	docker build using multi-stage Dockerfile; tag with commit SHA and latest	Image builds successfully; size < 300 MB
4. Push Registry	After successful build	Push to GitHub Container Registry (ghcr.io)	Image layers pushed without authentication errors
5. Integration Tests	After merge to main	API contract tests; PostgreSQL migration tests; Firebase emulator connectivity tests	All tests pass; no regression detected
6. Deploy Staging	After integration tests pass	SSH into staging VPS; docker compose pull && docker compose up -d	All containers healthy; health checks pass
7. Smoke Tests	After staging deployment	Verify critical endpoints: login, QR scan, leave submission, attendance count	Response time < 500 ms; expected HTTP status codes
8. Manual Approval	Staging passes smoke tests	Project supervisor or lead developer reviews staging environment	Explicit approval via GitHub Actions workflow

9. Deploy Production	Manual approval granted	Pull new image on production VPS; docker compose up -d with zero-downtime swap	Zero downtime; health checks pass; no session disruption
10. Rollback	On deployment failure	Revert to previous image tag; docker compose up -d with prior SHA	System restored to last known good state within 2 minutes

Table 13: CI/CD Pipeline Specifications

7.5.3 IoT Deployment Consideration

Because the ESP32 firmware is a separate C++ codebase compiled with PlatformIO, it is maintained in a dedicated directory within the same repository. A separate GitHub Actions workflow is triggered on changes to the firmware directory and performs the following steps:

- Compile the firmware using PlatformIO CI action and verify that the binary size does not exceed the ESP32's flash partition limits.
- Run unit tests for the QR parsing and HTTP transmission logic using a simulated serial environment.
- On approval, the compiled firmware binary is attached as a GitHub Release asset. Physical flashing of ESP32 nodes is performed manually by the system administrator or a developer using PlatformIO OTA or USB flash, as the hostel network does not support remote OTA firmware delivery in the current project scope.

This approach ensures that firmware changes are validated before physical deployment, preventing ESP32 nodes from entering an inoperative state that would block student entry or exit (FR011). The firmware pipeline is decoupled from the backend pipeline so that backend deployments are never blocked by pending firmware changes, and vice versa.